

ShiftSwap: A Shift Schedule and Substitute Management System for Small Part-Time Workplaces

소규모 매장 아르바이트 근무표·대타 관리 시스템
Team 7

Name	Student ID	Affiliation	E-mail
Hyewon Kim (김혜원)	24102028	Dept. of Industrial Engineering, ITM Program, SeoulTech	(to be provided by member)
Hyoungdo Kim (김형도)	18102075	Dept. of Industrial Engineering, ITM Program, SeoulTech	zloganway@gmail.com
Minkyung Kim (김민경)	23102004	Dept. of Industrial Engineering, ITM Program, SeoulTech	(to be provided by member)

Submission date	2026-12-04 (Week 14)
Deployed application	https://keemlogan.github.io/shiftswap/app/
Source repository	https://github.com/keemlogan/shiftswap
Submitted to	jihjim@gmail.com

Table of Contents

Roles of Team Members

Hyewon Kim — requirements lead

Hyoungdo Kim — project manager, design and implementation lead

Minkyung Kim — UI and quality lead

Abstract

1. Introduction

1.1 Motivation

1.2 Objectives

1.3 Scope

1.4 Development Approach

1.5 Organisation of the Report

2. Problem Statement, Background and Related Work

2.1 Current Process in the Reference Store

2.2 Problems of the Current Process

2.3 Legal Background

2.4 Related Work

2.5 The Gap ShiftSwap Fills

3. Requirements Specification

3.1 Requirements Elicitation

3.2 Stakeholders and Actors

3.3 Use-Case Model

3.4 Functional Requirements

3.5 Use-Case Descriptions

3.6 Business Rules

3.7 Non-Functional Requirements

3.8 Scenarios

3.9 Analysis Models

3.10 Requirements Validation

3.11 Requirements Management

4. Design Specification

4.1 Architecture

4.2 Deployment

4.3 Design Decisions

4.4 Class Design

4.5 Database Design

4.6 Dynamic Design: Sequence Diagrams

4.7 State Design

4.8 Algorithm Design

4.9 User-Interface Design

5. Implementation by Vibe Coding

5.1 Development Environment and Tools

5.2 The Vibe-Coding Process

5.3 Structure of the Final Prompt

5.4 Iterations of the Prompt

5.5 From Prompt to Code

5.6 Reproducibility Check

6. Testing and Validation

6.1 Test Strategy

6.2 Test Environment

6.3 Test Cases

6.4 Test Results

6.5 Boundary-Value Analysis

- 6.6 Concurrency Test
- 6.7 Verification of Non-Functional Requirements
- 6.8 Traceability Matrix
- 6.9 Defects and Specification Gaps Resolved During the Iterations

7. Deployment and User Guide

- 7.1 Deployment
- 7.2 User Guide

8. Conclusion

- 8.1 Achievement of the Objectives
- 8.2 Size and Complexity of the System
- 8.3 Limitations
- 8.4 Future Work
- 8.5 Lessons Learned

References

- Appendix A. Final Vibe-Coding Prompt (v5)
- Appendix B. Prompt Log

Roles of Team Members

Team 7 has three members. The topic was proposed by Hyewon Kim, who works part-time in a small café and has seen at first hand how substitute shifts are arranged there. In Week 4 the team compared seven candidate topics and on 2026-09-27 selected ShiftSwap because it is a conventional transaction-processing application with a clear owner–worker workflow, more than five core use cases and a relational data model of twelve tables, which satisfies the minimum scope set by the term project description (five use cases, four tables).

Work was divided along the work breakdown structure (WBS) of the proposal, which has eight top-level packages that follow the course deliverables and the life-cycle phases: 1 Project management, 2 Proposal, 3 Requirements (interim report), 4 Design, 5 Implementation by vibe coding, 6 Testing, 7 Final report, and 8 Presentation. Each work package has one owner who is accountable for its deliverables. Every deliverable was also reviewed by a member who did not write it: requirements by Hyoungdo Kim and Minkyung Kim, design models by Hyewon Kim and Minkyung Kim, and test cases by Hyewon Kim and Hyoungdo Kim. The team met once a week, and the Gantt chart was updated after each meeting. The table below lists the responsibilities; the paragraphs after it describe each member's work in more detail.

Table 1. Responsibilities of team members mapped to WBS work packages and deliverables

Member	Role	WBS work packages owned	Main deliverables
Hyewon Kim	Requirements lead	2.1 Topic selection; 2.2.1 Problem statement and scope; 3.1 Elicitation; 3.2.1 FR and NFR tables, scenarios; 7.1.1 Requirements and design chapters	Comparison of seven candidate topics; problem statement P1–P3 and as-is process (Chapter 2); interview plan and notes; actors and use-case list; FR-01...FR-21; NFR-01...NFR-13; scenarios S-1...S-7; use-case descriptions UC-01...UC-13; requirements and design chapters of this report
Hyoungdo Kim	Project manager; design and implementation lead	1 Project management (1.1, 1.2); 2.2.2 Reviewed proposal PDF; 3.2.2 Business rules and data model draft; 4.1 UML models; 5 Implementation by vibe coding (5.1, 5.2); 7.2.1 Traceability matrix; 8.2.2 Live demo	WBS, Gantt chart, weekly milestone tracking and meeting notes; canonical specification file <code>spec/spec.md</code> ; business rules BR-01...BR-12 and data model; use-case, class, ER, sequence, state and activity diagrams; final vibe-coding prompt and prompt log; increments 1 and 2; public URL on GitHub Pages; traceability matrix; live demonstration
Minkyung Kim	UI and quality lead	4.2 UI design; 6 Testing (6.1, 6.2); 7.1.2 Testing and results chapters; 7.2.2 Final report PDF; 8.1 Slide deck; 8.2.1 Rehearsed talk	Screen wireframes, prototype screens and visual design rules; test cases TC-01x...TC-12x and passing test run; scenario walkthrough record and screenshots; cross-browser and 360/1440 px checks; testing chapters; integrated final report PDF (50–150 pages); PowerPoint deck

Hyewon Kim — requirements lead

Hyewon Kim owned the problem domain. The topic came from her own part-time job, and her workplace is the reference store of the project; she prepared the comparison of seven candidate topics that led to its selection on 2026-09-27 (WBS 2.1) and wrote the problem statement and scope of the proposal (2.2.1). She described the current substitute procedure step by step and supplied the figure of three to five one-to-one contacts per substitute request that appears as problem P1. She collected the legal material on the 2026 minimum wage (KRW 10,320), the weekly holiday allowance (주휴수당) under the Labor Standards Act, the five-employee threshold for premium pay and the probation reduction of the Minimum Wage Act. For elicitation (3.1) she prepared the interview plan for the owner and two co-workers (Section 3.1) and the list of actors and use cases; for the specification (3.2.1) she wrote the functional and non-functional requirements, the scenarios of Section 3.8 and the use-case descriptions of Section 3.5. She wrote the requirements and design chapters of this report (7.1.1). In the presentation she covers the problem and the requirements.

Hyoungdo Kim — project manager, design and implementation lead

Hyoungdo Kim was responsible for project management (WBS 1): the WBS and the Gantt chart, weekly milestone tracking, the weekly meeting notes and the task list with owners. He reviewed and submitted the proposal (2.2.2). He turned the legal material into the business rules and drafted the data model (3.2.2), and he created the specification file `spec/spec.md`, which fixes every identifier (UC, FR, NFR, BR, TC), every state name and the seed data, so that the application, the UML

models and the reports use the same names and numbers. He drew the UML models (4.1): use-case, class and ER diagrams, and sequence, state and activity diagrams. For implementation by vibe coding (5) he wrote the final prompt and kept the prompt log of iterations against the specification (5.1), produced increments 1 and 2 with the AI coding assistant, and deployed the application on GitHub Pages (5.2). He built the traceability matrix of the final report (7.2.1). In the presentation he covers the design and the implementation by vibe coding and gives the live demonstration of the deployed application (8.2.2).

Minkyung Kim — UI and quality lead

Minkyung Kim owned the user-interface design (WBS 4.2): the screen wireframes and prototype screens (4.2.1) for sign-in, weekly board, inbox, approvals, attendance, weekly summary, payroll and the owner and worker forms, and the visual design rules (4.2.2) that were later written into the final prompt. She owned testing (6): she wrote the unit test cases TC-01x to TC-12x from the business rules and ran them (6.1), and she carried out the scenario walkthroughs with screenshots and the cross-browser checks at 360 px and 1440 px screen width required by NFR-02 (6.2). She wrote the testing and results chapters (7.1.2), integrated the chapters of the three members into the final report PDF and checked the course format rules — A4, 12 pt, single spacing, 1-inch margins, 50–150 pages (7.2.2). She prepared the slide outline and the PowerPoint deck (8.1) and rehearsed the 15-minute talk (8.2.1). In the presentation she covers testing, deployment and the conclusion.

Abstract

In small stores such as cafés and convenience stores, part-time workers have fixed weekly shifts, and when one of them cannot come the owner must find a substitute by contacting other workers one at a time, edit a spreadsheet by hand, and later recheck each worker's weekly hours to decide whether the weekly holiday allowance (주휴수당) is still owed. In the reference store of this project, one substitute request typically costs the owner three to five separate contacts, and legal pay rules — the 2026 minimum wage of KRW 10,320, the 15-hour threshold for the holiday allowance, the five-employee threshold for premium pay, and the probation reduction — are applied from memory. This report describes ShiftSwap, a web application built to remove these three problems. A worker requests a substitute for one of their own shifts; the system selects eligible candidates from registered availability and existing shifts, notifies all of them at once, and accepts only the first acceptance. The owner then approves the swap on a screen that shows the weekly contractual hours, scheduled hours and holiday-allowance eligibility of both workers before and after the change.

The system was developed with an incremental life-cycle model combined with UI prototyping, and every increment was implemented by vibe coding, that is, by generating code from natural-language prompts with an AI coding assistant. The report covers the problem statement and related work, a requirements specification of 13 use cases, 21 functional requirements, 13 non-functional requirements and 12 business rules, a design specification with UML 2.5 use-case, class, sequence, state and activity diagrams and a twelve-table SQLite schema, the final prompt that reproduces the system, the testing approach based on test cases derived from the business rules, and a user guide. The application runs entirely in the browser (HTML, CSS and JavaScript with SQLite compiled to WebAssembly) and is deployed at <https://keemlogan.github.io/shiftswap/app/>.

1. Introduction

This chapter explains why the team built ShiftSwap, what the project set out to achieve, which features are inside and outside the scope of the delivered system, and how the development process was organised. The last section outlines the rest of the report.

1.1 Motivation

Small stores in Korea — cafés, convenience stores, bakeries and small restaurants — depend on part-time workers (아르바이트). A typical store of this kind employs fewer than ten such workers, and each of them has a fixed weekly pattern of shifts, for example every Monday and Wednesday from 18:00 to 23:00. The pattern is agreed in the employment contract and changes rarely. What changes often is attendance on a particular day: a worker has an examination, falls ill or has a family event, and asks for someone else to cover the shift. In the café where one team member works, such requests are a routine part of running the store rather than an exception.

The owner, who in a store of this size is also the manager, handles each such case personally. The owner phones or messages the other workers one by one until someone agrees, updates the schedule spreadsheet, and tells the worker who asked. The work does not end there. Korean labour law ties part of a part-time worker's pay to the number of hours worked each week: a worker whose weekly contractual hours are 15 or more and who attended every scheduled working day is entitled to a paid weekly holiday, the weekly holiday allowance (주휴수당) [3], [5]. A swap therefore changes the hours and possibly the entitlement of two people at once, and the owner has to recheck both at the end of the month. In the reference store these checks are made from memory with a spreadsheet; the legal thresholds are not written down anywhere in the store.

The team chose this problem for three reasons. First, one member knows the process from the inside, so requirements could be drawn from a real workplace rather than from assumptions. Second, the problem is a transaction-processing problem of the kind the course recommends: it has several actors, a clear life cycle for each substitute request, and a relational data model. Third, the core difficulty is not the user interface but the rules — who is eligible, who wins when two people accept at the same moment, and how weekly and monthly pay change after a swap — which makes the requirements and design phases substantial and gives the testing phase concrete targets.

1.2 Objectives

The overall goal of the project is to deliver a working web application that replaces the telephone-and-spreadsheet process of a small store for three tasks: finding a substitute, keeping the schedule correct, and computing the weekly holiday allowance and monthly pay. The proposal broke this goal down into six measurable objectives, G1 to G6, which are kept unchanged in this report so that they can be checked at the end of the project (Chapter 8). Four of them are tied to the problems P1–P3 defined in Chapter 2; the other two follow from the course requirements.

Table 2. Project objectives, linked problems and measures of achievement

ID	Objective	Measure of achievement	Problem
G1	Remove the owner's one-to-one contacts when a substitute is needed.	The owner makes 0 contacts per request (reference store before the system: 3–5). All eligible workers are notified in the same transaction that creates the request (FR-07). The owner's only action is one approve or reject decision.	P1
G2	Let a candidate answer quickly.	A candidate accepts a request in at most 2 clicks from the inbox (NFR-01). Only the first acceptance can succeed (BR-02, NFR-04).	P1
G3	Keep the schedule and weekly hours of both workers correct after every swap.	On approval the shift is reassigned and the weekly summaries of both workers are recomputed with no manual step (FR-12). The approval screen shows both workers' hours and eligibility before and after the swap (FR-11); for the seed example it shows Choi Minho's scheduled hours changing from 16 to 21 with eligibility unchanged.	P2
G4	Apply the legal pay rules automatically and flag violations before payday.	Business rules BR-06 to BR-11 are implemented as tested functions, and every business rule has at least one unit test (spec §12), including the 15-hour boundary (14.99 h and 15 h) and the 40-hour cap. The payroll screen warns when a wage is below the applicable minimum and blocks confirmation until the warning is acknowledged (FR-16).	P3
G5	Deliver a system of the size the course requires.	At least 5 core use cases (delivered: 13, of which 9 are core) and at least 4 database tables (delivered: 12).	—

ID	Objective	Measure of achievement	Problem
G6	Deliver on time and reproducibly.	Working system deployed at a public URL by Week 14 (2026-12-04, NFR-07). Re-running the final prompt in an empty folder produces a system that passes the same unit tests.	—

G6 also reflects a practical need of the target users: a small store has no server and no IT staff, so the system must run in a browser without installation (NFR-06), and it must be possible to rebuild it from its documentation alone.

1.3 Scope

The team classified features with the MoSCoW scheme (Must, Should, Could, Won't). Use-case priorities are the ones recorded in spec §3. Nine of the thirteen use cases correspond one to one to the nine functions announced in the proposal (UC-01 to UC-09); UC-10 to UC-13 are supporting use cases that make settings, expiry, cancellation and role selection explicit, because the core use cases are incomplete without them.

Table 3. Scope of the delivered system

Priority	Included features
Must	UC-01 Register worker and fixed schedule; UC-02 Register availability; UC-03 Generate weekly schedule; UC-04 Request substitute (with automatic candidate selection, BR-01); UC-05 Respond to substitute request; UC-06 Approve or reject substitution; UC-08 Compute weekly hours and holiday allowance; UC-11 Expire overdue substitute request; UC-13 Sign in by role; in-app notification inbox (FR-21).
Should	UC-07 Record and confirm attendance; UC-09 Generate and confirm monthly payroll (base pay, holiday allowance, premium pay, minimum-wage warning, probation reduction); UC-10 Manage workplace settings and minimum wage table; UC-12 Cancel own substitute request; Korean/English language switch; adjustable demo clock.
Won't (this version)	Password authentication and user accounts on a server; push notifications, KakaoTalk or SMS messaging (notifications are in-app only); bank transfer of wages; GPS or QR-code clock-in (excluded on purpose by NFR-12); multiple stores per owner; automatic optimisation of the weekly schedule; payslip export, tax and social-insurance deductions; annual leave, severance pay and the four-week averaging of the 15-hour rule beyond the weekly contractual value (see BR-06).

Two scope decisions need comment. The system runs entirely in the browser with an embedded SQLite database, and sign-in is a choice among demo accounts (UC-13). This keeps the deployed system free of a server, but it also means that data stays in one browser and that the prototype is a demonstration system, not a multi-user production service. The second decision concerns payroll. The payroll function stops at gross pay: base pay, holiday allowance and premium pay. Deductions such as income tax and national insurance depend on rules that change every year and are outside the three problems the project addresses.

1.4 Development Approach

1.4.1 Choice of life-cycle model

Chapter 1 of the lecture presents the linear sequential (waterfall) model, the prototyping model, the incremental model, Boehm's spiral model, the Rational Unified Process and agile methods [6]. The team compared them against four facts about the project: the course fixes three document deadlines (proposal in Week 5, interim report in Week 7, final report in Week 14); the requirements for the substitute workflow were known well from the reference store, but the screens were not; code is produced by an AI coding assistant from prompts, so a working version can be generated in hours rather than weeks; and the team has three members with no dedicated customer on site.

Table 4. Life-cycle models considered

Model	Fit with this project	Decision
Waterfall	Matches the fixed course documents, but a phase must be completed before the next begins, and feedback on screens would arrive only after implementation.	Rejected as the main model; its document sequence is kept for the course deliverables.

Model	Fit with this project	Decision
Prototyping	Suited to the unknown part of the requirements (screens, approval comparison panel). The lecture warns that prototypes omit error checking and non-functional requirements.	Adopted for the user interface only.
Incremental	Delivers a minimal useful subset first and adds increments; reduces rework when requirements change; allows demonstration at each increment.	Adopted as the main model.
Spiral	Explicit risk analysis per loop; heavy for a three-person, fourteen-week project.	Not adopted; the main risks are tracked in the WBS instead.
RUP	Iterative phases with UML; its good practices (manage requirements, visual modelling) were adopted, but the four-phase structure adds little for a small system.	Practices adopted, process not.
Agile / XP	Test-first development and small releases fit vibe coding well; the course, however, requires full requirement and design documents.	Test-first practice adopted within increments.

The resulting approach is an **incremental model with UI prototyping**. The incremental model addresses the main weakness the lecture identifies in the waterfall model, the difficulty of accommodating change after the process is under way. The prototype addresses the part of the requirements that only becomes clear when users can see a screen. The known weaknesses of the incremental model are that the process is less visible and that structure degrades as increments are added [6]. The team countered the first with the fixed course documents and the prompt log, and the second with a layered architecture that was fixed before the first increment (Chapter 4) and with the rule that domain rules live only in one module of pure functions.

1.4.2 Increments

Following the proposal, the requirements and analysis models were written once for the whole system (WBS 3 Requirements and 4.1 UML models), and implementation was then delivered in two increments (WBS 5.2.1), followed by deployment (5.2.2), testing (6) and the freezing of the final prompt kept under prompt engineering (5.1). The table below shows the content of each increment.

Table 5. Increments of the ShiftSwap development

Increment	Content	Use cases	Planned week and exit criterion
Increment 1 (Must)	Database schema and seed data, demo clock, sign-in, worker and schedule registration, availability, weekly board, the complete substitute workflow (request, candidate notification, first acceptance, approval with before/after comparison, expiry), weekly holiday allowance, inbox.	UC-01, UC-02, UC-03, UC-04, UC-05, UC-06, UC-08, UC-11, UC-13	End of Week 10, 2026-11-06 (WBS 5.2.1). Seed request with two candidates reproduced; worked example (Choi Minho, 3.2 h, KRW 33,024) shown on the approval screen; test groups TC-01x, TC-02x, TC-05x, TC-06x, TC-08x, TC-12x pass.
Increment 2 (Should)	Attendance recording and confirmation, monthly payroll with premium pay, minimum-wage warning and probation reduction, workplace settings and minimum-wage table, cancellation of an own request.	UC-07, UC-09, UC-10, UC-12	End of Week 11, 2026-11-13 (WBS 5.2.1). Test groups TC-09x, TC-10x, TC-11x pass together with all groups of Increment 1 (regression).

Increment	Content	Use cases	Planned week and exit criterion
Deployment and testing	Deployment to GitHub Pages, test runs, scenario walkthroughs, cross-browser and responsive checks, reproduction run of the final prompt in an empty folder.	All	Weeks 12–14 (WBS 5.2.2, 6.1.2, 6.2). Deployed URL and all test groups run in Week 12; NFR-02 and NFR-06 checks and final prompt frozen in Week 13; final report, slides and live demonstration on 2026-12-04 (Week 14).

Increment 1 is the "minimal subset of the whole system that is still useful" in the sense of the lecture [6]: with it, a store can already find substitutes, approve swaps and see weekly holiday-allowance eligibility. If Increment 2 had slipped, the core of the project would still have been delivered. Placing all Must use cases in the first increment was the main mitigation of the scope-creep risk identified in the proposal.

1.4.3 UI prototyping

The prototyping model of the lecture runs requirement analysis, prototype development, prototype evaluation, and then implementation [6]. In this project the prototype was not a separate throwaway artefact. The first version of each screen was generated by the AI coding assistant from a short description, evaluated by the team against the scenarios of Section 3.8, and then either kept or regenerated. Three screens went through this loop more than once: the weekly board (how to show a handed-over shift), the inbox (how to reach the accept button in two clicks), and the approval comparison panel (which figures to show for the two workers). The decisions from these loops were written back into the specification as visual design rules, which is why the final prompt contains exact colours, spacing and a single decorative element (Chapter 5). The lecture notes that prototypes tend to skip error handling and non-functional requirements; to avoid this, the error messages and the non-functional requirements NFR-01 to NFR-06 were added to the prompt from the first increment onward rather than left for the end.

1.4.4 How vibe coding fits each increment

Vibe coding means that source code is produced by an AI coding assistant from natural-language prompts, and that the developer's work moves from typing code to specifying, reviewing and testing it. The team used the same cycle inside each increment:

1. **Specify.** The identifiers, rules, states and seed data needed by the increment are added to `spec/spec.md` first. Nothing is allowed into a prompt that is not in the specification.
2. **Prompt.** A prompt for the increment is written that refers to the specification by ID (for example, "implement BR-02 inside one transaction that re-checks the request state").
3. **Generate.** The AI coding assistant generates or modifies the code.
4. **Test.** The unit tests named after the spec test IDs are run with `node --test`, and the scenarios of Section 3.8 are walked through in the browser.
5. **Review and refine.** Failures and unwanted behaviour are traced either to an ambiguous prompt or to a gap in the specification. The prompt or the specification is corrected, never the generated code by hand alone, and the change is recorded in the prompt log.

This cycle keeps the course deliverables and the code consistent: the final prompt is not written after the fact but is the accumulated result of the refinements made in all four increments. Test-first practice from extreme programming fits this cycle naturally, because a test named TC-081 holiday allowance at exactly 15 h is itself a precise prompt. Chapter 5 presents the final prompt and the prompt log in detail.

1.5 Organisation of the Report

Chapter 2 describes the current substitute process in the reference store, the three problems it causes, the legal background, and existing shift-scheduling products. Chapter 3 is the requirements specification: actors, use cases with full descriptions, functional and non-functional requirements, business rules, scenarios, analysis models and the validation of the requirements. Chapter 4 is the design specification with the architecture, class diagram, sequence diagrams, state diagrams and database design. Chapter 5 covers implementation by vibe coding, including the final prompt and the prompt log. Chapter 6 presents testing and validation and the traceability matrix. Chapter 7 explains deployment and gives a user guide for owners and workers. Chapter 8 concludes the report and lists limitations and future work. The references follow Chapter 8; the source code is submitted separately as an attached file.

2. Problem Statement, Background and Related Work

This chapter describes the problem that ShiftSwap addresses. Section 2.1 describes how substitutes are arranged today in the reference store and models the process as a data flow diagram. Section 2.2 states the three problems P1–P3 that the process causes. Section 2.3 summarises the legal rules that govern part-time pay in Korea and that the system must apply. Section 2.4 compares existing shift-scheduling products, and Section 2.5 identifies the gap that ShiftSwap fills.

2.1 Current Process in the Reference Store

2.1.1 The reference store

The requirements were drawn from one real workplace, a small café where Hyewon Kim works part-time. To protect the store and its staff, the report does not name the store; the demonstration data of the system uses a fictitious café, Dalbit Café (달빛카페), with one owner and four part-time workers, which has the same structure. The real store has the following characteristics, which are typical of the stores the system targets:

- fewer than ten part-time workers; whether the store reaches five regular employees, the threshold for premium pay (Section 2.3.3), depends on how many workers it employs at a given time, so the system treats the number as a setting;
- the owner acts as the manager, sets the schedule and pays the wages; there is no separate HR or payroll staff;
- each worker has a fixed weekly pattern of shifts written in the employment contract, and the pattern changes rarely;
- the schedule is kept in a spreadsheet maintained by the owner, and messages about substitutes are sent by phone call or personal message;
- wages are paid monthly, and the owner calculates them at the end of the month from the spreadsheet.

2.1.2 The as-is substitute process, step by step

The following description is the current procedure for one substitute request, as reported by the team member who works in the reference store. It is written as the normal sequence; the variations are listed afterwards.

1. **The worker reports the absence.** A worker who cannot attend a coming shift sends the owner a personal message with the date and the reason, for example "I cannot come on Wednesday evening because of a midterm exam."
2. **The owner looks up possible substitutes.** The owner opens the schedule spreadsheet, checks who is not working at that time, and recalls from memory who is usually free on that weekday. Availability is not recorded anywhere.
3. **The owner contacts workers one by one.** The owner calls or messages the first person on the mental list and waits for an answer. If the person refuses or does not answer within a reasonable time, the owner contacts the next person. In the reference store this typically takes three to five contacts per request.
4. **A worker agrees.** When one worker agrees, the owner confirms to that worker and to the worker who asked. The other workers who were contacted may not be told that the shift has been filled.
5. **The owner updates the spreadsheet.** The owner changes the name in the cell for that shift and tells the workers concerned.
6. **The owner rechecks weekly hours at month end.** At the end of the month the owner adds up each worker's hours per week from the spreadsheet, including the swapped shifts.
7. **The owner decides the holiday allowance.** For each worker and week, the owner decides from memory whether the worker had at least 15 contractual hours and attended every working day, and whether a swapped shift counts as an absence.
8. **The owner calculates the monthly pay.** The owner multiplies hours by the hourly wage, adds the holiday allowance, and transfers the pay. Whether the hourly wage is still above the current minimum wage is not checked systematically.

Three variations occur regularly. Workers sometimes arrange a swap between themselves and only inform the owner afterwards, or not at all, in which case the spreadsheet does not match reality. Two workers may both agree when the owner has contacted them in parallel, and one of them must then be told that the shift is no longer free. Finally, when nobody agrees, the owner covers the shift personally, and this is often not recorded.

2.1.3 Data flow diagram of the as-is process

The figure below models the procedure as a data flow diagram (DFD) following the rules of Chapter 2 of the lecture [6]: boxes are processes named with verb phrases, arcs are data named with noun phrases, control is not shown, and actors are not included. Arcs that enter from outside the diagram carry data supplied by a person (the worker or the owner) whose identity is omitted by the DFD rules. Two outputs from one box mean that one or the other is produced: process 3 produces either a refusal, which feeds process 2 again, or a substitute agreement.

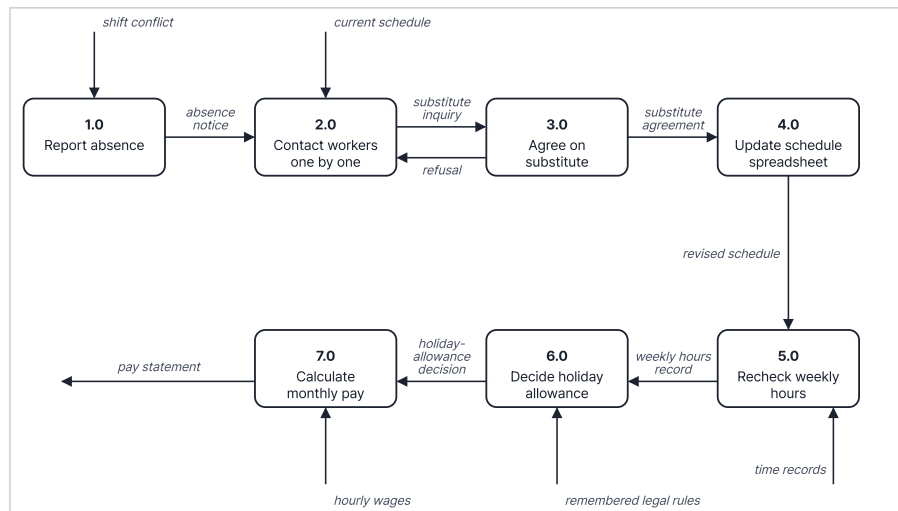


Figure 1. Data flow diagram of the as-is substitute and pay process in the reference store (lecture Chapter 2 notation: processes as verb phrases, data as noun phrases, actors omitted)

The diagram makes two properties of the current process visible. First, the loop between processes 2.0 and 3.0 carries one inquiry and one refusal at a time; the number of iterations of this loop is the cost described as problem P1. Second, processes 5.0 to 7.0 depend on data that is not recorded in the store: "time records" are notes kept by the owner, and "remembered legal rules" is literally the owner's memory. These two inputs are the source of problems P2 and P3.

2.2 Problems of the Current Process

The as-is process causes three problems. Each problem statement below gives the observable symptom, the cause in the data flow diagram of Section 2.1.3, and the consequence.

P1. Finding a substitute costs repeated one-to-one contacts

For every substitute request the owner contacts workers one at a time, typically three to five contacts per request in the reference store. The cause is that the information needed to choose a candidate — who is free at that time and who is willing to work extra — exists only in the owner's memory and in the workers' heads, so the owner has to ask. Because each inquiry waits for an answer, the elapsed time grows with the number of contacts, and a request made late in the evening may not be settled until the next day. When the owner contacts several people in parallel to save time, two of them may agree, and one has to be disappointed. The workers who refused or were not asked do not see whether the shift has been filled.

P2. Every swap changes the hours of two workers, and all recalculation is manual

A swap removes a shift from one worker's week and adds it to another's. Both workers' weekly scheduled hours change, and the holiday-allowance eligibility of either of them may change depending on how the missed shift is counted. In the current process the spreadsheet cell is edited (process 4.0), but the effect on hours is not calculated until the end of the month (process 5.0), when the owner must reconstruct from the edited spreadsheet which shifts were swapped and when. The owner approves a swap without seeing its effect on pay, and if the spreadsheet was not updated for a swap arranged directly between workers, the monthly calculation is wrong.

P3. Legal pay rules are applied from memory, so errors are not caught before payday

The weekly holiday allowance, the minimum wage, the five-employee threshold for premium pay and the probation reduction are all conditional rules with numeric thresholds (Section 2.3). In the reference store they are applied from memory at month end. Errors that this practice allows include paying a holiday allowance to a worker below 15 contractual hours, not paying it to a worker at or above the threshold, continuing to pay the previous year's minimum wage after 1 January, and applying the 90 % probation rate to a worker whose contract does not qualify for it. Because no check is made before the transfer, these errors are discovered, if at all, only when a worker complains.

Table 6. Summary of problems, causes in the as-is process and consequences

ID	Problem	Cause (DFD process)	Consequence
P1	Repeated one-to-one contacts to find a substitute	Availability not recorded; inquiries sent one at a time (2.0 ↔ 3.0)	3–5 contacts per request; delays; double acceptances when contacts are made in parallel

ID	Problem	Cause (DFD process)	Consequence
P2	Manual recalculation for two workers per swap	Spreadsheet edited without calculation (4.0); hours rebuilt at month end (5.0)	Swaps approved without knowing their pay effect; mismatches between schedule and reality
P3	Legal rules applied from memory	Rules not recorded (input "remembered legal rules" of 6.0 and 7.0)	Pay errors discovered only after payday

2.3 Legal Background

Part-time workers in Korea are covered by the same labour statutes as full-time workers. The provisions below determine the pay rules the system implements as business rules BR-06 to BR-10 (Chapter 3). The team read the statutes on the National Law Information Center [3], [4] and the explanations of the government's legal information service for part-time workers [5]. The system is a tool for applying these rules consistently; it is not legal advice, and the rules were simplified in the ways stated below.

2.3.1 Minimum wage

Under the Minimum Wage Act, the Minister of Employment and Labor announces the minimum hourly wage for each year. For 2026 the minimum hourly wage is KRW 10,320, up from KRW 10,030 in 2025, an increase of about 2.9 % [1], [2]. An employer who pays less than the minimum wage breaks the law regardless of what the contract says. For the system this means that the applicable minimum depends on the year of the pay period, so the minimum wage is stored as a table of yearly values (table MinimumWage, UC-10) rather than as a constant, and each payroll row is checked against the value for its year (BR-09).

2.3.2 Weekly paid holiday and the 15-hour threshold

Article 55 ① of the Labor Standards Act requires an employer to give a worker at least one paid holiday per week on average. Article 30 ① of the Enforcement Decree specifies that this paid holiday is given to a worker who has attended all the working days agreed for that week. Article 18 ③ excludes from this provision workers whose contractual working hours, averaged over four weeks, are less than 15 hours per week [3]. For a part-time worker the paid holiday is proportional to the worker's hours; the government's guidance computes the paid holiday hours as weekly contractual hours $\div 40 \times 8$, with the weekly hours capped at 40 [5]. The money paid for this holiday is what Korean workers call 주휴수당, rendered in this report as the weekly holiday allowance.

Two points required interpretation. The first is the four-week average. In the target stores a worker's contractual hours come from a fixed weekly schedule, which is the same every week; the four-week average therefore equals the weekly value, and the system uses the weekly contractual hours directly (BR-06). The second is a swapped shift. When a worker gives a shift to a colleague with the owner's approval, the working day has been changed by agreement, and the team's reading of [5] is that this should not count as a failure to attend. Because store practice differs, the system makes this a workplace setting (subAttendancePolicy = EXCUSED, the default, or ABSENT) rather than hard-coding one interpretation.

2.3.3 Premium pay and the five-employee threshold

Article 56 of the Labor Standards Act requires an additional 50 % of the ordinary wage for extended work (beyond 8 hours a day or 40 hours a week) and for night work between 22:00 and 06:00. Article 11 limits the application of the Act: a workplace that ordinarily employs fewer than five workers is subject only to the provisions listed in the Enforcement Decree, and Article 56 is not among them [3]. Many small caf  s fall below this threshold; the demonstration store Dalbit Caf   is configured with four regular employees. The system therefore stores the number of regular employees as a workplace setting and computes premium pay only when it is five or more (BR-08). Holiday-work premium under Article 56 ② is outside the scope of the system.

2.3.4 Probation reduction

Article 5 ② of the Minimum Wage Act allows an employer to pay 90 % of the minimum wage during the first three months of employment for a worker under probation, but only if the employment contract is for one year or more, and not for workers in simple labour occupations designated by the Ministry of Employment and Labor (Enforcement Decree Art. 3) [4]. Many part-time jobs in caf  s and convenience stores, such as serving and cashier work, fall into the designated simple labour occupations. The system requires all conditions to hold before it lowers the minimum-wage threshold for a worker (BR-10).

Table 7. Legal provisions and the business rules that implement them

Provision	Content used by the system	Business rule
-----------	----------------------------	---------------

Provision	Content used by the system	Business rule
Minimum Wage Act; MOEL notice for 2026 [1], [2]	Minimum hourly wage KRW 10,320 (2026), KRW 10,030 (2025)	BR-09
Labor Standards Act Art. 55 ①; Enforcement Decree Art. 30 ① [3]	One paid holiday per week for a worker who attended all agreed working days	BR-06, BR-07
Labor Standards Act Art. 18 ③ [3], [5]	No weekly paid holiday below 15 contractual hours per week (four-week average)	BR-06
Labor Standards Act Art. 11, Art. 56 [3]	+50 % for daily hours over 8, weekly hours over 40 and night hours 22:00–06:00; only with five or more regular employees	BR-08
Minimum Wage Act Art. 5 ②; Enforcement Decree Art. 3 [4]	90 % of the minimum wage for the first three months of probation, contract of one year or more, not simple labour	BR-10

2.4 Related Work

Shift scheduling is an established product category. The team examined five products or product groups named in [10]: Shiftee, a Korean workforce-management service; the schedule and work-management functions of the Korean job portals Albamon (알바몬 근무관리) and Alba Cheonguk (알바천국 스케줄); and three international products, When I Work, Homebase and Deputy. The comparison uses five features that correspond to the core of ShiftSwap:

- **Substitute broadcast:** a request for cover is sent to several workers at once, restricted to those who are eligible;
- **First-accept:** the first worker to accept is the substitute, and later acceptances are refused;
- **Owner approval:** the swap takes effect only after a manager or owner approves it;
- **Holiday-allowance recomputation at approval:** the effect of the swap on both workers' weekly holiday allowance is shown when the owner approves;
- **Korean legal rules:** the 15-hour threshold, the five-employee threshold and the probation reduction are applied in pay calculation.

The comparison table below records only what the team could confirm from the public product descriptions available to it. "Supported" means the team found the feature described; "not documented" means the team did not find a description of it, which does not prove that the feature is absent. The products change frequently, and the table reflects the team's reading at the time of writing.

Table 8. Feature comparison of shift-scheduling products (based on public product descriptions; see text for the meaning of each entry)

Product	Substitute broadcast	First-accept	Owner approval	Holiday-allowance recomputation at approval	Korean legal rules
Shiftee	not documented	not documented	not documented	not documented	not documented
Albamon work management / Alba Cheonguk schedule	not documented	not documented	not documented	not documented	not documented
When I Work	supported (shifts offered to eligible staff)	not documented	supported (manager approves swaps and drops)	not documented	not documented
Homebase	not documented	not documented	supported (manager approves trades and cover requests)	not documented	not documented
Deputy	supported (shift offers to qualified staff)	not documented	supported (manager approves swaps and offers)	not documented	not documented

Product	Substitute broadcast	First-accept	Owner approval	Holiday-allowance recomputation at approval	Korean legal rules
ShiftSwap	supported (FR-07, BR-01)	supported (FR-09, BR-02)	supported (FR-12, BR-04)	supported (FR-11)	supported (BR-06...BR-10)

The international products share a pattern: a worker offers or trades a shift, colleagues who are qualified for it can pick it up, and a manager approves the change. They are designed for the labour law of other countries and for larger organisations with several locations and roles, and their pay features, where present, are connected to external payroll services. The Korean services are closer to the target users in language and in their awareness of part-time work, but the team did not find a description of a substitute workflow with first acceptance and approval in them. None of the products, in the descriptions the team read, shows the owner the change in both workers' holiday-allowance eligibility at the moment of approving a swap [10].

Besides products, the course material provides the methods used in this project. The life-cycle models and the requirements classification come from the course textbook [6] and from Sommerville's classification of non-functional requirements used in the lecture on requirements [7]. The UML diagrams follow the OMG UML 2.5.1 specification [8], and the embedded database is SQLite compiled to WebAssembly (sql.js) [9].

2.5 The Gap ShiftSwap Fills

ShiftSwap targets the part of the problem that general scheduling products do not address for small Korean stores. Its contribution is the combination of four elements in one workflow:

1. candidate selection from recorded availability and existing shifts, so that a request reaches only the workers who can take it (addresses P1);
2. a first-acceptance-wins rule enforced in a single database transaction, so that two acceptances can never both succeed (addresses the double-acceptance variation of P1);
3. an approval step that shows, for both workers, contractual hours, scheduled hours and holiday-allowance eligibility before and after the swap, and recomputes the weekly summaries on approval (addresses P2);
4. pay rules taken from Korean law and stored as data where they change (minimum wage by year, number of regular employees, substitution-attendance policy), with warnings before payroll confirmation (addresses P3).

The system is deliberately small. It serves one store, runs without a server and does not replace a full payroll or HR system. Its value lies in making the rules explicit and applying them at the moment a decision is made, rather than at the end of the month. Chapter 3 turns these four elements into use cases, functional requirements and business rules.

3. Requirements Specification

This chapter specifies what ShiftSwap must do. It follows the requirements engineering activities of Chapter 8 of the lecture — elicitation, analysis, validation and management [6], [7] — and the contents recommended there for a software requirements specification: problem description, functional and non-functional requirements, scenarios, use-case diagram and supplementary diagrams. The problem description is Chapter 2. Section 3.1 explains how the requirements were elicited, and Section 3.2 identifies stakeholders and actors. Sections 3.3 to 3.5 present the use-case model, the functional requirements and a full description of each use case. Section 3.6 states the business rules, the domain requirements that carry the legal content of the system. Section 3.7 gives the non-functional requirements, Section 3.8 the scenarios, and Section 3.9 the analysis models. Sections 3.10 and 3.11 describe how the requirements were validated and how changes to them were managed.

All identifiers used in this chapter (A1–A3, UC-01–UC-13, FR-01–FR-21, NFR-01–NFR-13, BR-01–BR-12, TC groups) are defined in the specification file `spec/spec.md` of the repository, which the team treated as the single source of truth (Section 3.11). The wording of the requirements in this chapter is the wording of that file.

3.1 Requirements Elicitation

The lecture divides elicitation and analysis into requirements discovery, classification and organisation, prioritisation and negotiation, and specification [7]. For discovery the team used three sources.

3.1.1 Practice in the reference store

The first and main source was the practice of the reference store, described by Hyewon Kim from her own experience as a part-time worker there. She wrote down the as-is procedure of Section 2.1 step by step, including the variations (swaps arranged between workers, double agreement, the owner covering the shift). From this description the team extracted the entities the owner deals with — worker, fixed weekly shift, shift on a date, substitute request, the workers asked, the reply, attendance, weekly hours and monthly pay — and the decisions the owner makes, which became candidate use cases. This source is also the origin of the only quantitative statement about the current process used in the report, the figure of three to five contacts per substitute request (P1). The team did not collect further measurements in the store and does not report any.

3.1.2 Legal documents

The second source was the legislation and official guidance listed in Section 2.3: the Ministry of Employment and Labor notice on the 2026 minimum wage [1], [2], the Labor Standards Act and its Enforcement Decree [3], the Minimum Wage Act and its Enforcement Decree [4], and the government's legal information service for part-time workers [5]. These documents are the source of the domain requirements, which the lecture defines as constraints on the system from its domain of operation [7]. Each legal provision was converted into a business rule with explicit conditions and numbers (Section 3.6), and each business rule was reviewed against the text of the provision in the requirements review (Section 3.10). Where the law needed interpretation, the interpretation was either written into the rule with a note (the four-week average of BR-06) or made a workplace setting (the treatment of a handed-over shift, `subAttendancePolicy`).

3.1.3 Interview plan

The third source is a set of semi-structured interviews with the owner of the reference store and two co-workers, planned under WBS 3.1.1 (interview plan and notes) and scheduled to begin in Week 3. The interviews serve to confirm or correct what the team derived from the first two sources, not to produce statistics. The table below gives the plan. This report does not present interview results as numbers; where the requirements depend on a store-specific choice, the system makes it a setting instead of relying on the answer of one interviewee.

Table 9. Interview plan for requirements discovery (WBS 3.1.1)

Interviewee	Topic	Main questions	Requirements affected
Owner	Substitute procedure	How do you choose whom to contact first? What information do you need before agreeing to a swap? Would you accept a swap without seeing its effect on hours?	FR-06–FR-12, BR-01, BR-04
Owner	Pay calculation	How do you decide the weekly holiday allowance today? How do you treat a shift that a worker gave to a colleague? When do you update the hourly wage after a minimum-wage change?	FR-14–FR-16, BR-06–BR-11
Owner	Deadlines	How long before a shift must a substitute be confirmed? What happens when nobody answers?	FR-18, BR-03, BR-12

Interviewee	Topic	Main questions	Requirements affected
Co-workers (2)	Receiving requests	How do you prefer to be asked? How quickly do you usually answer? Have you ever agreed to a shift that someone else had already taken?	FR-07, FR-09, FR-10, NFR-01
Co-workers (2)	Availability and privacy	Would you record the times you are available for extra shifts? Which of your data should other workers not see?	FR-03, NFR-11
All	Devices	Which device do you use to check the schedule? Do you use a computer at the store?	NFR-02, NFR-06

3.1.4 Domain analysis and organisation

In addition, the team examined the public descriptions of existing shift-scheduling products (Section 2.4) to find what is common to the domain. Shift trades with manager approval appear in several products, which confirmed that owner approval (UC-06) is an expected step and not a peculiarity of the reference store. The candidate requirements from all sources were then classified into the groups used below (worker and schedule management, substitute workflow, attendance and pay, administration and access), prioritised with the MoSCoW scheme (Section 1.3), and specified in the specification file. One prioritisation decision needs explanation. Perfect attendance is a condition of the holiday allowance, which might suggest that attendance recording (UC-07) is a Must. It is a Should because the holiday-allowance rule treats a shift that has not been marked ABSENT as attended (Section 3.9.2); Increment 1 therefore computes a correct holiday allowance for every week without absences even before attendance recording exists.

3.2 Stakeholders and Actors

The lecture defines stakeholders as everyone with an interest in the system, including end users, managers, maintenance engineers and domain experts [7]. Actors are the subset of stakeholders, and of external systems, that interact with the system directly. The two tables below list the stakeholders and the three actors of the use-case model.

Table 10. Stakeholders of ShiftSwap

Stakeholder	Interest in the system	How represented in the requirements
Store owner	Fewer contacts per substitute; correct schedule; correct and lawful pay; no disputes on payday.	Actor A2; goals G1, G3, G4; FR-11, FR-12, FR-15, FR-16.
Part-time worker (requester)	Find a substitute quickly without personally asking colleagues; keep the holiday allowance where possible.	Actor A1; FR-06, FR-19; before/after comparison (FR-11).
Part-time worker (candidate)	Be asked only when actually available; know at once whether a shift is still free; privacy of wage and phone number.	Actor A1; BR-01, BR-02; NFR-01, NFR-11.
Legislator and labour authorities	Minimum wage, weekly paid holiday and premium pay applied as the law requires.	Business rules BR-06 to BR-10; NFR-10.
Course instructor	A system that demonstrates the full life cycle and can be reproduced from the final prompt.	Organisational requirements NFR-07 to NFR-09; goals G5, G6.
Development team	A scope that can be delivered in one semester; a system that three students can test.	MoSCoW scope; client-only architecture (NFR-08); demo clock (A3).

Table 11. Actors of the use-case model

ID	Actor	Type	Description
A1	Worker	Primary, human	Part-time employee. Views own schedule, registers availability, requests substitutes, responds to requests, records attendance.
A2	Owner	Primary, human	Store owner (also acts as manager). Registers workers and fixed schedules, approves/rejects substitutions, confirms attendance, reviews weekly summaries, confirms monthly payroll, edits settings.

ID	Actor	Type	Description
A3	System Clock	Secondary, system	Time-triggered actor. Expires unanswered/unapproved requests at their deadline and closes weeks for weekly summary. (In the demo app this is driven by the adjustable "demo clock".)

A worker who receives a substitute request is still actor A1; the use-case descriptions call this worker the *candidate* and the worker who made the request the *requester*. A separate candidate actor is not used, because every worker can be a requester in one request and a candidate in another, and the difference is a role in a particular request, not a kind of user. The System Clock is modelled as an actor because expiry (UC-11) happens without any user action: it is triggered by the passage of time. In the demonstration system, time is the adjustable demo clock, so that expiry can be shown in a few seconds instead of waiting for a real deadline.

The owner could in principle be a specialisation of worker, since the owner is also stored in the table Worker (with role OWNER). In the use-case model the two are kept as separate actors because they have no use cases in common except sign-in (UC-13) and attendance (UC-07), and in attendance their steps differ: the worker records times, the owner confirms them.

3.3 Use-Case Model

The figure below shows the use-case diagram. It contains thirteen use cases. UC-01 to UC-09 correspond one to one to the nine main functions announced in the proposal; UC-10 to UC-13 are supporting use cases, already listed in the proposal, which make settings, expiry, cancellation and role selection explicit so that the core use cases are complete. The table after the diagram lists the use cases with their primary actor, priority and the database tables they mainly use.

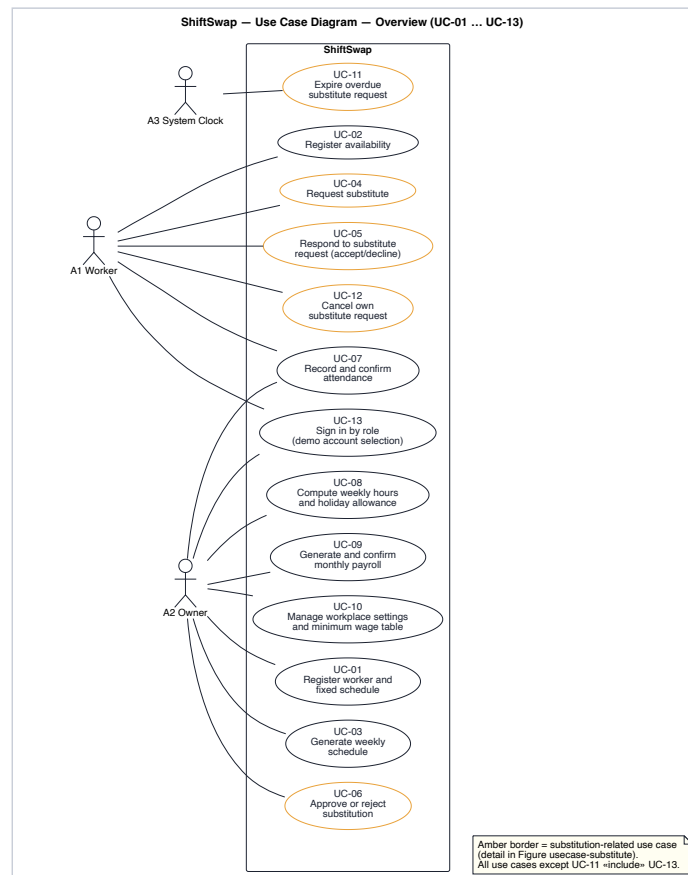


Figure 2. Use-case diagram of ShiftSwap (UML 2.5)

Table 12. Use cases of ShiftSwap

ID	Name	Primary actor	Priority	Main tables
UC-01	Register worker and fixed schedule	Owner	Must	Worker, FixedSchedule
UC-02	Register availability	Worker	Must	Availability
UC-03	Generate weekly schedule	Owner	Must	Shift

ID	Name	Primary actor	Priority	Main tables
UC-04	Request substitute	Worker	Must	SubRequest, SubRequestTarget, Notification
UC-05	Respond to substitute request (accept / decline)	Worker (candidate)	Must	SubRequestTarget, SubRequest
UC-06	Approve or reject substitution	Owner	Must	SubRequest, Shift, WeeklySummary
UC-07	Record and confirm attendance	Worker, Owner	Should	Attendance, Shift
UC-08	Compute weekly hours and holiday allowance	Owner (System)	Must	WeeklySummary, Workplace
UC-09	Generate and confirm monthly payroll	Owner	Should	Payroll, MinimumWage, Worker
UC-10	Manage workplace settings and minimum wage table	Owner	Should	Workplace, MinimumWage
UC-11	Expire overdue substitute request	System Clock	Must	SubRequest, SubRequestTarget, Notification
UC-12	Cancel own substitute request	Worker	Should	SubRequest, SubRequestTarget
UC-13	Sign in by role (demo account selection)	Worker, Owner	Must	Worker

The lecture describes «include» as a common function that a use case always uses and «extend» as an exceptional condition that adds behaviour to a base use case [6]. The diagram uses both relationships as follows.

- **UC-04 «include» Find eligible candidates.** Candidate selection (BR-01) is always performed when a request is created. It is drawn as a separate included use case because it is the most important rule of the substitute workflow and because it has its own activity diagram (activity diagram AD-2 in Section 3.9).
- **UC-06 «include» UC-08.** Approving a substitution always recomputes the weekly summaries of the requester and the acceptor. The before/after comparison shown before approval uses the same computation.
- **UC-09 «include» UC-08.** The monthly payroll takes its holiday allowance from the weekly summaries, so they are recomputed when a payroll draft is generated.
- **UC-11 «extend» UC-04.** The extension point is "deadline reached with no approved acceptance". Expiry is the exceptional ending of a request, triggered by the System Clock.
- **UC-12 «extend» UC-04.** Cancellation is the other exceptional ending, triggered by the requester while the request is still open.
- **All use cases except UC-11 «include» UC-13.** Every user action requires that the user has signed in, and the role chosen at sign-in decides which use cases are available. UC-11 is the only use case not started by a signed-in user.

In the diagram the Worker is also associated with UC-01, although the Owner is its primary actor. The association represents the worker as the subject of the registration, who afterwards sees the registered fixed schedule on the weekly board; the worker cannot create or change a fixed schedule.

3.4 Functional Requirements

The lecture defines functional requirements as statements of the services the system should provide, how it should react to particular inputs and how it should behave in particular situations [7]. Following the SRS contents of the lecture, each requirement below has an identifier, a group, a statement and the use case it belongs to; the priority is inherited from the use case. The statements use "shall" for mandatory behaviour. The table below is the complete list.

Table 13. Functional requirements

ID	Group	Requirement	UC	Priority
FR-01	Workers and schedules	The owner shall register a worker with name, phone, hourly wage (KRW), contract start date, contract end date, probation end date (optional), simple-labor flag, and active flag.	UC-01	Must
FR-02	Workers and schedules	The owner shall register one or more fixed weekly shifts per worker (weekday, start time, end time); fixed shifts of the same worker must not overlap. Deleting a fixed-schedule row keeps the shifts it already generated (their fixedScheduleId becomes NULL).	UC-01	Must

ID	Group	Requirement	UC	Priority
FR-03	Workers and schedules	A worker shall register availability slots (weekday, start, end) during which they can take substitute shifts.	UC-02	Must
FR-04	Workers and schedules	The system shall generate the shifts of a given week (Mon-Sun) from all active fixed schedules; generating the same week twice shall not create duplicates.	UC-03	Must
FR-05	Workers and schedules	The owner shall be able to add, move or delete a single shift of a generated week. Deleting a shift that has a substitute request or an attendance record is refused with an explanatory message.	UC-03	Must
FR-06	Substitute workflow	A worker shall create a substitute request for one of their own future SCHEDULED shifts, with an optional reason and a response deadline.	UC-04	Must
FR-07	Substitute workflow	On creation the system shall compute the eligible candidates (BR-01), create one SubRequestTarget per candidate, and put a notification in each candidate's inbox at once.	UC-04	Must
FR-08	Substitute workflow	If no worker is eligible, the system shall still create the request and notify the owner that no candidate exists.	UC-04	Must
FR-09	Substitute workflow	A candidate shall accept or decline a request from their inbox. The first acceptance makes that candidate the request's acceptor (BR-02); later acceptances shall be refused with an explanatory message.	UC-05	Must
FR-10	Substitute workflow	When a request is accepted, the system shall notify the requester and the owner, and close all other pending targets.	UC-05	Must
FR-11	Substitute workflow	The approval screen shall show, for both the requester and the acceptor, the weekly contractual hours, weekly scheduled hours and holiday-allowance eligibility before and after the swap, and highlight any eligibility change.	UC-06	Must
FR-12	Substitute workflow	On approval the system shall reassign the shift to the acceptor (keeping the original worker), recompute the weekly summaries of both workers, and notify both. On rejection the shift stays unchanged and both are notified.	UC-06	Must
FR-13	Attendance and pay	A worker shall record actual start and end time of a shift they worked; the owner shall confirm it or mark the shift ABSENT.	UC-07	Should
FR-14	Attendance and pay	The system shall compute, per worker and week, contractual hours, scheduled hours, actual hours, perfect-attendance flag, holiday-allowance eligibility and holiday-allowance hours (BR-06, BR-07).	UC-08	Must
FR-15	Attendance and pay	The owner shall generate a monthly payroll draft per worker (base pay, holiday allowance, premium pay, total) and confirm it; a confirmed payroll is read-only.	UC-09	Should
FR-16	Attendance and pay	The payroll screen shall warn when a worker's hourly wage is below the applicable minimum wage (BR-09), and apply the probation reduction only when BR-10 holds.	UC-09	Should
FR-17	Administration	The owner shall edit workplace settings: store name, number of regular employees, substitution-attendance policy (BR-06), and yearly minimum wage rows.	UC-10	Should
FR-18	Substitute workflow	When a request's deadline passes, or its shift starts, while it is still REQUESTED or ACCEPTED, the system shall mark it EXPIRED, close its targets and notify requester and owner.	UC-11	Must
FR-19	Substitute workflow	A requester shall cancel their own request while it is REQUESTED or ACCEPTED; all PENDING targets become CLOSED, and every target that was PENDING or ACCEPTED receives a REQUEST_CANCELLED notification.	UC-12	Should

ID	Group	Requirement	UC	Priority
FR-20	Access	Users shall sign in by choosing a demo account, either from a guided three-step demo tour or from the list of all accounts; the UI shows only the menus of that role.	UC-13	Must
FR-21	Access	Every user shall have an in-app notification inbox listing notifications newest first with unread count.	UC-04-06, 11, 12	Must

Every functional requirement maps to at least one use case, as the traceability rule of the specification requires (spec §12). The reverse also holds: every use case is realised by at least one functional requirement. UC-08 is realised by FR-14 alone, but the computation it describes is detailed by business rules BR-05 to BR-07. The notification inbox (FR-21) is not a use case of its own; it is the shared output of the five use cases that create notifications. The kinds of notification are fixed by the specification: REQUEST_RECEIVED, REQUEST_ACCEPTED, REQUEST_APPROVED, REQUEST_REJECTED, REQUEST_EXPIRED, REQUEST_CANCELLED, NO_CANDIDATE and TARGET_CLOSED.

The following conventions apply to all requirements. Dates are ISO YYYY-MM-DD, times are HH:MM in 24-hour notation, and datetimes are YYYY-MM-DDTHH:MM. A time interval [s, e) includes its start and excludes its end, so a shift ending at 18:00 does not overlap one starting at 18:00. An end time that is not later than the start time means that the shift ends on the following day. Money amounts are integers in Korean won.

3.5 Use-Case Descriptions

The lecture notes that a use case is represented both in a diagram, as an overview, and in a more detailed textual form [6]. This section gives the textual form for every use case, UC-01 to UC-13, in one fixed structure: identifier and name, actors, preconditions, trigger, main flow as numbered steps, alternative flows, exception flows, postconditions, the business rules applied, the related functional and non-functional requirements, and the screen of the application on which the use case is performed. The fields correspond to the scenario contents required by the SRS outline of the lecture (related actors, initial condition, final condition, exceptional case and step-by-step flow of action). Alternative flows describe other successful paths; exception flows describe paths on which the goal of the use case is not reached. The screens are described in Chapter 4 and shown in the user guide of Chapter 7.

Two conventions keep the descriptions short. "The system validates" always means that invalid input leaves the data unchanged and produces a message that states the cause and the correction, as required by the copy rules of the design. Every use case except UC-11 includes UC-13, which is therefore stated only as a precondition.

UC-01 Register worker and fixed schedule

Table 14. Use-case description UC-01

ID	UC-01
Name	Register worker and fixed schedule
Actors	Primary: Owner (A2). Secondary: Worker (A1), as the subject of the registration.
Preconditions	- The owner is signed in (UC-13). - The workplace exists (seed: Dalbit Café).
Trigger	A new worker is hired, or the contract or fixed weekly shifts of an existing worker change.

Main flow	1. The owner opens the <i>Staff</i> tab. 2. The system lists the workers of the workplace with their contract period, hourly wage and weekly contractual hours. 3. The owner selects <i>Add worker</i>. 4. The owner enters name, phone, hourly wage (KRW), contract start date, contract end date (empty for an open-ended contract), probation end date (optional), simple-labor flag and active flag. 5. The system validates the input: name present, hourly wage a positive whole number of won, contract start date present, contract end not before contract start, probation end not before contract start. 6. The system stores the worker with role WORKER and shows the worker in the list. 7. The owner adds a fixed weekly shift by choosing weekday, start time and end time. 8. The system checks that the new fixed shift does not overlap another fixed shift of the same worker (FR-02) and stores it. 9. Steps 7–8 are repeated for each fixed shift. The system shows the resulting weekly contractual hours (sum of fixed-shift hours, BR-06).
Alternative flows	• A1 Edit worker. At step 3 the owner selects an existing worker and changes its data; the flow continues at step 5. • A2 Deactivate worker. The owner clears the active flag. The worker is no longer a candidate for substitute requests (BR-01) and no new shifts are generated for them (FR-04); existing shifts and records remain. • A3 Remove a fixed shift. The owner deletes a fixed shift. Shifts already generated from it keep existing, and their link to the fixed schedule is cleared (FR-02); later weeks are generated without it.
Exception flows	• E1 Overlapping fixed shift. At step 8 the new fixed shift overlaps an existing one of the same worker. The system does not store it and names the conflicting weekday and times. • E2 Invalid data. At step 5 a required field is missing or a date is inconsistent. The system keeps the form open and states which field is wrong and how to correct it.
Postconditions	• The worker and the fixed schedule are stored. • The worker's weekly contractual hours are defined by the fixed schedule.
Business rules	BR-06 (contractual hours), BR-09 and BR-10 (wage, contract and probation fields used later)
Related FR / NFR	FR-01, FR-02; NFR-11 (phone and wage visible only to owner and the worker)
UI screen	Staff (직원): worker cards and worker detail with the fixed-schedule editor (owner)

UC-02 Register availability

Table 15. Use-case description UC-02

ID	UC-02
Name	Register availability
Actors	Primary: Worker (A1).
Preconditions	• The worker is signed in (UC-13).
Trigger	The worker wants to offer time in which they could take substitute shifts.
Main flow	1. The worker opens the <i>Me</i> tab and its availability editor. 2. The system lists the worker's availability slots by weekday. 3. The worker adds a slot by choosing weekday, start time and end time. 4. The system validates the slot and stores it. 5. The system shows the updated list.

Alternative flows	• A1 Remove a slot. The worker deletes a slot; the system removes it and shows the updated list. • A2 Change a slot. The worker removes a slot and adds a corrected one.
Exception flows	• E1 Invalid times. A time is missing, or the end time is not later than the start time (an availability slot must end on the same day). The system does not store the slot and explains the correction.
Postconditions	• The availability slots are stored and are used for all substitute requests created afterwards (BR-01). • Requests that already exist keep the candidates computed at their creation (FR-07).
Business rules	BR-01 (availability is an input of candidate selection)
Related FR / NFR	FR-03; NFR-02
UI screen	Me (내 정보): availability editor (worker)

UC-03 Generate weekly schedule

Table 16. Use-case description UC-03

ID	UC-03
Name	Generate weekly schedule
Actors	Primary: Owner (A2).
Preconditions	• The owner is signed in (UC-13). • At least one active worker has a fixed schedule.
Trigger	The owner opens a week (Monday to Sunday, BR-05) that has not yet been generated, or wants to adjust a generated week.
Main flow	1. The owner opens the <i>Schedule</i> tab. 2. The owner navigates with <i>previous</i> / <i>next</i> to the week to be generated. 3. The owner selects <i>Create this week's schedule</i>, the primary action shown when the displayed week has no shifts. 4. For each active worker and each of the worker's fixed shifts, the system creates a shift on the corresponding date of the week with status SCHEDULED, unless a shift for that fixed schedule and date already exists. 5. The system shows the week as seven columns, Monday to Sunday, with shifts ordered by start time.
Alternative flows	• A1 Week already generated. At step 4 all shifts already exist; the system creates no duplicates (FR-04) and shows the week unchanged. • A2 Add a single shift. The owner adds a shift for a worker on a date and time outside the fixed schedule (FR-05). • A3 Move or delete a shift. The owner changes the date or time of a shift, or deletes it (FR-05).
Exception flows	• E1 Overlap after edit. An added or moved shift would overlap another shift of the same worker. The system refuses the change and names the conflicting shift. • E2 Shift in use. The owner tries to delete a shift that has a substitute request or an attendance record. The system refuses and suggests changing its time or worker instead (FR-05).
Postconditions	• All shifts of the week exist exactly once per fixed schedule and date. • New shifts have status SCHEDULED and no original worker.
Business rules	BR-05 (week definition)
Related FR / NFR	FR-04, FR-05; NFR-02, NFR-03

UI screen	Schedule (근무표), owner view, with the shift detail panel
-----------	---

UC-04 Request substitute

Table 17. Use-case description UC-04

ID	UC-04
Name	Request substitute
Actors	Primary: Worker (A1) as requester. Secondary: other Workers (A1) as candidates, Owner (A2) as receiver of notifications.
Preconditions	- The worker is signed in (UC-13). - The worker is the current worker of a shift that is SCHEDULED and starts in the future. - The shift has no open (REQUESTED or ACCEPTED) request.
Trigger	The worker learns that they cannot work one of their coming shifts.
Main flow	- The worker starts the request flow with <i>Find a substitute</i> from the <i>Swaps</i> tab, from the next-shift card on Home, or from the detail panel of one of their own shifts (the shift is then preselected). - Step 1 of 3: the system lists the worker's own future SCHEDULED shifts; a shift that already has an open request is shown disabled with the reason. The worker chooses a shift. - Step 2 of 3: the worker chooses a reason (optional; chips or free text) and a deadline, preset to the earlier of now + 24 hours and the shift start; the chosen deadline is repeated as a sentence. - Step 3 of 3: the system finds the eligible candidates at this moment (included use case <i>Find eligible candidates</i>, BR-01) and shows a summary of shift, reason and deadline with the names of the recipients. - The worker selects <i>Send substitute request</i>. - The system validates the request (BR-03), creates it with status REQUESTED, determines the eligible candidates again inside the transaction, and creates one SubRequestTarget with response PENDING for each. - The system puts a REQUEST_RECEIVED notification in the inbox of every candidate at once. - The system shows a result screen with the progress tracker at its first step (Requested → Accepted → Approved) and the recipients. On the schedule the shift is marked with a dashed amber outline.
Alternative flows	A1 No eligible candidate. At step 4 the review already warns that nobody is available now; at step 6 no worker is eligible. The system still creates the request (step 7, with no targets), sends a NO_CANDIDATE notification to the owner, and tells the requester that no eligible worker was found (FR-08). Extension point <i>deadline reached with no approved acceptance</i>. UC-11 extends this use case when the deadline or the shift start passes while the request is open. Extension point <i>requester withdraws</i>. UC-12 extends this use case when the requester cancels the open request.
Exception flows	E1 Shift not eligible for a request. The shift belongs to another worker, is not SCHEDULED or has already started. The system refuses the request and states the reason. E2 Open request exists. The shift already has a REQUESTED or ACCEPTED request. The system refuses and points to the existing request. E3 Invalid deadline. The deadline is not after now or is later than the shift start. The system keeps the form open and states the allowed range.
Postconditions	- A request in state REQUESTED exists with one target per eligible candidate. - Each candidate has an unread REQUEST_RECEIVED notification, or the owner has a NO_CANDIDATE notification. - The shift itself is unchanged.
Business rules	BR-01, BR-03

Related FR / NFR	FR-06, FR-07, FR-08, FR-21; NFR-11
UI screen	Swaps (대체) tab and the three-step request flow; Home next-shift card; Schedule shift detail (worker)

UC-05 Respond to substitute request (accept / decline)

Table 18. Use-case description UC-05

ID	UC-05
Name	Respond to substitute request (accept / decline)
Actors	Primary: Worker (A1) as candidate. Secondary: requester (A1) and Owner (A2), who are notified.
Preconditions	- The candidate is signed in (UC-13). - The candidate has a target with response PENDING for a request.
Trigger	A REQUEST_RECEIVED notification arrives (unread count on the bell icon; card on Home).
Main flow	- The candidate opens Home (or follows the notification from the bell icon). - Home shows, first, a card for each pending request: who asks, the shift, the reason, a deadline countdown and the effect on the candidate's own week (scheduled hours before → after, holiday allowance unchanged or changed), with <i>Accept</i> and <i>Decline</i>. - The candidate selects <i>Accept</i> (one tap). - Within one database transaction the system reads the request again and confirms that it is still REQUESTED (BR-02). - The system sets the request to ACCEPTED with the candidate as acceptor, sets the candidate's target to ACCEPTED and all other PENDING targets to CLOSED. - The system sends REQUEST_ACCEPTED to the requester and the owner, and TARGET_CLOSED to the other candidates whose targets were closed (FR-10). - The system confirms the result in a message that also says what happens next (the owner's approval).
Alternative flows	A1 Decline. At step 3 the candidate selects <i>Decline</i>. The target becomes DECLINED; the request stays REQUESTED and remains open for the other candidates. A decline notifies no one.
Exception flows	E1 Already taken. At step 4 the request is no longer REQUESTED because another candidate accepted first. The transaction makes no change, and the card says that the other candidate was faster, naming them, and closes (FR-09). E2 Request closed. At step 4 the request has been cancelled or has expired. The system makes no change and shows the state of the request. E3 Candidate now busy. Since the request was created, the candidate has been given another shift that overlaps the requested one. The acceptance is refused (ACCEPTOR_BUSY) and nothing changes (BR-02).
Postconditions	- On success: request ACCEPTED, exactly one target ACCEPTED, no target PENDING; requester and owner notified. - On decline: the candidate's target is DECLINED; the request is unchanged.
Business rules	BR-02
Related FR / NFR	FR-09, FR-10, FR-21; NFR-01 (at most 2 clicks), NFR-04 (transaction)
UI screen	Home (홈) request card; Swaps tab, received requests (worker)

UC-06 Approve or reject substitution

Table 19. Use-case description UC-06

ID	UC-06
Name	Approve or reject substitution

Actors	Primary: Owner (A2). Secondary: requester and acceptor (A1), who are notified.
Preconditions	- The owner is signed in (UC-13). - A request is in state ACCEPTED.
Trigger	The owner receives a REQUEST_ACCEPTED notification or opens Home, where accepted requests are listed first.
Main flow	- The owner opens Home. - Home lists a decision card for each ACCEPTED request; open REQUESTED requests are listed separately, read-only, with their deadline countdown and each candidate's response. - The decision card states the swap in one sentence ("Seoyeon's Wednesday shift is covered by Minhoo"). - For the requester and the acceptor, the system computes the weekly contractual hours, weekly scheduled hours, holiday-allowance eligibility and holiday-allowance amount for the week of the shift, before and after the swap (uses UC-08). - The card shows one impact line per person; a change of eligibility is highlighted in amber with the words "holiday allowance changes" (FR-11). <i>Details</i> opens the full before/after table. - The owner selects <i>Approve</i>. - The system confirms that the request is still ACCEPTED, assigns the shift to the acceptor, records the requester as the shift's original worker if none is recorded yet, and sets the request to APPROVED with the decision time (BR-04). - The system recomputes the weekly summaries of both workers (includes UC-08). - The system sends REQUEST_APPROVED to the requester and the acceptor. - On the schedule the shift shows the handover marker: the acceptor's name in bold, the original worker struck through, an amber left edge.
Alternative flows	A1 Reject. At step 6 the owner selects <i>Reject</i>. The request becomes REJECTED, the shift is unchanged, and REQUEST_REJECTED is sent to both workers.
Exception flows	E1 Request no longer ACCEPTED. Between steps 3 and 7 the request expired (UC-11) or was cancelled (UC-12). The system refuses the decision and shows the current state. E2 Acceptor now busy. Since accepting, the acceptor has been given another shift that overlaps the requested one. The approval is refused (ACCEPTOR_BUSY) and the request stays ACCEPTED, so the owner can reject it (BR-02).
Postconditions	- On approval: shift assigned to the acceptor, original worker kept; request APPROVED; weekly summaries of both workers up to date. - On rejection: request REJECTED; shift and summaries unchanged.
Business rules	BR-04, BR-06, BR-07
Related FR / NFR	FR-11, FR-12, FR-21; NFR-03
UI screen	Home (홈) decision card with the <i>Details</i> table (owner); Schedule

UC-07 Record and confirm attendance

Table 20. Use-case description UC-07

ID	UC-07
Name	Record and confirm attendance
Actors	Primary: Worker (A1) and Owner (A2).
Preconditions	- The user is signed in (UC-13). - The shift has started (start ≤ now). For recording, the shift is still SCHEDULED and the worker is its current worker.
Trigger	A shift has been worked, or the owner reviews the past shifts.

Main flow	1. The worker opens the <i>Me</i> tab (or the "shifts to record" section of Home). 2. The system lists the worker's past shifts that have no attendance record. 3. The worker enters the actual clock-in and clock-out times and saves. 4. The system stores the attendance record as unconfirmed. 5. The owner opens Home, section "attendance records to confirm". 6. The system lists unconfirmed records with the scheduled and the recorded times. 7. The owner confirms a record. 8. The system marks the record confirmed and sets the shift to WORKED; the worker's weekly summary is recomputed.
Alternative flows	• A1 Mark absent. At step 7 (or for a shift with no record) the owner marks the shift ABSENT. The shift status becomes ABSENT, which breaks perfect attendance for that week (BR-06). • A2 Correct a record. Before confirmation the worker changes the recorded times.
Exception flows	• E1 Incomplete times. A time is missing or clock-in equals clock-out. The system does not store the record and explains the correction. • E2 Already closed. Once the owner has confirmed the record or marked the shift absent, the worker can no longer change it. • E3 Nothing to confirm. The owner tries to confirm a shift the worker has not recorded. The system asks the owner to have it recorded or to mark the shift absent.
Postconditions	• The shift is WORKED with a confirmed record, or ABSENT. • Actual hours are available for the weekly summary and the payroll.
Business rules	BR-06 (perfect attendance), BR-11 (actual hours for base pay)
Related FR / NFR	FR-13; NFR-12 (no location data; self-reported and owner-confirmed)
UI screen	Me (내 정보) and Home (worker); Home (owner)

UC-08 Compute weekly hours and holiday allowance

Table 21. Use-case description UC-08

ID	UC-08
Name	Compute weekly hours and holiday allowance
Actors	Primary: Owner (A2); the computation is performed by the system and is also included by UC-06 and UC-09.
Preconditions	• The owner is signed in (UC-13).
Trigger	The owner opens the weekly summary in the <i>Pay</i> tab, or UC-06 / UC-09 includes this use case.
Main flow	1. The owner opens the <i>Pay</i> tab, segment <i>Weekly summary</i>, and selects a week (identified by its Monday, BR-05). 2. For each worker who has a shift in the week, as current or as original worker, the system computes the contractual hours from the fixed schedule. 3. The system computes the scheduled hours from the shifts currently assigned to the worker in the week, and the actual hours from confirmed attendance. 4. The system determines perfect attendance from the shifts whose original worker is the worker, applying the workplace's substitution-attendance policy (BR-06). 5. The system determines holiday-allowance eligibility and holiday-allowance hours (BR-06, BR-07). 6. The system stores one weekly summary per worker and week, replacing an earlier one. 7. The system shows the table with the columns of FR-14 and, for each worker who is not eligible, a short reason such as "contract hours below 15" or "absent on Tue 09-29".

Alternative flows	• A1 Included by UC-06. Only the requester and the acceptor are computed, once with the current assignment (before) and once with the shift reassigned (after); after approval the result is stored. • A2 Included by UC-09. All weeks whose Sunday falls in the payroll month are computed before the payroll draft is built.
Exception flows	• E1 Week not generated. The selected week has no shifts, so no worker has a summary for it. The system says that there are no shifts in the week and offers to open the week on the schedule (UC-03).
Postconditions	• Weekly summaries of the week reflect the current shifts, attendance and settings.
Business rules	BR-05, BR-06, BR-07
Related FR / NFR	FR-14; NFR-03, NFR-10
UI screen	Pay (급여), weekly summary segment (owner); Home decision card

UC-09 Generate and confirm monthly payroll

Table 22. Use-case description UC-09

ID	UC-09
Name	Generate and confirm monthly payroll
Actors	Primary: Owner (A2).
Preconditions	• The owner is signed in (UC-13). • The minimum-wage table has a row for the year of the month or an earlier year.
Trigger	The end of a pay month, or the owner wants to check the pay of the current month.
Main flow	1. The owner opens the <i>Pay</i> tab, segment <i>Monthly payroll</i>, and selects a month. 2. The owner selects <i>Create September payroll draft</i> (the month named in the button). 3. The system recomputes the weekly summaries of the weeks whose Sunday falls in the month (includes UC-08). 4. For each worker with shifts or holiday pay in the month the system computes base hours and base pay (worked hours for WORKED shifts, scheduled hours for SCHEDULED shifts, nothing for ABSENT shifts), holiday pay, premium pay and total (BR-11, BR-08). 5. The system checks each worker's hourly wage against the applicable minimum wage, taking the probation reduction into account only when all its conditions hold (BR-09, BR-10). 6. The system stores one DRAFT row per worker, replacing earlier DRAFT rows of the month. 7. The system shows the table with the flags <i>estimated</i> and <i>below minimum wage</i>. 8. The owner acknowledges each minimum-wage warning; the acknowledgement is stored on the row (minWageAck = 1). 9. The owner selects <i>Confirm</i>. 10. The system sets the rows to CONFIRMED; they are read-only from now on.
Alternative flows	• A1 No warning. If no row has a minimum-wage warning, step 8 is skipped. • A2 Regenerate. The owner generates the month again before confirming; only DRAFT rows are replaced, so acknowledgements given on the old draft must be given again (BR-09).
Exception flows	• E1 Warning not acknowledged. At step 9 a warning is still unacknowledged. Confirmation is blocked and the row is highlighted. • E2 Month already confirmed. Confirmed rows are not replaced by regeneration and cannot be edited. • E3 Minimum wage not yet updated. The table has no row for the year of the month. The system uses the latest earlier year, which may be too low; the owner adds the new year in Settings (UC-10) and generates the draft again.

Postconditions	- Payroll rows for the month exist in DRAFT or CONFIRMED state. - No confirmed row has an unacknowledged minimum-wage warning.
Business rules	BR-07, BR-08, BR-09, BR-10, BR-11
Related FR / NFR	FR-15, FR-16; NFR-10
UI screen	Pay (급여), monthly payroll segment (owner); Home payroll-warning section

UC-10 Manage workplace settings and minimum wage table

Table 23. Use-case description UC-10

ID	UC-10
Name	Manage workplace settings and minimum wage table
Actors	Primary: Owner (A2).
Preconditions	The owner is signed in (UC-13).
Trigger	The store changes (name, number of regular employees), the owner decides the substitution-attendance policy, or a new minimum wage is announced.
Main flow	- The owner opens <i>Settings</i> from the header menu. - The system shows store name, number of regular employees, substitution-attendance policy (EXCUSED or ABSENT) and the minimum-wage table by year. - The owner changes a value or adds a minimum-wage row (year, hourly amount). - The system validates the input and stores it. - The system confirms the change; computations made afterwards use the new values.
Alternative flows	A1 Update an existing year. The owner changes the amount of an existing year; the row is updated.
Exception flows	E1 Invalid value. The number of regular employees is negative, or the hourly amount is not a positive integer. The system refuses the value and explains the valid range.
Postconditions	- Settings and minimum-wage table are stored. - Existing CONFIRMED payroll rows are not changed.
Business rules	BR-06 (policy), BR-08 (regular employees), BR-09 (minimum wage)
Related FR / NFR	FR-17; NFR-10
UI screen	Settings (설정), from the header menu (owner)

UC-11 Expire overdue substitute request

Table 24. Use-case description UC-11

ID	UC-11
Name	Expire overdue substitute request
Actors	Primary: System Clock (A3). Secondary: requester (A1), candidates (A1) and Owner (A2), who are notified.
Preconditions	At least one request is REQUESTED or ACCEPTED.
Trigger	The application starts, the user navigates to another screen, or the demo clock is changed.

Main flow	1. The system reads the current time from the clock. 2. The system selects all REQUESTED or ACCEPTED requests whose deadline $\leq$ now or whose shift start $\leq$ now (BR-12). 3. For each selected request the system sets the status to EXPIRED and closes its PENDING targets without notifying them. 4. The system sends REQUEST_EXPIRED to the requester and the owner.
Alternative flows	• A1 Nothing overdue. At step 2 no request qualifies; the use case ends without change.
Exception flows	• E1 Accepted but not approved. A request that was ACCEPTED but not decided by the owner before the deadline also expires; the swap does not take place and the shift stays with the requester.
Postconditions	• No open request has a deadline or shift start in the past. • The shifts of expired requests are unchanged.
Business rules	BR-12
Related FR / NFR	FR-18, FR-21
UI screen	None of its own (effects visible on Home, in the Swaps tab and in the notification list)

UC-12 Cancel own substitute request

Table 25. Use-case description UC-12

ID	UC-12
Name	Cancel own substitute request
Actors	Primary: Worker (A1) as requester. Secondary: candidates (A1), who are notified.
Preconditions	• The requester is signed in (UC-13). • The requester has a request in state REQUESTED or ACCEPTED.
Trigger	The requester can work the shift after all, or has arranged the matter otherwise.
Main flow	1. The requester opens the <i>Swaps</i> tab, section "sent requests" (or Home). 2. The system shows each open request as a progress tracker with the candidates' answers so far. 3. The requester selects <i>Cancel request</i> on an open request. 4. The system sets the request to CANCELLED and sets every PENDING target to CLOSED. 5. The system sends REQUEST_CANCELLED to every target that was PENDING or ACCEPTED; targets already DECLINED or CLOSED are not notified. 6. The list shows the request as CANCELLED; the shift stays with the requester.
Alternative flows	• A1 Accepted request. If the request was ACCEPTED, the acceptor's target keeps the response ACCEPTED, and the acceptor receives the REQUEST_CANCELLED notification, so the acceptor knows that the swap will not take place.
Exception flows	• E1 Request already decided. The request is APPROVED, REJECTED or EXPIRED. The <i>Cancel</i> action is not offered.
Postconditions	• The request is CANCELLED and no target is PENDING.
Business rules	BR-03 (only one open request per shift; cancelling frees the shift for a new request)
Related FR / NFR	FR-19, FR-21
UI screen	Swaps (대체) tab, sent requests; Home (worker)

UC-13 Sign in by role (demo account selection)

Table 26. Use-case description UC-13

ID	UC-13
Name	Sign in by role (demo account selection)
Actors	Primary: Worker (A1) and Owner (A2).
Preconditions	The application is loaded in the browser.
Trigger	A user opens the application or signs out.
Main flow	- The system shows the sign-in screen as a demo guide: a title, a note that the demo has no passwords, a three-step tour card (① view Lee Seoyeon's request, ② accept it as Choi Minhoo, ③ approve it as the owner Park Jiyoung), and the list of all five accounts with role and one-line summary. - The user selects a tour step, which signs in as that person and opens the screen where the step happens, or selects any account from the list. - The system keeps the chosen account for the browser session and shows only the navigation of that role (FR-20).
Alternative flows	A1 Switch account or sign out. The user selects <i>View as someone else</i> in the header or <i>Sign out</i> in the menu; the account list opens without changing any data. A2 Reset demo data. In the <i>Demo tools</i> panel the user selects <i>Reset demo data</i> and confirms; the system restores the seed data and the demo clock (NFR-05).
Exception flows	E1 Session ended. A new browser tab or session has no chosen account; the sign-in screen is shown.
Postconditions	A user with a role is active, and the menus shown correspond to that role.
Business rules	—
Related FR / NFR	FR-20; NFR-11
UI screen	Sign-in demo guide; header account switch; Demo tools panel

3.6 Business Rules

The business rules are the domain requirements of ShiftSwap. They state precisely, with conditions and numbers, what the functional requirements refer to by name: which workers are eligible, when an acceptance succeeds, when the holiday allowance is owed, and how pay is computed. The lecture warns that a system that does not satisfy its domain requirements may be unworkable [7]; for this system, a wrong rule means a wrong pay slip. The rules are therefore written so that each can be implemented as one pure function in the domain-rule module (Chapter 4) and tested on its own (Section 3.10).

The table below lists the twelve rules. The column "Rule" repeats the wording of the specification. The column "Rationale" explains why the rule has this form, and the column "Source" names the law or the store practice it comes from.

Table 27. Business rules with rationale and source

ID	Rule	Rationale	Source
BR-01	Eligible candidates. A worker W is eligible for a request on shift S (date d, start s, end e) iff: W is active; W ≠ requester; W has no shift on date d overlapping [s, e); W has an availability slot on weekday(d) with slotStart ≤ s and slotEnd ≥ e.	Only workers who can actually take the shift are asked, which removes the owner's search (P1). The slot must cover the whole shift; partial coverage would lead to a second swap. Half-open intervals let back-to-back shifts (e.g. 12:00–18:00 and 18:00–23:00) coexist.	Store practice (Section 2.1, steps 2–3)

ID	Rule	Rationale	Source
BR-02	First acceptance wins. Acceptance succeeds only if the request is REQUESTED at the moment of the transaction and the candidate still has no overlapping shift on that date (re-checked at acceptance and again at approval, error ACCEPTOR_BUSY); it sets request.status = ACCEPTED, request.acceptorId = W, target(W).response = ACCEPTED, and all other PENDING targets → CLOSED.	A broadcast request can be answered by several candidates at nearly the same time. The rule makes the outcome deterministic and avoids the double agreement observed in the current process. The state check must be inside the transaction; a check before it would allow a race.	Store practice (double agreement, Section 2.1.2); NFR-04
BR-03	Request validity. Only the shift's current worker can request; shift must be SCHEDULED and start in the future; at most one open (REQUESTED/ACCEPTED) request per shift; deadline must be after now and no later than the shift start.	Prevents requests on someone else's shift, on past or completed shifts, and competing requests for one shift. A deadline after the shift start would be meaningless because the shift would already have to be covered.	Workflow consistency
BR-04	Approval effect. Approve: shift.workerId = acceptor; shift.originalWorkerId keeps the first worker (set only if null); request → APPROVED. Reject: request → REJECTED, shift unchanged. Owner can approve only an ACCEPTED request.	The owner stays responsible for the schedule, as in the current process. Keeping the first worker as original worker lets the holiday-allowance rule count a given-away shift for the right person, even after a chain of swaps.	Store practice (owner decides); BR-06 needs the original worker
BR-05	Week. A week runs Monday 00:00 to Sunday 24:00 (ISO week). Weekly records are keyed by the Monday date.	The weekly holiday allowance is defined per week; a fixed week definition makes weekly summaries unambiguous and gives each one a single key.	ISO 8601 week [11]; Labor Standards Act Art. 55 [3]
BR-06	Holiday-allowance eligibility (주휴). Eligible iff (a) weekly contractual hours ≥ 15 , and (b) perfect attendance: every shift of the week whose <i>original</i> worker is W and that was not given away was not ABSENT (a shift that is still SCHEDULED does not break attendance, so the weekly result is a projection until the week is over). Shifts W gave away by an APPROVED substitution count according to Workplace.subAttendancePolicy: EXCUSED (default — the working day was changed by agreement, so it does not break attendance) or ABSENT (it breaks attendance). Contractual hours = sum of W's fixed-schedule hours per week. Substitute shifts W took do not change contractual hours.	Implements the two legal conditions. Contractual hours come from the contract, not from the shifts actually worked, so taking extra substitute shifts cannot create an entitlement and giving one away cannot remove it. The law averages 15 h over four weeks; the fixed schedule is constant, so the average equals the weekly value. The treatment of a handed-over shift is not stated in the law and is therefore a setting.	Labor Standards Act Art. 55 ①, Enforcement Decree Art. 30 ①, Art. 18 ③ [3]; [5]
BR-07	Holiday-allowance hours. hours = $\min(\text{contractualHours}, 40) / 40 \times 8$, rounded to 2 decimals; pay = hours $\times$ Worker.hourlyWage, rounded down to the won per week (monthly holiday pay is the sum of the rounded weekly amounts).	Proportional paid holiday for part-time workers, capped at one full 8-hour day. Rounding down money avoids overstating an amount by fractions of a won; hours are rounded to 2 decimals so that the displayed and the computed value agree.	[5]; Labor Standards Act Art. 18 ③ [3]

ID	Rule	Rationale	Source
BR-08	Premium pay. Applies only if $\text{Workplace.regularEmployees} \geq 5$. Then +50 % of wage for (i) daily hours beyond 8, (ii) weekly hours beyond 40 not already counted in (i), (iii) night hours 22:00–06:00. Night premium adds to overtime premium. If $\text{regularEmployees} < 5 \rightarrow \text{premium} = 0$. In a monthly payroll, weekly overtime uses only the days of that ISO week that fall inside the month. Premium pay = premium hours $\times$ wage $\times$ 0.5, rounded down.	Small stores below the threshold are exempt, but a store can grow; making the number of employees a setting keeps the rule correct in both cases. Counting weekly overtime only beyond what daily overtime already covers avoids paying the same hour twice; restricting a week to its days inside the month keeps each hour in one payroll.	Labor Standards Act Art. 11, Art. 56 [3]
BR-09	Minimum-wage check. Warn if $\text{worker.hourlyWage} < \text{MinimumWage}(\text{year}).\text{hourly} \times (0.9 \text{ if BR-10 applies else } 1)$. The payroll still computes with the worker's wage; the warning is shown on the payroll row and blocks confirmation until the owner acknowledges it, which sets $\text{Payroll.minWageAck} = 1$. Re-generating a DRAFT month replaces its rows, so acknowledgements must be given again.	The system cannot change a contract, but it must stop an unlawful wage from being paid unnoticed (P3). Blocking confirmation until acknowledgement forces a conscious decision without hiding the actual amount. Storing the acknowledgement on the row keeps it across reloads; because a regenerated draft may contain different amounts, it must be acknowledged again.	Minimum Wage Act; 2026 notice [1], [2]
BR-10	Probation reduction. The minimum may be reduced to 90 % only if all hold: contract length ≥ 1 year ($\text{contractEnd} - \text{contractStart} \geq 365$ days, or contractEnd empty = open-ended), the date is within 3 months of contractStart and $\leq \text{probationEnd}$, and $\text{simpleLabor} = \text{false}$. In a monthly payroll the conditions are evaluated on the first day of the month, or on contractStart if later. "Within 3 months" means from contractStart (inclusive) to the same calendar date three months later (exclusive); a worker without probationEnd is never on probation.	All legal conditions are required together; each is a separate field of the worker record so that the rule can be checked mechanically. An open-ended contract counts as one year or more. A monthly payroll needs one evaluation date; the first day of the month (or the contract start, if later) is the date on which the month's wage begins to apply.	Minimum Wage Act Art. 5 ②, Enforcement Decree Art. 3 [4]
BR-11	Monthly payroll. Month = calendar month. Base pay = Σ actual hours of WORKED shifts in the month $\times$ wage (recorded clock-in/out); SCHEDULED shifts in the month count with their scheduled hours and set estimated = 1 on the row; ABSENT shifts count 0. Holiday allowance = Σ holiday pay of weeks whose Sunday falls in the month. Total = base + holiday + premium. Amounts are integers in KRW, rounded down.	Wages are paid monthly while the holiday allowance is weekly; assigning each week to the month of its Sunday makes every week count exactly once. The "estimated" flag lets the owner check pay during the month without mistaking it for the final amount; an absent shift earns nothing.	Store practice (monthly pay)
BR-12	Expiry. A REQUESTED or ACCEPTED request whose deadline $\leq$ now, or whose shift start $\leq$ now, becomes EXPIRED.	An open request must not remain open after it can no longer be useful. Expiring ACCEPTED requests as well means that an owner who does not decide in time leaves the shift with the original worker, which is the safe default.	Workflow consistency

A worked example shows how BR-06 and BR-07 combine. Choi Minho has fixed shifts on Tuesday and Thursday 18:00–23:00 and Saturday 12:00–18:00, which is $5 + 5 + 6 = 16$ contractual hours per week. Since $16 \geq 15$, condition (a) holds. If he attended all his shifts, his holiday-allowance hours are $\min(16, 40) / 40 \times 8 = 3.2$ hours, and at his wage of KRW 10,320 the holiday pay is $3.2 \times 10,320 = \text{KRW } 33,024$. If he also takes Lee Seoyeon's Wednesday shift, his scheduled hours for that week rise from 16 to 21, but his contractual hours and his holiday pay stay the same. Lee Seoyeon's contractual hours are 10, so she is not eligible either way. This example is used throughout the report and is reproduced by the application.

3.7 Non-Functional Requirements

Non-functional requirements are constraints on the services of the system, such as timing, reliability, process and standards, and they often apply to the system as a whole [7]. Following the classification used in the lecture, the table below groups them into product requirements (how the delivered product must behave), organisational requirements (consequences of the policies of the developing organisation, here the course) and external requirements (factors outside the system and its development, here the law). Each requirement is stated so that it can be verified; the last column names the verification method used in Chapter 6.

Table 28. Non-functional requirements (Product / Organisational / External)

ID	Class	Sub-class	Requirement	Verification
NFR-01	Product	Usability	A candidate can accept a request in at most 2 clicks from the inbox.	UI walkthrough
NFR-02	Product	Usability	All screens are usable at 360 px width (mobile) and 1440 px (desktop) without horizontal scrolling.	Browser check at both widths
NFR-03	Product	Efficiency / Performance	With 20 workers and 12 weeks of shifts, any screen renders in under 1 second on a mid-range laptop.	Timed load with seeded data
NFR-04	Product	Reliability	"First acceptance wins" is enforced inside one database transaction that re-checks the request state; two acceptances can never both succeed.	Unit test TC-05x
NFR-05	Product	Reliability	Data persists across page reloads (browser storage) and can be reset to seed data.	Manual test
NFR-06	Product	Portability	Runs in current Chrome, Edge, Safari, Firefox without installation or server.	Manual test
NFR-07	Organisational	Delivery	Working system deployed by Week 14 (2026-12-04).	Deployed URL
NFR-08	Organisational	Implementation	Static web application (HTML/CSS/JavaScript ES modules) with an embedded SQLite database (sql.js, WebAssembly); built by vibe coding from the final prompt; hosted on GitHub Pages.	Repository
NFR-09	Organisational	Standards	Models follow UML 2.5 notation; code identifiers in English.	Review
NFR-10	External	Legislative	Pay rules follow the Minimum Wage Act (2026 minimum hourly wage KRW 10,320) and the Labor Standards Act (Art. 18 ☹, Art. 55, Enforcement Decree Art. 30; Art. 11 / 56 premium threshold).	Unit tests TC-08x, TC-09x
NFR-11	External	Privacy	A worker's phone number and wage are visible only to the owner and that worker (Personal Information Protection Act principle of minimum disclosure).	Role test
NFR-12	External	Ethical	The system does not collect location data; attendance is self-reported and owner-confirmed.	Design review
NFR-13	Product	Usability	Task-first design: each role starts on a Home screen that lists the items waiting for that person with the action on the item; every screen has exactly one primary action; any action that changes hours or pay shows its effect (hours before → after, holiday-allowance amount) before the user commits; the default UI language is Korean with a complete English alternative.	Headless check counts one primary button per screen; walkthrough

Several of these requirements shape the design more than any single functional requirement. NFR-08 fixes the technology: without a server, all data lives in the browser, which is why NFR-05 requires persistence in browser storage and a reset to seed data, and why sign-in is a choice of demo account (FR-20) rather than a password check. NFR-04 turns the business rule BR-02 into a reliability requirement with a concrete mechanism, one transaction that re-checks the state. NFR-01 and NFR-02 follow from the users: candidates answer on their phones, often between classes, and the owner works both on a phone and on a computer. NFR-13 was added in iteration 4 (Chapter 5), after a usability review of the running application: it turns the design principles of the final user interface — a Home screen per role that lists what is waiting, one primary ac-

tion per screen, the effect on hours and pay shown before the user commits, and Korean as the default language — into a checkable requirement. NFR-12 records a deliberate choice. Location-based clock-in would make attendance harder to falsify, but it would collect personal data that the problem does not require, and the reference store does not use it today.

The lecture observes that non-functional requirements may be more critical than functional ones, because a system that violates them may be useless [7]. For ShiftSwap the most critical one is NFR-10: a system that computes pay faster but not lawfully would make problem P3 worse, not better. For this reason NFR-10 is verified by unit tests at the rule level rather than by inspection of screens.

3.8 Scenarios

A scenario is a real-life example of how the system can be used, and according to the lecture it should include the starting situation, the normal flow of events, what can go wrong, other concurrent activities, and the state when the scenario finishes [7]. The seven scenarios below use the demonstration data of the specification: Dalbit Café (regular employees 4, substitution-attendance policy EXCUSED), the owner Park Jiyoung and the workers Lee Seoyeon, Choi Minho, Jung Hana and Kang Doyun. The first table below summarises the workers; the demo clock starts at Monday 2026-09-28 09:00, and the weeks beginning 2026-09-21 and 2026-09-28 have been generated, the first with confirmed attendance for all shifts. Together the scenarios cover every use case at least once and include the concurrent acceptance and the expiry of a request.

Table 29. Workers of the demonstration data (seed)

Worker (id)	Wage (KRW)	Contract	Fixed shifts (contractual h/week)	Availability
Lee Seoyeon (2)	10,500	2026-03-02 ~ 2027-02-28	Mon, Wed 18:00–23:00 (10 h)	Tue 17:00–23:00; Thu 17:00–23:00; Sat 10:00–22:00
Choi Minho (3)	10,320	2026-06-01 ~ 2026-12-31	Tue, Thu 18:00–23:00; Sat 12:00–18:00 (16 h)	Mon 17:00–23:00; Wed 17:00–23:00
Jung Hana (4)	10,320	2026-09-01 ~ open-ended; probation end 2026-11-30; not simple labour	Fri 17:00–23:00; Sun 10:00–16:00 (12 h)	Mon 18:00–23:00; Sat 10:00–18:00
Kang Doyun (5)	10,000	2026-08-15 ~ 2026-11-15; simple labour	Sat 10:00–16:00; Sun 16:00–22:00 (12 h)	Wed 18:00–23:00; Thu 18:00–23:00; Fri 17:00–23:00

S-1 Requesting a substitute for an exam evening (UC-13, UC-04)

Starting situation. Lee Seoyeon has a midterm exam on Thursday morning and cannot work her fixed shift on Wednesday 2026-09-30, 18:00–23:00. No request exists for that shift.

Normal flow. Seoyeon opens ShiftSwap on her phone and signs in by choosing her demo account. In the *Swaps* tab she starts *Find a substitute*, chooses her Wednesday shift in step 1, gives the reason "Midterm exam" and sets the deadline to Tuesday 2026-09-29 21:00 in step 2, and in step 3 sees that the request will go to Choi Minho and Kang Doyun. The system checks the request (BR-03) and then checks every worker against BR-01. Choi Minho is available on Wednesday 17:00–23:00 and has no shift that day, so he is eligible. Kang Doyun is available on Wednesday 18:00–23:00 and has no shift that day, so he is eligible too. Jung Hana has no availability on Wednesday and is not eligible. The system creates the request with two targets and puts a notification in Minho's and Doyun's inboxes. Seoyeon sees the result screen with the tracker at "Requested" and the two recipients, and the Wednesday shift on the schedule now has a dashed amber outline.

What can go wrong. If Seoyeon had tried to set the deadline after the shift start, the flow would have refused it and shown the allowed range. If she had already had an open request for the shift, step 1 would have shown the shift disabled with the reason.

Concurrent activities. None; the owner is not involved at this stage and receives no notification because candidates exist.

Final state. Request REQUESTED with targets for Minho and Doyun, both PENDING. This is the state stored in the seed data.

S-2 Two candidates accept at nearly the same time (UC-05)

Starting situation. The request of S-1 is REQUESTED. Choi Minhø and Kang Doyun both see the request card on their Home screens.

Normal flow. Minhø taps *Accept* on the request card on his Home screen, which also tells him that his week will go from 16 to 21 scheduled hours with the holiday allowance unchanged (NFR-01, NFR-13). In one transaction the system reads the request, finds it REQUESTED, sets it to ACCEPTED with Minhø as acceptor, sets Minhø's target to ACCEPTED and Doyun's target to CLOSED. Seoyeon and the owner Park Jiyoung receive REQUEST_ACCEPTED; Doyun receives TARGET_CLOSED.

Concurrent activities. Doyun had opened his Home screen before Minhø's acceptance and still sees the *Accept* button. He taps it a moment after Minhø. His transaction reads the request again, finds it ACCEPTED instead of REQUESTED, and changes nothing. His card now says that Minhø was faster and closes.

What can go wrong. Without the re-check inside the transaction, both acceptances could succeed and two workers would come to the same shift — the double agreement of the current process. BR-02 and NFR-04 exclude this, and test group TC-05x tests it.

Final state. Request ACCEPTED, acceptor Choi Minhø; Minhø's target ACCEPTED, Doyun's CLOSED.

S-3 The owner approves the swap after checking its effect (UC-06, UC-08)

Starting situation. The request of S-2 is ACCEPTED. Park Jiyoung has an unread REQUEST_ACCEPTED notification.

Normal flow. Jiyoung signs in as owner. Her Home screen starts with a decision card for Seoyeon's request, with one impact line per person; *Details* opens the full comparison for the week of 2026-09-28:

Table 30. Before/after comparison shown to the owner in scenario S-3

Worker	Contractual h	Scheduled h (before → after)	Eligible (before → after)	Holiday h
Lee Seoyeon (requester)	10	10 → 5	no → no	0
Choi Minhø (acceptor)	16	16 → 21	yes → yes	3.2

No eligibility changes, so no line is highlighted. Jiyoung selects *Approve*. The system assigns the Wednesday shift to Minhø, records Seoyeon as its original worker, sets the request to APPROVED, recomputes both weekly summaries and notifies both workers. On the weekly board the shift now shows Minhø's name in bold, Seoyeon's name struck through, and the amber handover edge.

What can go wrong. If Jiyoung had waited until after Tuesday 21:00, the request would have expired (S-4 shows this case) and *Approve* would not be offered. The demonstration walkthrough S7 of the specification shows the case in which the comparison changes the decision. With the policy set to ABSENT, Choi Minhø requests a substitute for his Saturday 2026-10-03 shift, 12:00–18:00; Lee Seoyeon and Jung Hana are eligible, and Kang Doyun is excluded because his own Saturday 10:00–16:00 shift overlaps. When Seoyeon accepts, the panel shows Minhø's eligibility changing from yes to no and highlights that line, because under the ABSENT policy a given-away shift breaks his perfect attendance for the week of 2026-09-28.

Concurrent activities. Seoyeon could cancel the request (UC-12) while Jiyoung has the card open; the approval would then be refused because the request is no longer ACCEPTED.

Final state. Request APPROVED; shift worker Choi Minhø, original worker Lee Seoyeon; Minhø's holiday allowance for the week remains 3.2 h (KRW 33,024).

S-4 A request expires without an answer (UC-04, UC-11)

Starting situation. The demo clock shows Monday 2026-09-28 09:00. Jung Hana cannot work her Friday shift on 2026-10-02, 17:00–23:00.

Normal flow. Hana requests a substitute without changing the proposed deadline, which is the earlier of now + 24 h and the shift start, i.e. Tuesday 2026-09-29 09:00. Under BR-01 only Kang Doyun is eligible: he is available on Friday 17:00–23:00 and has no Friday shift. Choi Minhø and Lee Seoyeon have no Friday availability. Doyun receives the

notification but does not respond. On Tuesday the owner uses *+1 day* in the *Demo tools* panel, which moves the clock to 2026-09-29 09:00. The System Clock actor runs the expiry check (UC-11): the deadline is $\leq$ now, so the request becomes EXPIRED, Doyun's target is closed, and Hana and the owner receive REQUEST_EXPIRED.

What can go wrong. If Doyun had accepted but the owner had not approved before the deadline, the request would expire in state ACCEPTED in the same way (BR-12), and the shift would stay with Hana.

Concurrent activities. The request of S-1 to S-3 is independent; each request expires on its own deadline.

Final state. Request EXPIRED; the Friday shift is still Hana's. Hana may create a new request for the same shift, because no open request exists any more.

S-5 No worker is eligible (UC-04 alternative flow, UC-12)

Starting situation. Kang Doyun cannot work his shift on Sunday 2026-10-04, 16:00–22:00.

Normal flow. Doyun creates a request. No worker has availability on Sunday, so BR-01 returns no candidates. The system still creates the request (FR-08), sends NO_CANDIDATE to the owner, and tells Doyun that no eligible worker was found. The owner can now handle the case personally, as in the current process, but knows at once that asking around is unnecessary. Later Doyun finds that he can work after all and cancels the request in the *Swaps* tab (UC-12); it becomes CANCELLED.

What can go wrong. If Doyun forgets to cancel, the request expires at its deadline (BR-12); either way it does not stay open.

Concurrent activities. Jung Hana works Sunday 10:00–16:00; her shift ends when Doyun's begins and does not overlap it, but she has no Sunday availability and is not a candidate.

Final state. Request CANCELLED with no targets; the owner's inbox holds the NO_CANDIDATE notification.

S-6 Reviewing the weekly holiday allowance (UC-07, UC-08)

Starting situation. All shifts of the week of 2026-09-21 have confirmed attendance (seed data). The owner wants to know who is owed a weekly holiday allowance for that week.

Normal flow. Jiyoung opens the weekly summary in the *Pay* tab and selects the week of 2026-09-21. The system computes, for each worker, contractual, scheduled and actual hours and applies BR-06 and BR-07. Lee Seoyeon (10 h), Jung Hana (12 h) and Kang Doyun (12 h) are below 15 contractual hours; each row shows "contract hours below 15". Choi Minhø (16 h) worked all his shifts and is eligible with 3.2 holiday hours.

What can go wrong. Suppose instead that Minhø had not come on Tuesday 2026-09-22 and the owner had marked that shift ABSENT (UC-07). Perfect attendance would fail, Minhø would not be eligible, and his row would show "absent on Tue 09-22".

Concurrent activities. A worker may still be recording attendance for the current week; this does not affect the closed week of 2026-09-21.

Final state. Weekly summaries of the week of 2026-09-21 stored for all four workers; only Minhø is eligible.

S-7 Monthly payroll with a minimum-wage warning (UC-09, UC-10)

Starting situation. The owner prepares the September 2026 payroll. The minimum-wage table holds 10,030 for 2025 and 10,320 for 2026.

Normal flow. Jiyoung opens the monthly payroll in the *Pay* tab, selects September 2026 and creates the draft. Among the generated weeks, only the week of 2026-09-21 has its Sunday in September, so its holiday pay — KRW 33,024 for Minhø — is included in September (BR-11). Premium pay is zero for everyone, because the workplace has 4 regular employees (BR-08). The system then checks each wage. Kang Doyun earns KRW 10,000, below the 2026 minimum of 10,320; the probation reduction does not apply to him, because his contract (2026-08-15 to 2026-11-15) is shorter than one year and his work is simple labour (BR-10). His row therefore carries a minimum-wage warning. Jung Hana's contract is open-ended, she is within three months of her contract start and before her probation end, and her work is not simple labour, so all BR-10 conditions hold and her threshold is 90 % of 10,320 = KRW 9,288; her wage of 10,320 is above it. Seoyeon (10,500) and Minhø (10,320) are not below the minimum. Jiyoung acknowledges Doyun's warning,

which is stored on his row (`minWageAck = 1`), decides to correct his wage with him, and confirms the payroll; the rows become read-only.

What can go wrong. If Jiyoung tries to confirm without acknowledging the warning, confirmation is blocked (BR-09). If she generates the draft before attendance for 28–30 September is recorded, the base pay for those days is computed from scheduled hours and the affected rows are flagged "estimated".

Concurrent activities. If the café hired a fifth regular employee and the owner changed the setting in Settings (UC-10) before generating the draft, premium pay would apply; each seed evening shift from 18:00 to 23:00 contains one night hour (22:00–23:00), which would then earn an additional 50 % of the wage.

Final state. September 2026 payroll CONFIRMED for four workers; Doyun's minimum-wage warning acknowledged.

The table below shows which use cases each scenario exercises. Every use case appears in at least one scenario, and the Must use cases appear in several.

Table 31. Coverage of use cases by scenarios

Scenario	UC-01	02	03	04	05	06	07	08	09	10	11	12	13
S-1	(seed)	(seed)	(seed)	●									●
S-2					●								●
S-3						●		●					●
S-4				●							●		●
S-5				●								●	●
S-6							●	●					●
S-7								●	●	●			●

UC-01, UC-02 and UC-03 are exercised through the seed data, which is the result of registering four workers with their fixed schedules and availability and generating two weeks. The user guide in Chapter 7 walks through these three use cases on the screens.

3.9 Analysis Models

The use-case descriptions state the flow of each use case, but three parts of the system are easier to check in a diagram: the candidate selection of BR-01, the holiday-allowance decision of BR-06 and BR-07, and the life cycle of a substitute request. The lecture lists activity diagrams and state diagrams among the supplementary diagrams of an SRS [7]. This section presents these three analysis models. The class, sequence and remaining state diagrams belong to the design and are presented in Chapter 4.

3.9.1 Activity diagram AD-2: candidate selection

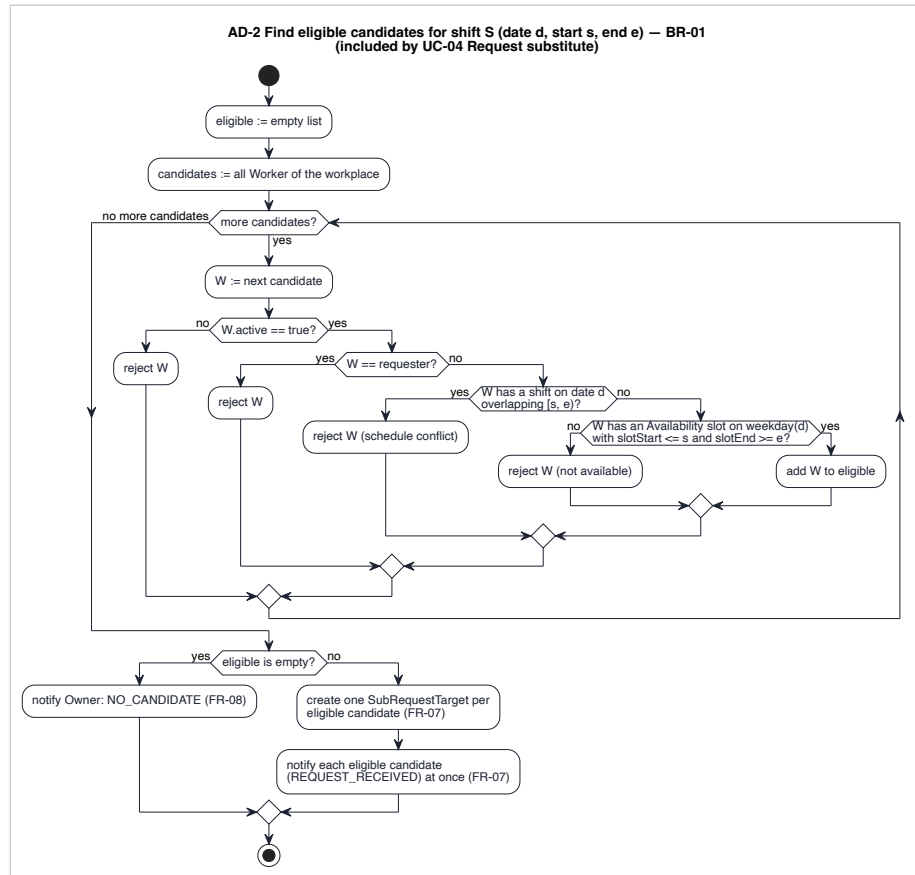


Figure 3. Activity diagram AD-2 — find eligible candidates for a shift (BR-01), included by UC-04

The figure above shows the activity included by UC-04. The system starts with an empty list and examines every worker of the workplace in turn. A worker is rejected at the first failed condition, in the order of BR-01: inactive, the requester, a conflicting shift on the date of the request, or no availability slot on that weekday that covers the whole shift. A worker who passes all four checks is added to the list. After the loop the diagram branches: an empty list leads to the NO_CANDIDATE notification to the owner (FR-08); otherwise one target is created per candidate and all candidates are notified at once (FR-07). The notification step is outside the loop, which is what makes the notification simultaneous rather than sequential — the difference between the new process and the loop of processes 2.0 and 3.0 in the data flow diagram of Section 2.1.3.

The table below traces the activity for the seed request of scenario S-1 (Wednesday 2026-09-30, 18:00–23:00). The owner Park Jiyoung is also a row of the table Worker; she has no availability slots and is therefore never eligible.

Table 32. Trace of AD-2 for the seed request (Wednesday 2026-09-30, 18:00–23:00)

Worker	Active	Not requester	No conflicting shift	Availability covers 18:00–23:00	Result
Park Jiyoung (owner)	yes	yes	yes	no slot	rejected
Lee Seoyeon	yes	no	—	—	rejected
Choi Minho	yes	yes	yes	yes (Wed 17:00–23:00)	eligible
Jung Hana	yes	yes	yes	no (Mon, Sat only)	rejected
Kang Doyun	yes	yes	yes	yes (Wed 18:00–23:00)	eligible

3.9.2 Activity diagram AD-1: holiday-allowance determination

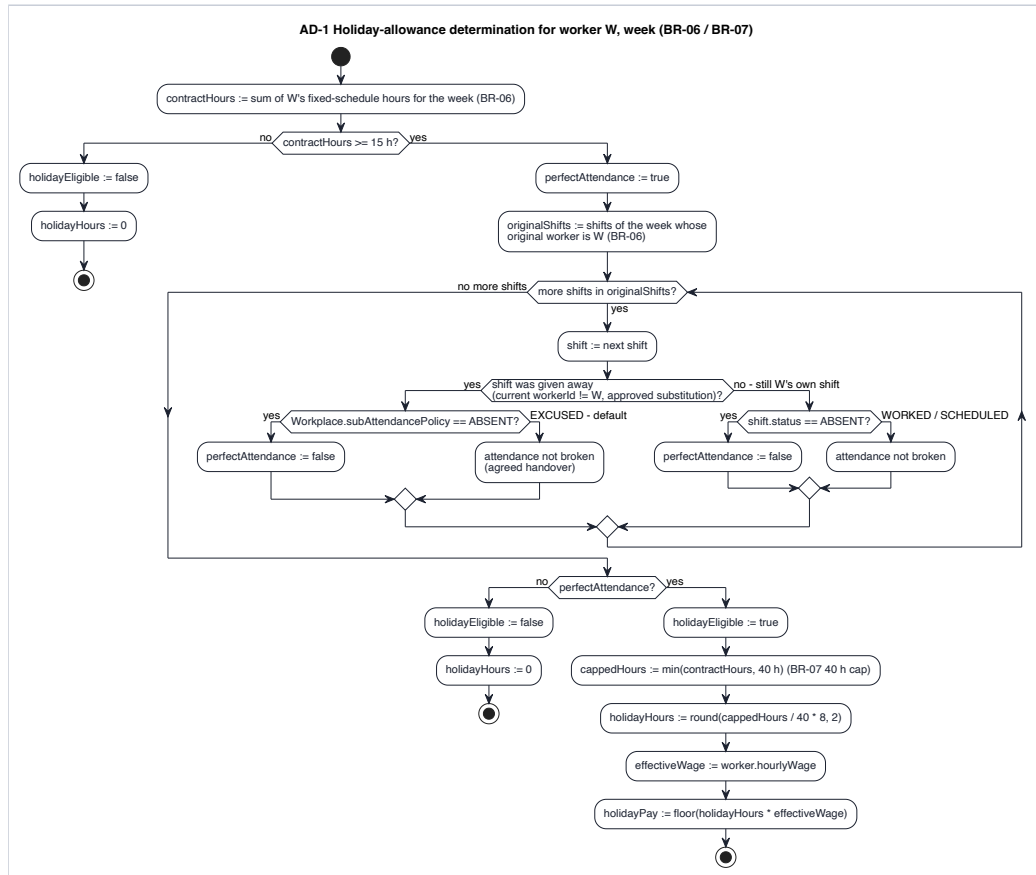


Figure 4. Activity diagram AD-1 — holiday-allowance determination for one worker and week (BR-06, BR-07)

The figure above shows the decision of BR-06 and BR-07 for one worker and one week. The first decision tests the contractual hours against 15; a worker below the threshold is not eligible, and no attendance is examined. For a worker at or above the threshold, the loop examines every shift of the week whose *original* worker is the worker. A shift that was given away by an approved substitution is judged by the workplace policy: under EXCUSED it does not break attendance, under ABSENT it does. A shift still held by the worker breaks attendance only if it is ABSENT. A shift that is WORKED or still SCHEDULED does not break it, so the weekly summary of the current week can be shown before the week is over and is updated if the owner later marks an absence. If attendance is perfect, the hours are computed with the 40-hour cap and rounded to two decimals, and the pay is rounded down to the won.

The diagram makes two properties of the rule easy to check with the users. First, substitute shifts that the worker *took* do not appear in the diagram at all: they affect scheduled hours but not the holiday allowance, which is the reason Choi Minho's allowance in scenario S-3 does not change. Second, the only place where the policy setting matters is one branch inside the loop, which shows that the setting cannot affect a worker below 15 hours.

3.9.3 State diagram: substitute request

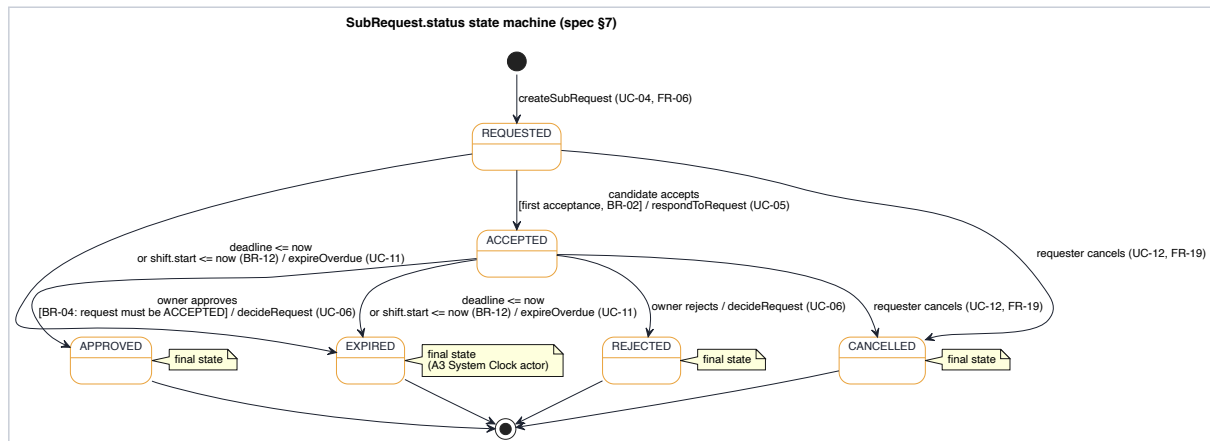


Figure 5. State machine diagram of a substitute request (SubRequest.status)

The figure above shows the life cycle of a substitute request. A request is created in REQUESTED (UC-04). The first acceptance moves it to ACCEPTED (UC-05, BR-02); the owner's decision moves it to APPROVED or REJECTED (UC-06, BR-04). From both open states, the System Clock can move it to EXPIRED (UC-11, BR-12) and the requester can move it to CANCELLED (UC-12). The four states APPROVED, REJECTED, EXPIRED and CANCELLED are final. Each transition is labelled with the event, the guard in brackets where there is one, and the operation that performs it.

The lecture gives three rules for state diagrams used in a requirements specification: all states must be domain significant, all sequences from the scenarios must be allowed, and all prohibited sequences must not be allowed [7]. The lecture also gives the general rules that there is one initial state, every state is reachable from it, every state has a path to a stop state, and every transition is labelled with an event [6]. The table below checks the diagram against these rules.

Table 33. Check of the SubRequest state diagram against the lecture's rules

Rule	Check
One initial state; every state reachable; path to a stop state from every state	Satisfied. The initial transition leads to REQUESTED; every other state is reachable from REQUESTED; the four final states lead to the stop state, and both open states have transitions to final states.
Every transition labelled with an event	Satisfied. The initial transition and each of the seven transitions between states are labelled (e.g. "candidate accepts [first acceptance, BR-02]").
All states domain significant	Satisfied. Each state corresponds to a situation the owner and workers talk about ("asked", "someone took it", "approved", "rejected", "too late", "withdrawn"). The internal state of targets (PENDING, ACCEPTED, DECLINED, CLOSED) is kept out of this diagram.
All scenario sequences allowed	Satisfied. S-1-S-3: REQUESTED → ACCEPTED → APPROVED. S-4: REQUESTED → EXPIRED (and ACCEPTED → EXPIRED in its variant). S-5: REQUESTED → CANCELLED. UC-06 alternative flow A1: ACCEPTED → REJECTED.
Prohibited sequences not allowed	Satisfied. There is no transition REQUESTED → APPROVED (the owner cannot approve without an acceptor, BR-04), no transition ACCEPTED → ACCEPTED (a second acceptance cannot replace the first, BR-02), and no transition leaving a final state (an approved swap cannot be cancelled or expire).

3.10 Requirements Validation

Requirements validation demonstrates that the requirements define the system the customer really wants; the lecture stresses that fixing a requirements error after delivery can cost up to 100 times as much as fixing an implementation error [7]. The lecture names five checks — validity, consistency, completeness, realism and verifiability — and three techniques — requirements reviews, prototyping and test-case generation. ShiftSwap used all three techniques; they are carried out within the requirements package (WBS 3), the UI design package (4.2) and the testing package (6).

3.10.1 Requirements reviews

Reviews were held while the requirements were being written, at the weekly team meeting. Following the review rule of the proposal, the requirements written by Hyewon Kim were reviewed by Hyoungdo Kim and Minkyung Kim; the analysis

models by Hyewon Kim and Minkyung Kim. The review of each requirement applied the five checks of the lecture, using the questions in the table below. Findings and their resolution were entered in the weekly meeting notes (WBS 1.2.1); a finding was closed only when the specification file had been changed or the reason for no change had been written down.

Table 34. Review checks applied to each requirement

Check	Question asked in the review	Example of how the specification answers it
Validity	Does the requirement support a problem P1–P3 or a goal G1–G6?	Each FR belongs to a use case, and each use case to a problem (objectives table of Section 1.2 and Section 2.2).
Consistency	Does the requirement conflict with another requirement or rule?	FR-18 (expire when the shift starts) and BR-03 (deadline no later than the shift start) use the same limit.
Completeness	Are all outcomes covered, including failures and time-outs?	Every open state of a request has an exit by time (UC-11) and by the requester (UC-12).
Realism	Can it be implemented in one semester without a server?	Notifications are in-app only (FR-21); push and SMS are in the Won't list.
Verifiability	Can a test or an inspection decide whether it is met?	Adjectives such as "fast" or "easy" are replaced by numbers: 2 clicks (NFR-01), 360 px (NFR-02), 1 second with 20 workers (NFR-03).

Applying these checks to the domain produced several points where the first reading of the problem was ambiguous. The table below lists them together with the resolution that the specification contains. They are the points most likely to be questioned by a store owner, and they show why the business rules are written with explicit conditions.

Table 35. Ambiguities in the domain and their resolution in the specification

Question	Resolution	Where
Does a shift handed over by agreement break perfect attendance?	The law does not say. Made a workplace setting, EXCUSED by default; both variants tested.	BR-06, FR-17
The 15-hour threshold is a four-week average. Which weeks?	Contractual hours come from the fixed schedule, which is the same every week, so the average equals the weekly value.	BR-06
Do extra shifts taken as a substitute raise the holiday allowance?	No. Contractual hours are not changed by substitute shifts; only scheduled hours change.	BR-06
What happens when two candidates accept?	The first acceptance wins inside one transaction; the second is refused with the name of the acceptor.	BR-02, NFR-04
What if the owner never decides on an accepted request?	It expires at its deadline or at the shift start; the shift stays with the requester.	BR-12
Who is the "original" worker after two swaps of the same shift?	The first worker; the field is set only when empty.	BR-04
How is a shift that ends after midnight measured?	An end time not later than the start time means the next day; supported by the rule functions and tested, although the seed data has no such shift.	spec §8, TC-09x
In which month is the holiday pay of a week that spans two months paid?	In the month of the week's Sunday.	BR-11
Who is told when a request is cancelled or expires, and who when someone declines?	A recipients table per notification kind: cancel notifies every target that was PENDING or ACCEPTED; expiry notifies requester and owner and closes pending targets silently; a decline notifies no one.	spec §8, FR-19
Is a week that has not ended yet already "perfect"?	Only an ABSENT shift breaks attendance; a SCHEDULED shift does not, so the weekly result is a projection until the week is over.	BR-06

3.10.2 Prototyping

Prototyping uses an executable model of the system to check requirements [7]. The UI prototype of the key screens — weekly board, inbox and approval panel — belongs to WBS 4.2.1 (screen wireframes and prototype screens, owner

Minkyung Kim), with wireframes scheduled to start in Week 7. It was generated with the same tool chain as the final system, so that it could be evolved rather than thrown away. The proposal planned to show it to the owner of the reference store and two co-workers during validation. The prototype was used to check the requirements that are hard to judge on paper: whether the before/after table of FR-11 contains the figures an owner needs, whether the accept button can be reached in two clicks (NFR-01), and whether a handed-over shift is recognisable on the weekly board. Prototype feedback is recorded in the meeting notes together with the review findings; like the interviews, it is used to confirm or correct requirements and is not reported as statistics. Because the prototype was generated by the AI coding assistant from the same specification, a change requested on the prototype was made in the specification first, following the change procedure of Section 3.11.

3.10.3 Test-case generation

The third technique develops tests from the requirements to check that they are testable [7]. For ShiftSwap the business rules were the natural starting point, because each rule is a function with inputs and a result. The specification requires that every business rule maps to at least one test case (spec §12). Test cases are grouped by a two-digit prefix, and each test is named with its full identifier, for example TC-081 holiday allowance at exactly 15 h. The table below gives the mapping from business rules to test groups and the boundary cases each group must contain. The test results are reported in Chapter 6.

Table 36. Mapping of business rules to test-case groups

Business rule	Test group	Cases required by the specification and the final prompt	Related requirements
BR-01	TC-01x	Interval overlap including back-to-back shifts; each eligibility condition failing on its own; the seed request yielding exactly Choi Minho and Kang Doyun.	FR-07, FR-08
BR-03	TC-02x	Request on someone else's shift, on a past or non-SCHEDULED shift, a second open request, a deadline before now or after the shift start.	FR-06
BR-02	TC-05x	Two acceptances of the same request, of which only the first succeeds; other targets closed.	FR-09, FR-10, NFR-04
BR-04	TC-06x	Approve sets the acceptor and keeps the first original worker; reject leaves the shift unchanged; approval of a non-ACCEPTED request refused.	FR-12
BR-05, BR-06, BR-07	TC-08x	Eligibility at 14.99 h (not eligible) and 15 h (eligible); the 40 h cap; both substitution-attendance policies; the worked example (Choi Minho, 3.2 h, KRW 33,024).	FR-14, NFR-10
BR-08	TC-09x	Premium zero at 4 regular employees and non-zero at 5; daily and weekly overtime without double counting; night hours, including a shift that ends after midnight.	FR-15, NFR-10
BR-09, BR-10	TC-10x	Warning below the minimum; probation reduction only when all conditions hold, each condition failing on its own (contract under one year, after three months, after probation end, simple labour).	FR-16, NFR-10
BR-11	TC-11x	Monthly totals; holiday pay assigned by the week's Sunday; estimated flag when attendance is missing; rounding down to the won.	FR-15
BR-12	TC-12x	Expiry at the deadline and at the shift start, for REQUESTED and ACCEPTED requests; no expiry of final states.	FR-18

Writing the test cases before implementation had a validation effect of its own: a rule for which no precise expected result could be written was, by the lecture's definition, not verifiable, and had to be made more precise. The rounding clauses of BR-07 and BR-11 ("rounded to 2 decimals", "rounded down to the won") are examples of precision that a test demands and a prose rule easily omits. The non-functional requirements that are not covered by unit tests have the verification methods listed in the table of Section 3.7 (walkthrough, browser checks, timed load, role test, design review).

3.11 Requirements Management

Requirements management keeps track of individual requirements and of the links between dependent requirements, so that the impact of a change can be assessed, and it needs a formal process for change proposals [7]. In a vibe-coding project this activity has an additional purpose: the requirements are also the input of the code generator, so a requirement that changes in one document but not in the prompt produces a system that no longer matches its specification. The team therefore organised requirements management around two files in the repository.

3.11.1 The specification file as single source of truth

The file `spec/spec.md` contains every identifier and every fact that more than one artefact uses: actors, use cases, functional and non-functional requirements, business rules, states, the database schema, the seed data, the test groups and the reference list. Its first line states the rule that governs it: every artefact must use the identifiers, names, states, rules and numbers defined there, and anything not defined there must be added there first before it is used anywhere else. The final prompt (Chapter 5) instructs the AI coding assistant to treat the specification as the contract and not to add features that are not in it, and the prompt is always given together with the full specification. The UML sources, the reports and the test names all refer to the same identifiers. As a result, a requirement has one wording, and the report quotes it rather than paraphrasing it.

3.11.2 Traceability

Traceability links are part of the specification itself. Each functional requirement names its use case; each use case names its tables; each business rule is referenced by the requirements that use it; each test group names the rules it tests; and the source code cites the business rule it implements in the comment of each rule function. The specification states the traceability rule explicitly (spec §12): every FR maps to at least one UC, every UC to at least one screen and service function, and every BR to at least one test case. The full traceability matrix is given in Chapter 6. With these links, the impact of a change can be found by searching for an identifier across the repository.

3.11.3 Change procedure and the prompt log

A proposed change to the requirements followed five steps:

1. The change is raised at the weekly meeting and entered in the meeting notes (WBS 1.2.1), with its reason (a review finding, prototype feedback, a failing test or a legal source).
2. The requirements lead and the project manager assess its impact by following the traceability links: which use cases, rules, tables, screens and test cases are affected, and whether the change moves a feature across the MoSCoW boundary.
3. If accepted, the change is made in `spec/spec.md` first. Identifiers are never reused: a removed requirement keeps its number retired, and a new one receives the next free number.
4. The prompt is updated to refer to the changed specification, the code is regenerated or modified by the AI coding assistant, and the affected test groups are run together with all others as regression tests.
5. The change is recorded in the prompt log with the prompt version, the specification change, and the result of the test run.

The prompt log serves as the change history that the lecture requires. Each entry records a prompt version, what was asked, what the AI coding assistant produced, what was wrong with it, and whether the correction was made in the prompt or in the specification. Chapter 5 presents the log. The distinction matters: a correction in the prompt concerns how the system is built, while a correction in the specification changes what the system is required to do and therefore passes through the steps above.

Several kinds of change were anticipated in the requirements and turned into data rather than code, so that they do not require a requirements change at all. A new minimum wage for 2027 is a new row in the minimum-wage table (UC-10), not a change to BR-09. A store that reaches five regular employees changes a setting, and BR-08 applies automatically. A store that reads the law differently on handed-over shifts changes the substitution-attendance policy. These are the changes that the domain makes most likely, and the design of Chapter 4 keeps them out of the code.

4. Design Specification

This chapter describes how ShiftSwap provides the functions specified in Chapter 3. The lecture divides design into architectural design (the structure of the system), interface design (the interfaces between its parts) and detailed design (the algorithms of the individual parts) [6]. Section 4.1 presents the architecture and Section 4.2 the deployment. Section 4.3 records the main design decisions and the alternatives that were rejected. Sections 4.4 and 4.5 give the static design: the class model and the database. Sections 4.6 and 4.7 give the dynamic design: sequence diagrams for the four central use cases and state machines for the three entities with a life cycle. Section 4.8 specifies the algorithms of the business rules, and Section 4.9 the design of the user interface.

The design is described as it is implemented. Every module, function and table named in this chapter exists under the same name in the repository, and the functions are the ones called by the tests of Chapter 6. Where a UML diagram shows an operation at a more abstract level than the code, the text gives the name of the implementing function.

4.1 Architecture

4.1.1 Layers

ShiftSwap has a layered, client-only architecture. All code runs in the user's browser; the only server is the static web host that delivers the files. The table below lists the five parts of the architecture. Each layer may call only the layers below it, and the domain-rule layer calls nothing at all.

Table 37. Layers and modules of ShiftSwap

Layer	Module (file)	Responsibility
Presentation	app/index.html, app/main.js, app/ui/*.js, app/styles.css	Screens per role; hash routing (#/schedule, #/requests, ...); role menus; demo-clock and language controls; translation of error codes into English or Korean text.
Application services	app/core/services.js	One exported function per use-case step, each running inside one transaction; read-only query functions for the screens; creation of Notification rows.
Domain rules	app/core/rules.js	Pure functions for BR-01 to BR-11: overlap, duration, week start, eligibility, request validity, contractual hours, weekly summary, holiday hours, premium hours, minimum wage, probation, payroll row. No database, no DOM.
Data access	app/core/db.js	sql.js start-up, schema of spec §8, seed data of spec §10, tx(), all(), get(), run(), persistence to localStorage, reset.
Clock	app/core/clock.js	Demo clock: now(), setNow(), advance(). The application never reads the real time.

The most important boundary is the one around rules.js. Every decision that depends on a business rule — whether a worker is eligible, whether a request is valid, whether a holiday allowance is owed, how much premium is paid — is made by a function of this module that receives plain values and returns plain values. The service layer reads the data, calls the rule, and writes the result. This has three consequences. The rules can be unit-tested without a database (the 23 tests of tests/rules.test.js). The rules can be read by someone who knows the labour law but not SQL. And a change of a rule cannot accidentally change how data is stored, because the rule module has no access to storage.

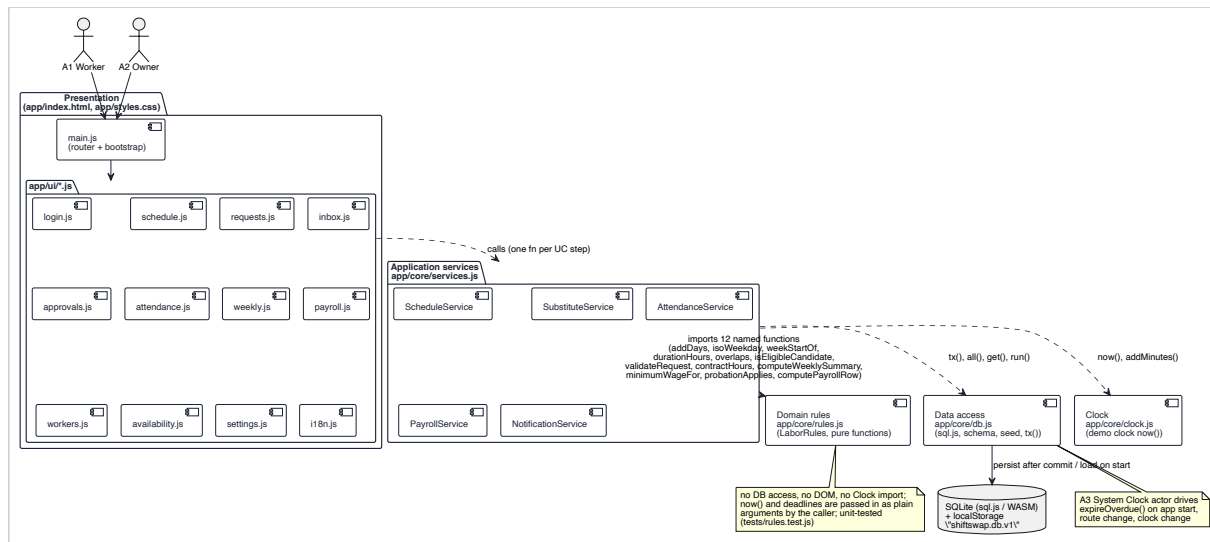


Figure 6. Component diagram of ShiftSwap

The component diagram above shows the same structure. The actors reach only the router `main.js`, which renders one screen module of `app/ui/`. Screens call the services, and the services depend on the rules, the data-access module and the clock. The diagram lists the twelve functions that `services.js` imports from `rules.js`, and it shows no dependency from the rules to the clock: the current time is passed to the rules as an argument (for example, `validateRequest` receives `now` from the service), and `rules.js` imports nothing. The rule functions are therefore deterministic, which is what allows tests such as TC-023 to fix the time in the test itself.

4.1.2 Transactions and persistence

The data-access module keeps one SQLite database in memory, created by `sql.js` [9]. Every service command is wrapped in `tx(fn)`, which executes `BEGIN`, runs the function, and then executes `COMMIT`; if the function throws, it executes `ROLLBACK` and passes the error on. After a successful commit the whole database is exported as bytes, encoded in base64 and saved to `localStorage` under the key `shiftswap.db.v1`. A nested call of `tx()` joins the outer transaction instead of opening a second one, so that `decideRequest` can call `recomputeWeek` and both changes commit or roll back together. On start, open-Database loads the saved copy if one exists and has the current schema; otherwise it creates the schema, inserts the seed data and saves it (NFR-05).

Because a failing service throws a `ServiceError` inside `tx()`, a rejected operation never leaves a partial change behind. This is the mechanism behind NFR-04: the acceptance of a request re-reads the request inside the transaction and, if it is no longer `REQUESTED`, throws before anything is written.

4.1.3 The demo clock and the System Clock actor

All time-dependent behaviour — whether a shift is in the future, whether a deadline has passed, the default deadline, the time stamps of notifications — uses `now()` from `clock.js`. The demo clock starts at 2026-09-28 09:00 (the Monday of the seed data), is stored under `shiftswap.clock.v1`, and can be set or advanced by one hour or one day from the header. The System Clock actor A3 of the use-case model is realised by the router: before rendering any screen, `main.js` calls `expireOverdue()`. The function therefore runs on start, on every route change and after every change of the demo clock, and a request whose deadline has passed is expired before any user can see or act on it.

4.2 Deployment

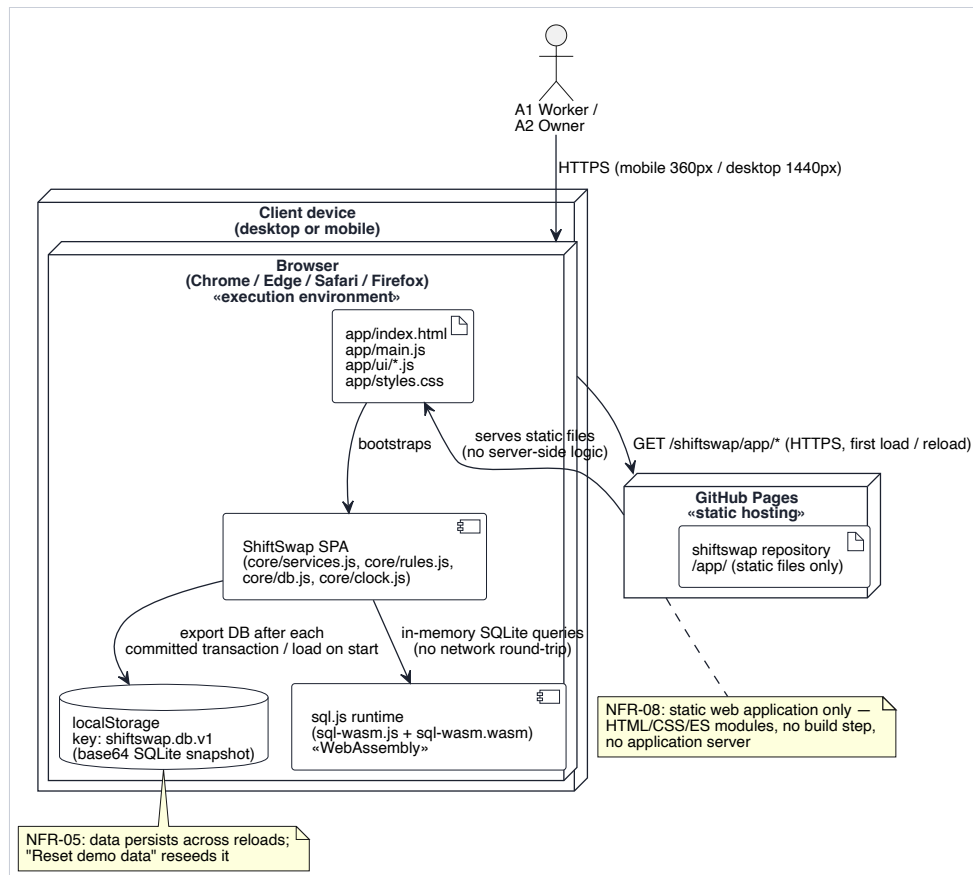


Figure 7. Deployment diagram of ShiftSwap

The deployment diagram above has two nodes. GitHub Pages serves the static files of the repository over HTTPS [13]; it runs no application code. The browser on the user's device downloads `index.html`, the JavaScript modules, the style sheet and the two `sql.js` files, and from then on all processing is local. The `sql.js` files (`sql-wasm.js` and `sql-wasm.wasm`, version 1.14.2) are copied into `app/vendor/`, so the application does not depend on a content-delivery network at run time. All URLs in the application are relative, so the same files work under the sub-path `/shiftswap/app/` of the deployed site and under any local static server. Chapter 7 describes the deployment procedure.

The browser keeps four values between visits, listed in the table below. If the browser refuses storage (for example in some private-browsing modes), `main.js` falls back to an in-memory store, and the application still works for the length of the visit.

Table 38. Browser storage used by ShiftSwap

Key	Storage	Content
<code>shiftswap.db.v1</code>	localStorage	The whole SQLite database, base64-encoded; rewritten after every committed transaction.
<code>shiftswap.clock.v1</code>	localStorage	The demo clock, YYYY-MM-DDTHH:MM.
<code>shiftswap.lang</code>	localStorage	The chosen language, en or ko.
<code>shiftswap.user</code>	sessionStorage	The id of the signed-in demo account (UC-13); cleared when the tab is closed or the user signs out.

4.3 Design Decisions

The table below records the design decisions that shaped the system, the alternatives considered, and the reason for the choice. Several decisions follow directly from the organisational requirement NFR-08; the others were made during design and the two build iterations recorded in the prompt log (Chapter 5).

Table 39. Design decisions, alternatives and rationale

#	Decision	Alternatives considered	Rationale
D1	Client-only application on static hosting	Server with a REST API and a hosted database; serverless functions	NFR-08 and NFR-06 require a system that runs without installation or server. A static site can be reproduced from the final prompt without accounts, keys or running costs, and anyone can open it from the URL. The price is that data is local to one browser (Section 8.3).
D2	SQLite in the browser (sql.js, WebAssembly)	Plain JSON objects in localStorage; IndexedDB	The course requires a relational database of at least four tables, and the business rules need joins, uniqueness constraints and transactions. sql.js provides real SQL with CHECK, UNIQUE and BEGIN/COMMIT/ROLLBACK, and the same schema runs unchanged in the Node tests.
D3	Persist the whole database after each commit	Persist individual rows; persist on page unload	The database of a small store is a few hundred kilobytes at most, so exporting it after each commit is cheap. Saving on unload would lose data if the tab is killed.
D4	Business rules as pure functions in one module	Rules inside SQL queries; rules inside the service functions	Pure functions can be tested at boundary values without a database (Section 6.4) and read against the legal source. Each function cites its BR in a JSDoc comment.
D5	One service function per use-case step, each in one transaction	Object-oriented service classes; direct SQL from the screens	The mapping from use case to code becomes one-to-one, which makes the traceability matrix derivable from the code. A transaction per step guarantees that a step either happens completely or not at all.
D6	First acceptance enforced by a conditional update inside the transaction	Optimistic locking with a version column; a lock table	UPDATE SubRequest SET status = 'ACCEPTED' ... WHERE id = ? AND status = 'REQUESTED' changes exactly one row for the first acceptance and none for any later one; the service checks the number of changed rows (NFR-04).
D7	Adjustable demo clock instead of the real time	Real system time; time only in tests	Expiry, deadlines and weekly results must be demonstrable in a presentation of fifteen minutes. The seed data is fixed to the week of 2026-09-28, so the clock must be too.
D8	Weekly summaries and payroll stored as derived tables	Compute on every screen and never store	A confirmed payroll must not change when shifts change later (FR-15), so payroll must be stored. Weekly summaries are stored so that payroll can sum the holiday hours of the month's weeks with one query; they are recomputed whenever their inputs change.
D9	Notifications as rows of a table, written by the services	Events published to the screens; browser notifications	An in-app inbox (FR-21) that survives reloads needs stored rows. Writing them in the same transaction as the state change guarantees that a notification exists exactly when the change happened.
D10	Errors as <code>ServiceError(code, message, params)</code>	Plain exceptions with English text	The UI translates the code into English or Korean and fills in the parameters (for example the name in "Already taken by ..."), while the tests assert on the code.
D11	No framework, no build step; hash routing	A component framework with a bundler	NFR-08 forbids a build step. Hash routing (<code>#/inbox</code>) works on static hosting without server rewrites. Eleven screens are few enough to render with template strings.
D12	An old saved database is replaced by the seed, not migrated	Migration with ALTER TABLE	When <code>Payroll.minWageAck</code> was added in iteration 2, replacing kept the schema exactly as spec §8. Only demonstration data is lost (TC-002).

4.4 Class Design

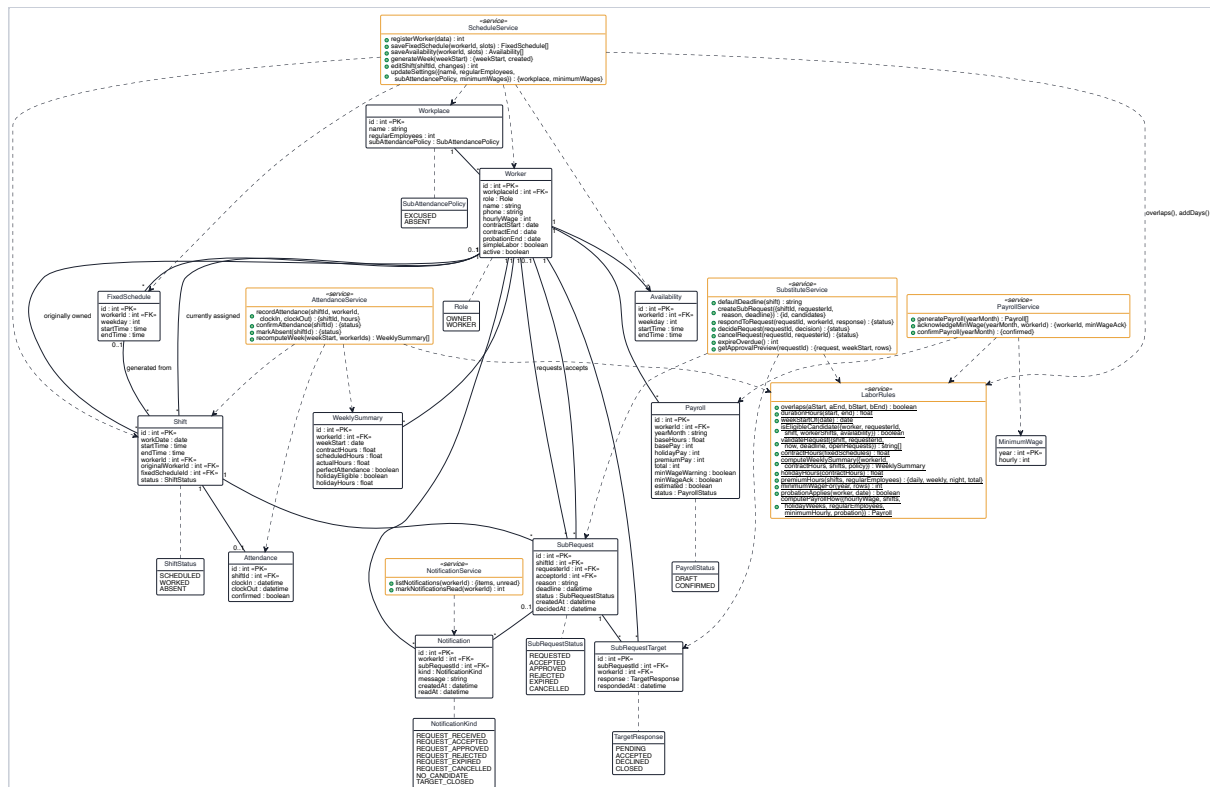


Figure 8. Class diagram of ShiftSwap (domain classes, services and rules)

The class diagram above has three groups. The twelve domain classes correspond one to one to the tables of Section 4.5; their attributes are the columns, and the associations and multiplicities are those of the foreign keys. Worker has two associations to Shift: a shift is *currently assigned* to exactly one worker and *originally owned* by at most one other worker, which records a handover (BR-04). The enumerations Role, SubAttendancePolicy and ShiftStatus give the permitted values of the corresponding columns.

The five service classes are a logical grouping of the exported functions of `services.js` by concern; in the code they are functions of one ES module, not JavaScript classes. The class `LaborRules` stands for the module `rules.js`, whose functions are static in the sense that they have no state. The tables below list the implementing functions with their parameters.

Table 40. Service classes and their implementing functions in services.js

Logical class	Function (real signature)	UC	Responsibility
ScheduleService	registerWorker(data)	UC-01	Validate and insert or update a worker (FR-01).
	saveFixedSchedule(workerId, slots)	UC-01	Replace a worker's fixed shifts; refuse overlaps; unlink shifts of deleted rows (FR-02).
	saveAvailability(workerId, slots)	UC-02	Replace a worker's availability slots (FR-03).
	generateWeek(weekStart)	UC-03	Create the week's shifts from active fixed schedules without duplicates (FR-04).
	editShift(shiftId, changes)	UC-03	Add, move or delete one shift; refuse overlaps and deletion of shifts in use (FR-05).
	updateSettings({name, regularEmployees, subAttendancePolicy, minimumWages})	UC-10	Workplace settings and minimum-wage table (FR-17).

Logical class	Function (real signature)	UC	Responsibility
SubstituteService	createSubRequest({shiftId, requesterId, reason, deadline})	UC-04	Validate (BR-03), select candidates (BR-01), create targets and notifications (FR-06–FR-08).
	respondToRequest(requestId, workerId, response)	UC-05	Accept or decline; first acceptance wins (BR-02, FR-09, FR-10).
	decideRequest(requestId, decision)	UC-06	Approve or reject; reassign the shift and recompute both weeks (BR-04, FR-12).
	cancelRequest(requestId, requesterId)	UC-12	Cancel an open request; close pending targets; notify (FR-19).
	expireOverdue()	UC-11	Expire open requests past deadline or shift start (BR-12, FR-18).
AttendanceService	recordAttendance(shiftId, workerId, clockIn, clockOut)	UC-07	Worker records times of a started shift (FR-13).
	confirmAttendance(shiftId)	UC-07	Owner confirms; shift becomes WORKED; week recomputed.
	markAbsent(shiftId)	UC-07	Owner marks a started shift ABSENT; week recomputed.
	recomputeWeek(weekStart, workerIds)	UC-08	Compute and store WeeklySummary rows (FR-14, BR-06, BR-07).
PayrollService	generatePayroll(yearMonth)	UC-09	Build DRAFT rows of a month (FR-15, FR-16, BR-08–BR-11).
	acknowledgeMinWage(yearMonth, workerId)	UC-09	Set minWageAck = 1 on a warned DRAFT row (BR-09).
	confirmPayroll(yearMonth)	UC-09	Confirm the month's drafts if every warning is acknowledged.
NotificationService	listNotifications(workerId)	FR-21	Inbox newest first, unread count, live state of each request.
	markNotificationsRead(workerId)	FR-21	Mark all of the user's notifications read.

The operations in the diagram carry the real parameters of the implementation, including `acknowledgeMinWage(yearMonth, workerId)`, which was added to the payroll service in iteration 2 when the acknowledgement became a stored column, and `confirmPayroll(yearMonth)`, which confirms all DRAFT rows of a month at once. In addition to the commands, `services.js` exports read-only query functions used by the screens, which do not change data: `getWorkplace`, `getSettings`, `getWorker`, `listWorkers`, `getFixedSchedules`, `getAvailability`, `getWeekShifts`, `listRequestableShifts`, `listMyRequests`, `listOwnerRequests`, `getApprovalPreview`, `listAttendance`, `getPayroll`, `unreadCount` and `defaultDeadline`. Among them, `listWorkers(viewer)` implements the privacy requirement NFR-11: for a worker as viewer, it removes the phone number and wage of every other person before the data reaches the screen.

The class `LaborRules` corresponds to the twelve functions of `rules.js` named in the final prompt: `overlaps`, `durationHours`, `weekStartOf`, `isEligibleCandidate` (BR-01), `validateRequest` (BR-03), `contractHours`, `holidayHours` and `computeWeeklySummary` (BR-06, BR-07), `premiumHours` (BR-08), `minimumWageFor` (BR-09), `probationApplies` (BR-10) and `computePayrollRow` (BR-07–BR-11). Each receives plain values and returns plain values; their algorithms are given in Section 4.8.

4.5 Database Design

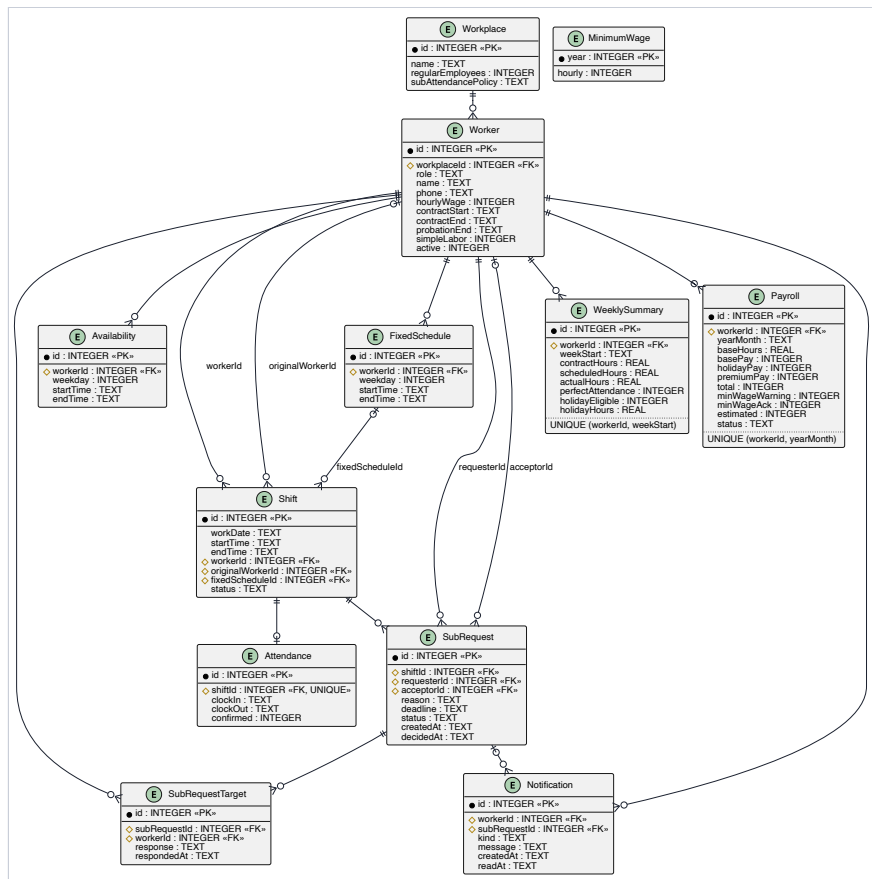


Figure 9. Entity-relationship diagram of the ShiftSwap database (12 tables)

The database has twelve tables, three times the minimum of four set by the course. The schema is the SQL of spec §8, executed verbatim by `db.js`. All identifiers are English (NFR-09). SQLite has few column types; the schema uses `INTEGER` for keys, counts, flags (0/1) and amounts in won, `REAL` for hours, and `TEXT` for dates (YYYY-MM-DD), times (HH:MM), datetimes (YYYY-MM-DDTHH:MM) and enumerations, which are restricted by `CHECK` constraints. Storing dates and times as ISO text has a practical advantage: they sort and compare correctly as strings, so the services can compare a deadline with `now()` directly. Money is never stored as a floating-point number.

The tables fall into four groups: master data (Workplace, Worker, FixedSchedule, Availability, MinimumWage), transactions (Shift, SubRequest, SubRequestTarget, Attendance), derived results (WeeklySummary, Payroll) and communication (Notification). The table below summarises each table with its key columns and the requirements and rules that use it; every table has an `INTEGER PRIMARY KEY` column `id` except MinimumWage, whose key is the year. The complete schema with every column, type and constraint is the SQL of spec §8 in the specification file `spec/spec.md` in the repository (<https://github.com/keemlogan/shiftswap/blob/main/spec/spec.md>).

Table 41. Database tables: purpose, key columns and the requirements that use them

Table	Purpose	Key columns and constraints	Used by
Workplace	The store and its settings	name; regularEmployees (default 4); subAttendancePolicy ∈ {EXCUSED, ABSENT}	FR-17; BR-06, BR-08
Worker	Owner and workers with contract and wage data	role ∈ {OWNER, WORKER}; name; phone; hourlyWage (KRW); contractStart, contractEnd (null = open-ended), probationEnd; simpleLabor; active	FR-01, FR-20; BR-01, BR-09, BR-10; NFR-11
FixedSchedule	Fixed weekly shifts of a worker	workerId → Worker; weekday 1–7 (CHECK); startTime, endTime	FR-02, FR-04; BR-06
Availability	Times a worker can take substitute shifts	workerId → Worker; weekday 1–7; startTime, endTime	FR-03; BR-01

Table	Purpose	Key columns and constraints	Used by
Shift	One shift on a date	workDate, startTime, endTime; workerId (current) and originalWorkerId (first owner after a swap) → Worker; fixedScheduleId (null for added shifts); status ∈ {SCHEDULED, WORKED, ABSENT}	FR-04, FR-05, FR-12, FR-13; BR-01, BR-03, BR-04, BR-06, BR-11
SubRequest	A substitute request and its state	shiftId → Shift; requesterId, acceptorId → Worker; reason; deadline; status ∈ six states (CHECK); createdAt, decidedAt	FR-06–FR-12, FR-18, FR-19; BR-02–BR-04, BR-12
SubRequestTarget	One eligible candidate of a request	subRequestId → SubRequest; workerId → Worker; response ∈ {PENDING, ACCEPTED, DECLINED, CLOSED}; respondedAt	FR-07, FR-09, FR-10, FR-19; BR-01, BR-02
Attendance	Recorded clock-in and clock-out of a shift	shiftId → Shift, UNIQUE; clockIn, clockOut (HH:MM on the shift date); confirmed 0/1	FR-13; BR-06, BR-11
WeeklySummary	Derived weekly hours and holiday allowance	workerId, weekStart (Monday), UNIQUE together; contractHours, scheduledHours, actualHours; perfectAttendance, holidayEligible; holidayHours	FR-11, FR-14; BR-05–BR-07
Payroll	Monthly pay per worker	workerId, yearMonth, UNIQUE together; baseHours, basePay, holidayPay, premiumPay, total (KRW integers); minWageWarning, minWageAck, estimated; status ∈ {DRAFT, CONFIRMED}	FR-15, FR-16; BR-07–BR-11
MinimumWage	Minimum hourly wage per year	year (primary key); hourly	FR-16, FR-17; BR-09
Notification	In-app message to one user	workerId → Worker; subRequestId → SubRequest; kind (eight kinds); message; createdAt; readAt (null = unread)	FR-07, FR-08, FR-10, FR-12, FR-18, FR-19, FR-21

4.5.1 Normalisation and integrity

The master and transaction tables are in third normal form: every non-key column depends on the whole primary key and on nothing else. There are no repeating groups — a worker's several fixed shifts and availability slots are rows of their own tables rather than columns of Worker — and names of workers are stored only in Worker and joined where needed. Two tables deliberately hold derived data. WeeklySummary is a cache of values that can be recomputed from Shift, Attendance, FixedSchedule and Workplace; it is kept consistent by recomputing the affected rows inside the transaction that changes their inputs (approval, attendance confirmation, marking absent, payroll generation, and opening the weekly summary screen). Payroll is a snapshot: once CONFIRMED, it must keep the amounts that were paid even if shifts change afterwards (TC-115), which is why it cannot be a view.

Integrity rules are enforced at two levels. The schema enforces what SQL can express locally: primary keys, NOT NULL, the permitted values of every status column, one attendance record per shift (Attendance.shiftId UNIQUE), one summary per worker and week, and one payroll row per worker and month. The services enforce the rules that involve several rows or the clock: no overlapping fixed shifts or shifts of one worker (FR-02, FR-05), at most one open request per shift (BR-03), first acceptance wins (BR-02), and no deletion of a shift that has a request or attendance record (FR-05). Foreign keys are declared with REFERENCES for documentation and for the ER diagram; the services never delete a row that is still referenced.

4.6 Dynamic Design: Sequence Diagrams

The lecture describes a sequence diagram as the interactions that take place during a particular use case, with the objects along the top and function calls as horizontal arrows [6]. Five sequence diagrams — SD-1, SD-2, SD-3a, SD-3b and SD-4 — cover the use cases with the most interaction. For each sequence diagram, the numbered steps name the functions of the implementation.

4.6.1 SD-1 Request substitute (UC-04)

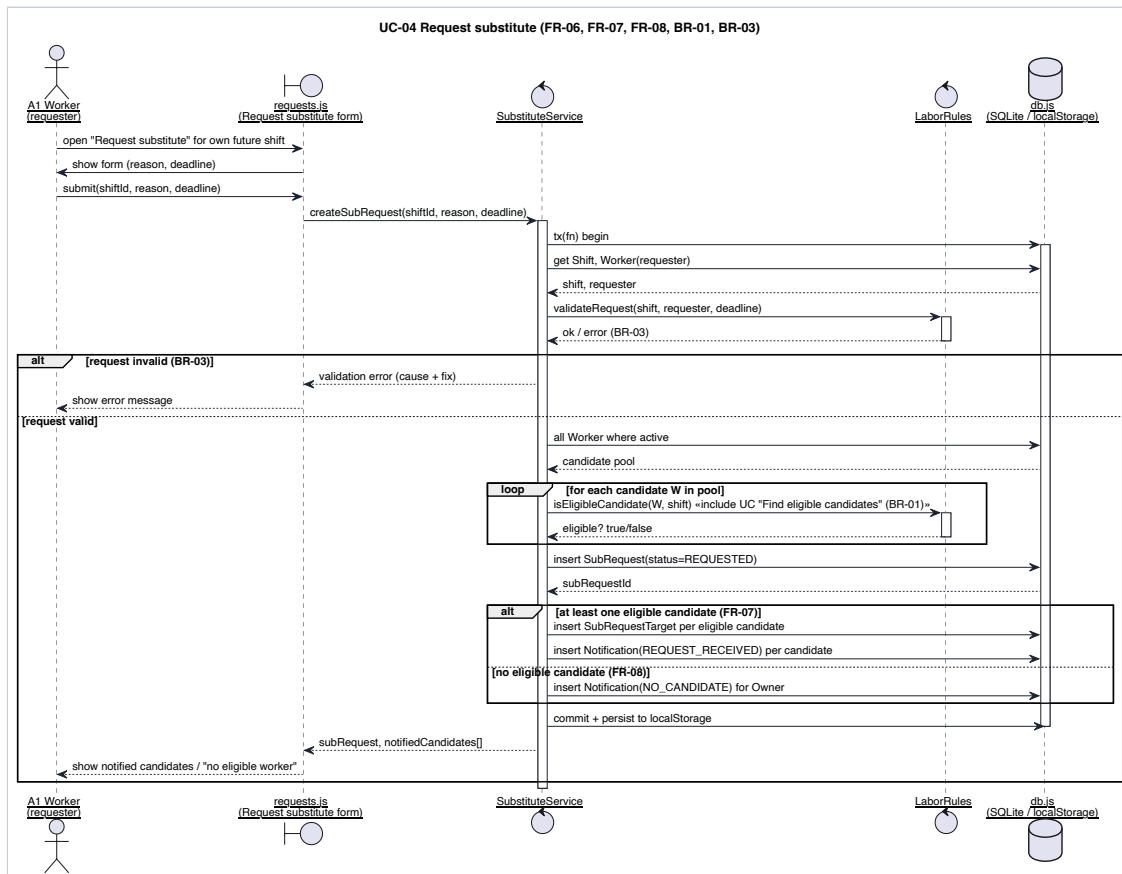


Figure 10. SD-1 — request substitute (UC-04; FR-06–FR-08, BR-01, BR-03)

1. The worker starts the request flow. Its first step lists the worker's requestable shifts with `listRequestableShifts(workerId)`, and its second step fills the deadline with `defaultDeadline(shift)`, the earlier of `now + 24 h` and the shift start.
2. When the worker sends the request from the review step, the screen calls `createSubRequest({shiftId, requesterId, reason, deadline})`, which opens a transaction with `tx()`.
3. The service reads the shift and counts the open requests on it, then calls `validateRequest` with the current time from `now()`. If the result contains an error code, it throws a `ServiceError` with that code; the transaction rolls back and the screen shows the translated message (the "request invalid" branch).
4. Otherwise the service inserts the `SubRequest` with status `REQUESTED`.
5. For every worker, the service reads that worker's shifts on the date and availability slots and calls `isEligibleCandidate` (the loop of the diagram; the included use case *Find eligible candidates*).
6. For each eligible candidate it inserts a `SubRequestTarget` with response `PENDING` and a `REQUEST_RECEIVED` notification. If there is none, it inserts one `NO_CANDIDATE` notification for the owner.
7. The transaction commits and the database is saved. The service returns the request id and the list of candidates, which the screen shows.

4.6.2 SD-2 Accept request with concurrent second acceptance (UC-05)

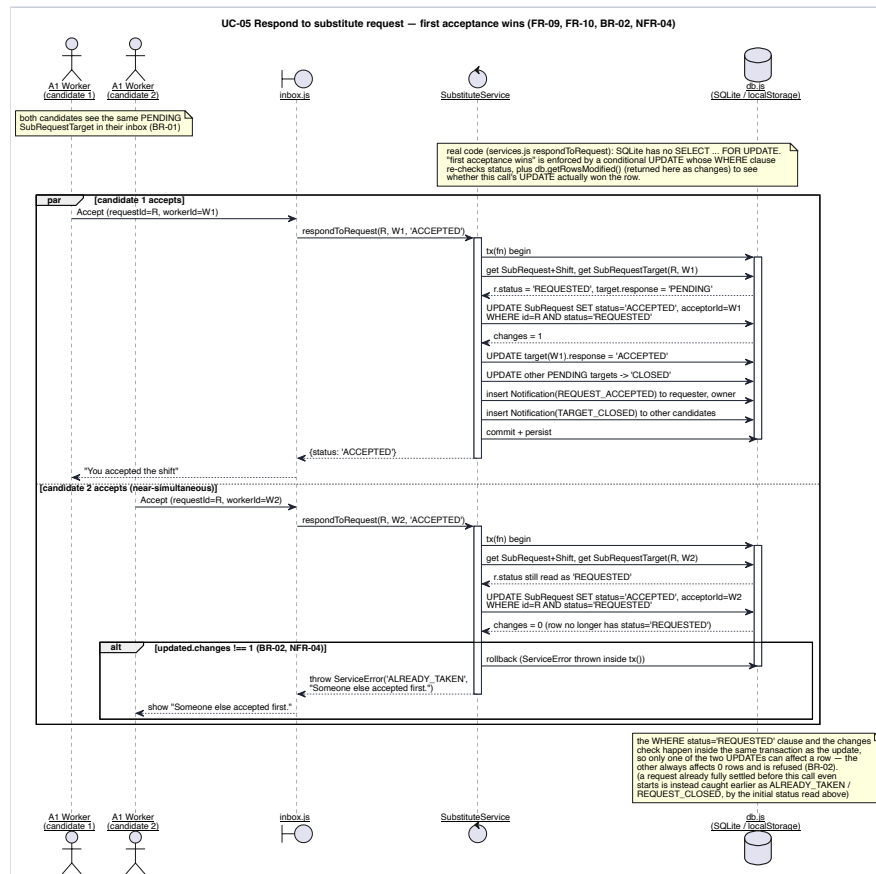


Figure 11. SD-2 — respond to a request; first acceptance wins (UC-05; FR-09, FR-10, BR-02, NFR-04)

- Both candidates see the same pending request, built for their screens by the service queries (`listNotifications`, and on the Home screen `getWorkerHome`), with *Accept* and *Decline*.
- Candidate 1 selects *Accept*; the screen calls `respondToRequest(requestId, workerId, 'ACCEPTED')`, which opens a transaction.
- Inside the transaction the service reads the request and the candidate's target. Both are REQUESTED and PENDING, so the service executes the conditional update `UPDATE SubRequest SET status = 'ACCEPTED', acceptorId = ? WHERE id = ? AND status = 'REQUESTED'`. The database reports one changed row.
- The service sets the candidate's target to ACCEPTED, sets every other PENDING target to CLOSED with a TARGET_CLOSED notification, and sends REQUEST_ACCEPTED to the requester and the owner. The transaction commits.
- Candidate 2 selects *Accept* at nearly the same moment. The diagram shows the critical case, in which candidate 2's first read still sees REQUESTED. The same conditional update then changes zero rows, because the row no longer has the status REQUESTED. The service checks the number of changed rows and throws `ServiceError('ALREADY_TAKEN')`; the transaction rolls back without any change.
- If candidate 2 acts later, after candidate 1's transaction has committed, the first read already finds the request ACCEPTED with another acceptor, and the service refuses at once with `ALREADY_TAKEN` and the acceptor's name, which the screen shows as "already taken by Choi Minh".

SQLite has no `SELECT ... FOR UPDATE`, so the design does not lock the row before reading it. Instead, the condition `status = 'REQUESTED'` is part of the update itself, and the check of the changed-row count happens in the same transaction. Only one of two competing updates can affect the row. In the deployed system both accounts use the same database in one browser, and JavaScript executes one transaction at a time, so the two acceptances are in fact serialised; the design does not rely on this, and the same property would hold with a shared database server (Section 8.4).

4.6.3 SD-3 Approve substitution with weekly recomputation (UC-06)

The approval use case is shown in two diagrams: the comparison that the owner sees before deciding (SD-3a), and the decision itself (SD-3b).

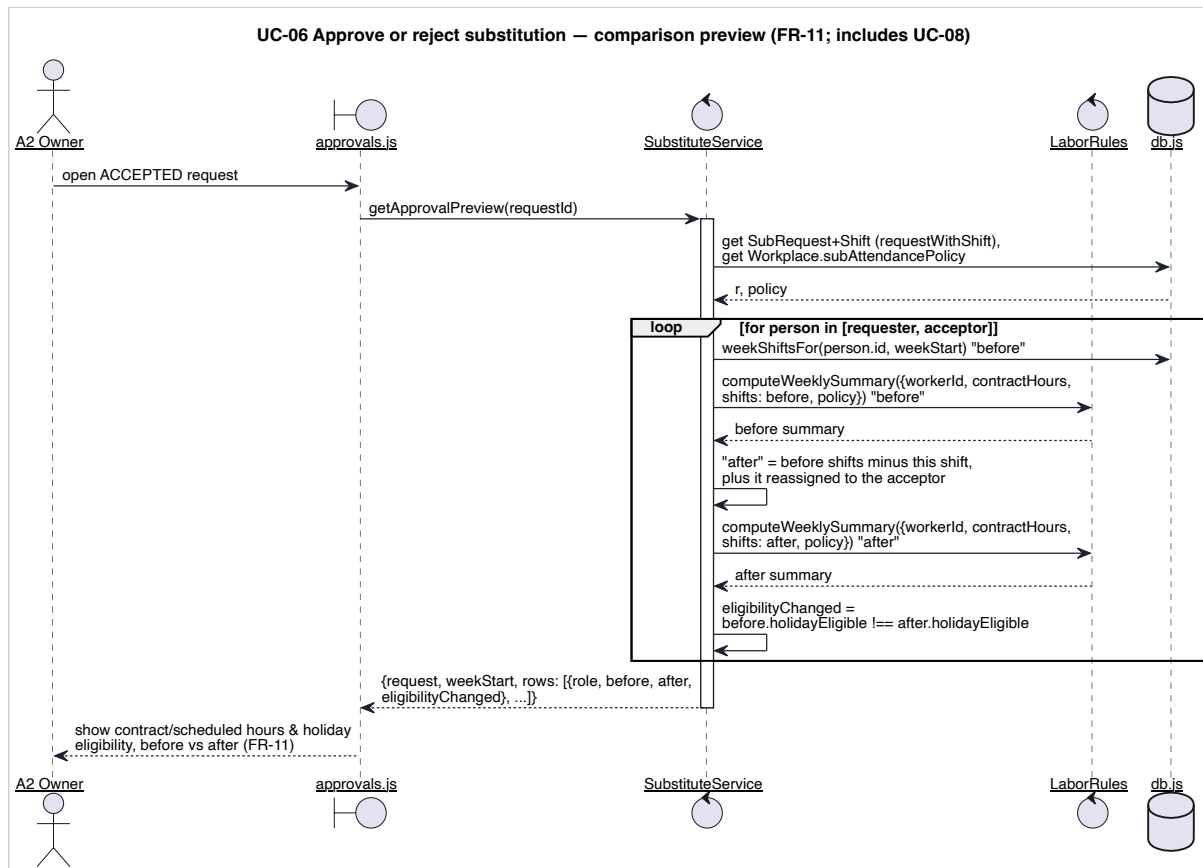


Figure 12. SD-3a — approval preview: before/after comparison for requester and acceptor (UC-06; FR-11; uses UC-08)

1. The owner opens an accepted request; the screen calls `getApprovalPreview(requestId)`. The service reads the request with its shift (`requestWithShift`) and the workplace's substitution-attendance policy.
2. For the requester and then the acceptor, the service reads the shifts of the week in which the person is current or original worker (`weekShiftsFor`) and calls `computeWeeklySummary` for this "before" list.
3. It builds the "after" list by removing the requested shift and adding it back reassigned to the acceptor, and calls `computeWeeklySummary` again.
4. It sets `eligibilityChanged` when the eligibility before and after differ, and returns one row per person with both summaries. The screen shows the impact lines and highlights a changed eligibility (FR-11). The preview writes nothing to the database.

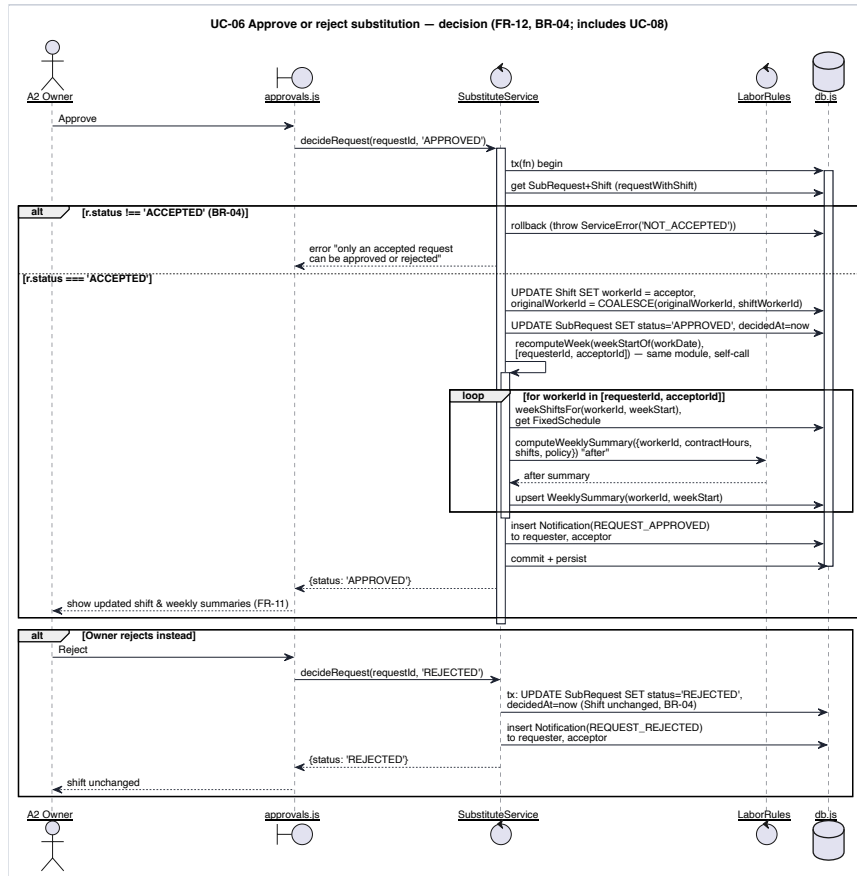


Figure 13. SD-3b — approve or reject: shift reassignment and weekly recomputation (UC-06; FR-12, BR-04; includes UC-08)

1. The owner selects *Approve*; the screen calls `decideRequest(requestId, 'APPROVED')`, which opens a transaction and reads the request with its shift. If the request is not `ACCEPTED` (BR-04), the service throws `NOT_ACCEPTED` and the transaction rolls back.
2. Otherwise it executes `UPDATE Shift SET workerId = acceptor, originalWorkerId = COALESCE(originalWorkerId, shiftWorkerId)`, so the first owner of the shift is kept even after several swaps, and sets the request to `APPROVED` with the decision time.
3. It calls `recomputeWeek(workDate, [requesterId, acceptorId])` in the same module. The nested call joins the same transaction; for each of the two workers it reads the week's shifts and fixed schedule, calls `computeWeeklySummary` and writes the result with an upsert (`INSERT ... ON CONFLICT (workerId, weekStart) DO UPDATE`).
4. It sends `REQUEST_APPROVED` to the requester and the acceptor and commits. The schedule, read with `getWeekShifts(weekStart)`, now shows the handover marker because the shift has an original worker.
5. If the owner rejects instead, `decideRequest(requestId, 'REJECTED')` sets the request to `REJECTED`, leaves the shift unchanged and sends `REQUEST_REJECTED` to both workers.

4.6.4 SD-4 Generate monthly payroll (UC-09)

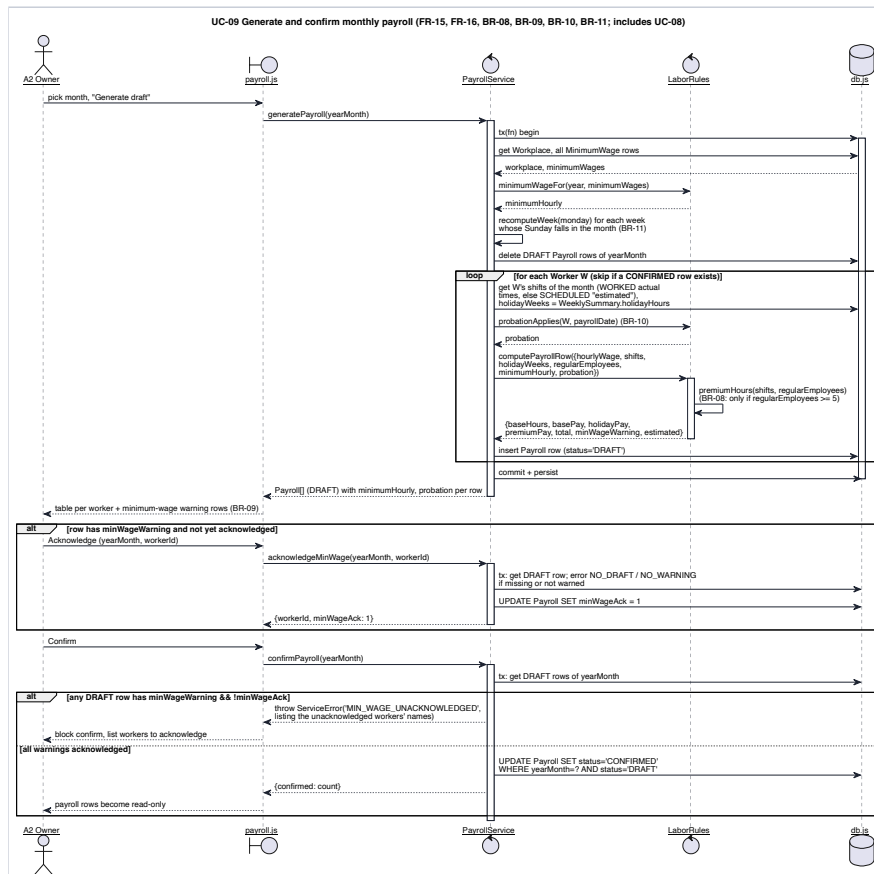


Figure 14. SD-4 — generate and confirm monthly payroll (UC-09; FR-15, FR-16, BR-08–BR-11; includes UC-08)

1. The owner selects a month in the monthly payroll of the *Pay* tab and creates the draft; the screen calls `generatePayroll(yearMonth)`.
2. The service looks up the minimum wage of the year with `minimumWageFor` and determines the weeks whose Sunday falls in the month. For each of these weeks it calls `recomputeWeek` (the included UC-08).
3. It deletes the DRAFT rows of the month; CONFIRMED rows are kept and their workers are skipped.
4. For each worker it reads the month's shifts that are not ABSENT. A WORKED shift with recorded times contributes its clock-in and clock-out; any other shift contributes its scheduled times and is marked estimated (BR-11). It reads the holiday hours of the month's weeks from `WeeklySummary`.
5. It evaluates `probationApplies(worker, date)` on the first day of the month or the contract start, if later (BR-10), and calls `computePayrollRow`, which computes base pay, holiday pay, premium pay through `premiumHours`, the total and the minimum-wage warning. The row is inserted as DRAFT.
6. The screen shows the rows. A row with a warning has a button to acknowledge it, which calls `acknowledgeMinWage(yearMonth, workerId)` and sets `minWageAck = 1`.
7. Confirming the month calls `confirmPayroll(yearMonth)`. If any DRAFT row has a warning without acknowledgement, the service throws `MIN_WAGE_UNACKNOWLEDGED`; otherwise all DRAFT rows of the month become CONFIRMED.

The division of work between the service and the rules is visible in the diagram: the service calls `minimumWageFor` and `probationApplies` and passes their results to `computePayrollRow`, which calls `premiumHours` itself. This keeps `computePayrollRow` independent of dates and of the database.

4.7 State Design

Three entities have a life cycle that the business rules constrain: the substitute request, the shift and the payroll row. The lecture's rules for state diagrams — one initial state, every state reachable, a path to a final state from every state, and every transition labelled with an event [6] — hold for all three. For each transition the text names the function that performs it, which is how the design was checked against the code.

4.7.1 SubRequest

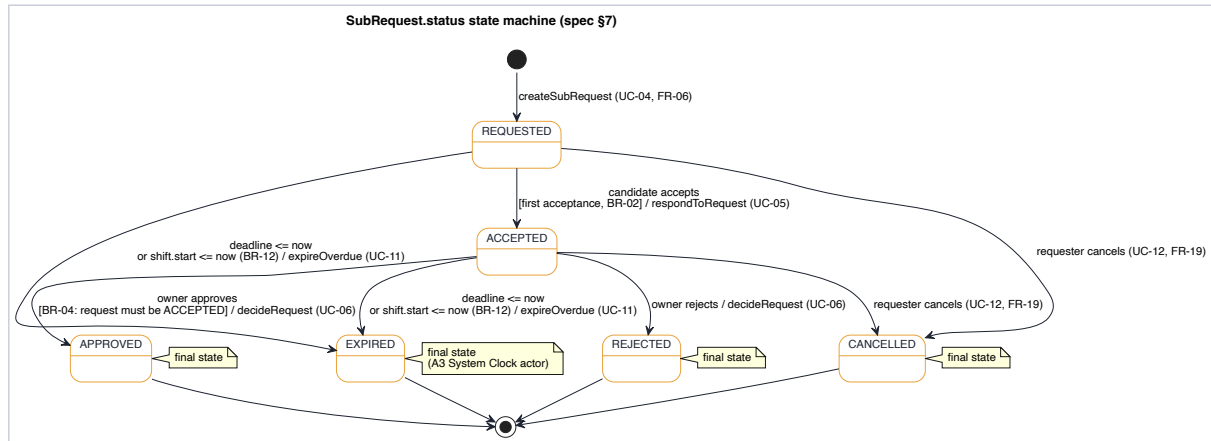


Figure 15. State machine of SubRequest.status

Table 42. State transitions of SubRequest.status

From	Event [guard]	To	Function
(initial)	worker submits a valid request [BR-03]	REQUESTED	createSubRequest
REQUESTED	candidate accepts [status = REQUESTED at the moment of the transaction]	ACCEPTED	respondToRequest(..., 'ACCEPTED')
REQUESTED	candidate declines	REQUESTED	respondToRequest(..., 'DECLINED') (target only)
REQUESTED, ACCEPTED	deadline $\leq$ now or shift start $\leq$ now	EXPIRED	expireOverdue
REQUESTED, ACCEPTED	requester cancels [requester = caller]	CANCELLED	cancelRequest
ACCEPTED	owner approves [status = ACCEPTED]	APPROVED	decideRequest(..., 'APPROVED')
ACCEPTED	owner rejects [status = ACCEPTED]	REJECTED	decideRequest(..., 'REJECTED')

The targets of a request have a smaller life cycle of their own: PENDING $\rightarrow$ ACCEPTED (the accepting candidate), DECLINED (a decline), or CLOSED (another candidate accepted, the request expired, or the requester cancelled). Only PENDING targets are ever closed; a target that has accepted keeps the response ACCEPTED even if its request is later cancelled or expires.

4.7.2 Shift

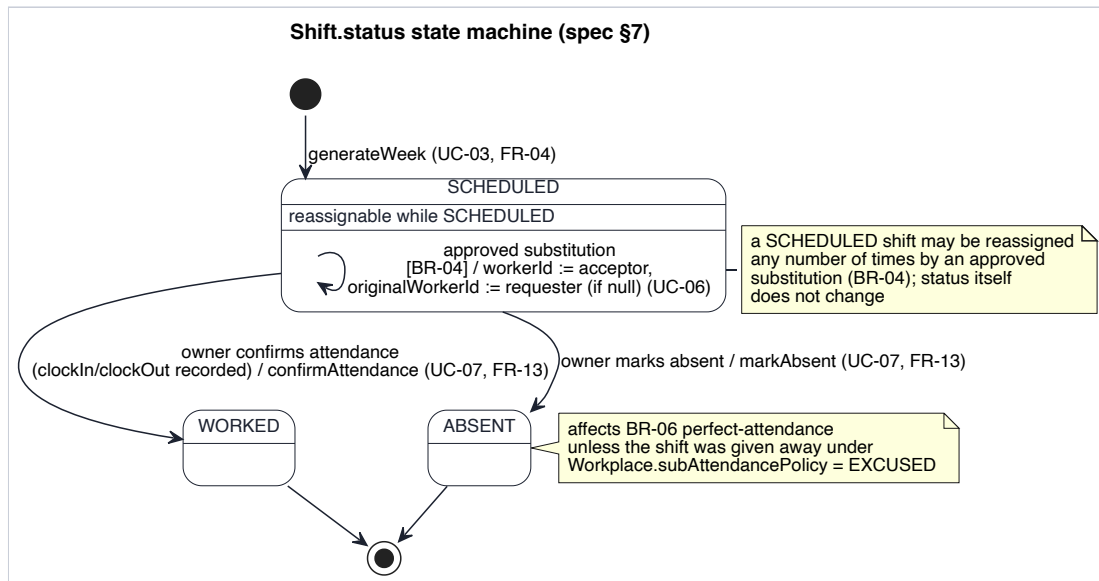


Figure 16. State machine of Shift.status

A shift is created as SCHEDULED by generateWeek or editShift. An approved substitution (decideRequest) changes its worker but not its status. The owner moves it to WORKED with confirmAttendance, which requires a recorded attendance, or to ABSENT with markAbsent, which requires that the shift has started. WORKED and ABSENT are final with respect to requests and attendance recording: validateRequest refuses a request on a shift that is not SCHEDULED, and recordAttendance refuses to change a shift that the owner has closed. The attendance record itself has two states, recorded (confirmed = 0) and confirmed (confirmed = 1).

4.7.3 Payroll

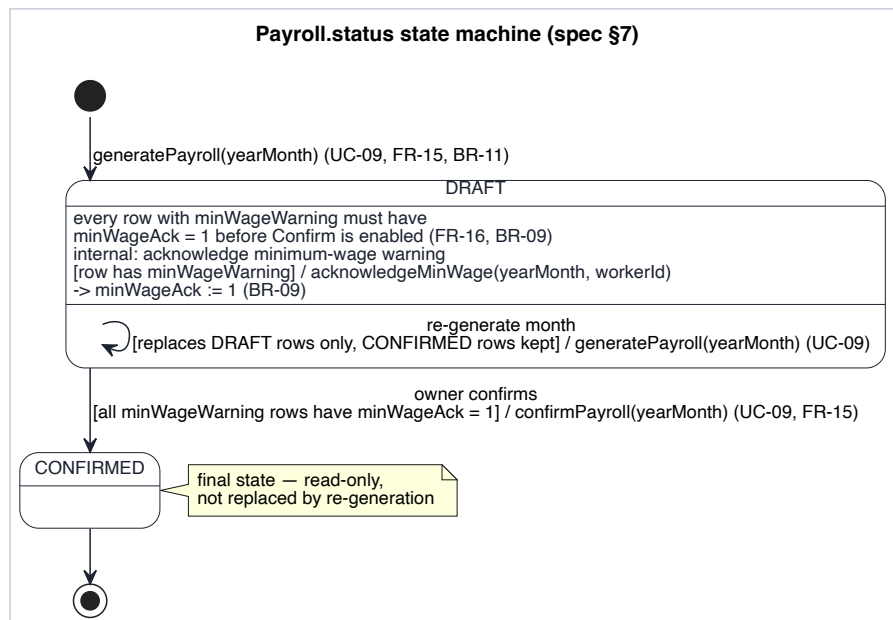


Figure 17. State machine of Payroll.status

A payroll row is created as DRAFT by generatePayroll. Regenerating the month replaces the DRAFT row with a new one whose minWageAck is 0. acknowledgeMinWage sets minWageAck = 1 on a warned DRAFT row, and confirmPayroll moves all DRAFT rows of the month to CONFIRMED, but only if every warned row is acknowledged. CONFIRMED rows are never replaced or changed.

4.8 Algorithm Design

This section specifies the algorithms of the business rules in pseudocode. The pseudocode was derived from `rules.js` and uses the same names; it omits type conversions and formatting. Times are handled on a minute axis of the shift's date: `span(start, end)` returns the start and end in minutes after midnight and adds 1,440 minutes to the end when the end is not later than the start, so that an overnight shift is one continuous interval.

4.8.1 Candidate selection (BR-01)

```
function isEligibleCandidate(worker, requesterId, shift, workerShifts, availability):
  if not worker.active: return false
  if worker.id = requesterId: return false
  for x in workerShifts: -- the worker's shifts on shift.workDate
    if x.workDate = shift.workDate and overlaps(x, shift):
      return false
  (s, e) := span(shift.startTime, shift.endTime)
  for slot in availability:
    if slot.weekday = isoWeekday(shift.workDate):
      (as, ae) := span(slot.startTime, slot.endTime)
      if as ≤ s and ae ≥ e: return true -- slot covers the whole shift
  return false

function overlaps(a, b): -- half-open intervals [start, end)
  (as, ae) := span(a); (bs, be) := span(b)
  return as < be and bs < ae
```

The service applies this function to every worker with role WORKER (`createSubRequest`). The owner is not in this set, and the order of the checks follows the activity diagram AD-2 in Section 3.9. With five workers and at most a few shifts per worker and day, the cost is negligible. The half-open comparison makes shifts that meet at a boundary, such as 10:00–16:00 and 16:00–22:00, non-overlapping (TC-011).

4.8.2 Weekly summary and holiday allowance (BR-06, BR-07)

```
function computeWeeklySummary(W, contract, shifts, policy):
  -- shifts: the week's shifts whose current worker or original worker is W
  scheduled := 0; actual := 0; reasons := []
  if contract < 15: reasons.add(CONTRACT_BELOW_15)
  for s in shifts ordered by date and start:
    original := s.originalWorkerId if set, else s.workerId
    if s.workerId = W:
      scheduled += duration(s)
      if s.status = WORKED and s has clockIn/clockOut:
        actual += duration(s.clockIn, s.clockOut)
    if original ≠ W: continue -- a shift W took as substitute
    if s.workerId = W and s.status = ABSENT: reasons.add(ABSENT on s.workDate)
    if s.workerId ≠ W and policy = ABSENT: reasons.add(GAVE_AWAY on s.workDate)
  perfect := no reason other than CONTRACT_BELOW_15
  eligible := contract ≥ 15 and perfect
  hours := round2(min(contract, 40) / 40 × 8) if eligible else 0
  return (contract, scheduled, actual, perfect, eligible, hours, reasons)
```

Three properties of the rule are visible in the pseudocode. A shift the worker took as a substitute adds to scheduled hours but is skipped for attendance (the `continue`), so an absence on someone else's shift does not cost the substitute his own allowance (TC-087). A SCHEDULED shift is never a reason, so a week in progress is a projection (TC-085). And the policy is consulted in exactly one place. Holiday pay is computed later, per week: $\text{floor}(\text{hours} \times \text{hourlyWage})$, and the month sums the rounded weekly amounts (BR-07). For Choi Minho, $\text{contract} = 16$, so $\text{hours} = \text{round2}(16 / 40 \times 8) = 3.2$ and $\text{pay} = \text{floor}(3.2 \times 10,320) = 33,024$.

4.8.3 Premium hours (BR-08)

```
function premiumHours(shifts, regularEmployees):
  if regularEmployees < 5: return (daily 0, weekly 0, night 0, total 0)
  byDay := map date → hours; night := 0
  for s in shifts: -- worked or scheduled intervals of the
    month
      (a, b) := span(s)
```

```

    byDay[s.workDate] += (b - a) / 60
    night += minutes of [a, b) inside 22:00–06:00 windows / 60
    daily := 0; byWeek := map monday → regular hours
    for (date, h) in byDay:
        extra := max(0, h - 8); daily += extra
        byWeek[weekStartOf(date)] += h - extra    -- only hours not already counted as daily
overtime
    weekly := Σ over weeks of max(0, regular - 40)
    return (daily, weekly, night, daily + weekly + night)

premiumPay := floor(total × hourlyWage × 0.5)

```

The night windows are checked against three intervals on the minute axis, from 22:00 on the previous day to 06:00, from 22:00 to 06:00 of the next day, and the following night, which covers any shift of less than 24 hours. Weekly overtime is computed from the regular hours only, so an hour that already earned daily overtime is not counted again (TC-094). Because the function receives only the shifts of the payroll month, a week that spans two months contributes only its days inside the month, as BR-08 requires.

4.8.4 Probation reduction and minimum-wage check (BR-09, BR-10)

```

function probationApplies(worker, date):
    if worker.contractStart is empty or worker.probationEnd is empty: return false
    if worker.simpleLabor: return false
    if worker.contractEnd is set and daysBetween(contractStart, contractEnd) < 365: return false
    if date < contractStart: return false
    if date ≥ addMonths(contractStart, 3): return false
    return date ≤ probationEnd

minimumWageFor(year) := hourly of the latest MinimumWage row with row.year ≤ year
floor    := minimumWageFor(year) × (0.9 if probationApplies(worker, evaluationDate) else 1)
warning  := worker.hourlyWage < floor
evaluationDate := max(first day of the month, contractStart)

```

Every condition of BR-10 is a separate early return, so each can be tested on its own (TC-102): the simple-labour flag, a contract shorter than 365 days, a date three months or more after the contract start, a date after the probation end, and a missing probation end. `addMonths` clamps the day to the length of the target month, so a contract starting on 31 August ends its three months on 30 November. For the seed data in September 2026, Jung Hana's floor is $0.9 \times 10,320 = 9,288$ won and Kang Doyun's is 10,320 won; his wage of 10,000 won produces the warning.

4.8.5 Monthly payroll (BR-11)

```

function generatePayroll(yearMonth):                -- services.js, inside tx()
    weeks := Mondays of the weeks whose Sunday lies in yearMonth
    for monday in weeks: recomputeWeek(monday)
    delete DRAFT rows of yearMonth
    for W in workers with role WORKER:
        if W has a CONFIRMED row for yearMonth: continue
        shifts := W's shifts in yearMonth with status ≠ ABSENT, each as
            (clockIn, clockOut, estimated = false) if WORKED with a record
            else (startTime, endTime, estimated = true)
        holidayWeeks := W's WeeklySummary.holidayHours for the weeks above
        if shifts empty and no holiday hours: continue
        row := computePayrollRow(W.hourlyWage, shifts, holidayWeeks,
                                regularEmployees, minimumWageFor(year),
                                probationApplies(W, evaluationDate))
        insert row as DRAFT

function computePayrollRow(...):                    -- rules.js
    baseHours := round2(Σ duration(s))
    basePay   := floor(baseHours × wage)
    holidayPay := Σ floor(h × wage) for h in holidayWeeks
    premiumPay := floor(premiumHours(shifts, regularEmployees).total × wage × 0.5)
    return (baseHours, basePay, holidayPay, premiumPay,
            total = basePay + holidayPay + premiumPay,
            minWageWarning = wage < floor, estimated = any s.estimated)

```

Applied to Choi Minhø in September 2026 with the seed data, the week of 2026-09-21 was worked (16 h) and his shift on Tuesday 2026-09-29 is still scheduled (5 h, estimated), so base hours are 21 and base pay is $21 \times 10,320 = 216,720$ won. The week of 2026-09-21 is the only generated week with its Sunday in September, so holiday pay is 33,024 won and the total is 249,744 won; the row carries the estimated flag. These values are asserted by TC-114.

4.9 User-Interface Design

The user interface was redesigned in iteration 4 (Chapter 5) after a usability review of the running application. The earlier design followed the use cases: one screen per function (inbox, approvals, weekly summary, payroll and so on), each reached from a menu. The review found that this made the user find the right screen before acting, and that the most important information — what an action does to someone's hours and pay — was on a separate screen from the action. The final design, fixed in section 2 of the final prompt and in NFR-13, is organised around the user's tasks instead. This section describes its principles, its navigation, its main screens and its visual tokens; Chapter 7 shows the screens.

4.9.1 Design principles

Eight principles apply to every screen and take precedence over the description of an individual screen.

Table 43. User-interface principles of the final design

#	Principle	Meaning in ShiftSwap
1	One screen, one job	Every screen has exactly one primary action, drawn as the only filled blue button; on phones it is a full-width button fixed above the tab bar. Other actions are grey or text buttons.
2	What needs me comes first	Each role opens on a <i>Home</i> screen that lists what is waiting for this person, most urgent first, each item with its own action button.
3	Show the consequence before the commitment	Accepting, approving and confirming payroll show their effect in one sentence with numbers before the button, for example "Minho: this week 16 h → 21 h · holiday allowance unchanged (33,024 won)".
4	Ask one thing at a time	Requesting a substitute is a three-step flow (choose shift → reason and deadline → review and send) with a step indicator and a review that names the recipients.
5	Say what happened and what happens next	Every action ends with a result screen or message that states the outcome and the next step; no state changes silently.
6	Status as progress	A request is shown as a three-step tracker Requested → Accepted → Approved; rejected, expired and cancelled requests show a labelled end state with the reason.
7	The user's words	Korean is the default language, in the polite everyday style; English is available in the menu. Identifiers such as FR, UC, BR and status codes never appear. Dates read like "9월 30일(수) 18:00–23:00" or "Wed, Sep 30 · 18:00–23:00", with relative times where they help ("by 21:00 tomorrow", "D-2").
8	Nothing blocks without a reason	A disabled button always has a one-line explanation under it, for example that one minimum-wage warning must be acknowledged before the payroll can be confirmed.

Principles 2 and 3 carry the core idea of ShiftSwap into the interface. The system already knew, at the moment of each decision, what the decision would do to both workers' hours and holiday allowance (FR-11); the redesign moves that knowledge onto the decision itself, so a candidate sees the effect of accepting and the owner sees the effect of approving without opening another screen.

4.9.2 Navigation

Each role has four main destinations, shown as a left navigation column of 220 px on screens of 900 px and wider and as a bottom tab bar of 64 px on narrower screens. The tab bar is hidden while the request flow is open. The table below maps the destinations to the use cases.

Table 44. Navigation of the final design: destinations per role and the use cases they serve

Role	Destination	Content	Use cases
------	-------------	---------	-----------

Role	Destination	Content	Use cases
Worker	Home (홈)	Requests received with Accept / Decline; own open requests as trackers; next shift with <i>Find a substitute</i> ; this week's hours and holiday allowance; shifts to record	UC-04, UC-05, UC-07, UC-12, FR-21
	Schedule (근무표)	Week as time grid (desktop) or day list (phone); shift detail with the actions the viewer may take	UC-03 (view), UC-04
	Swaps (대타)	Received, sent and past requests; primary action <i>Find a substitute</i> starts the three-step flow	UC-04, UC-05, UC-12
	Me (내 정보)	Availability editor, attendance records, own phone and wage	UC-02, UC-07
Owner	Home (홈)	"N things to handle today": decision cards for accepted swaps with impact lines and <i>Approve</i> ; requests waiting for replies; attendance to confirm; payroll warnings; today's and this week's staffing	UC-06, UC-07, UC-09, FR-21
	Schedule (근무표)	Time grid; <i>Create this week's schedule</i> for an empty week, otherwise <i>Add shift</i> ; shift detail with change time, change worker, delete	UC-03
	Staff (직원)	Worker cards; worker detail with the fixed-schedule editor; primary <i>Add worker</i>	UC-01
	Pay (급여)	Segments <i>Weekly summary</i> and <i>Monthly payroll</i> ; month and week chosen with < >	UC-08, UC-09
Both	Header	Store name; bell icon with unread count opening the notification list; compact demo-clock chip opening <i>Demo tools</i> ; current person with <i>View as someone else</i> ; menu with Settings (owner), language and sign-out	UC-13, UC-10, UC-11, FR-21

The menus per role are no longer fixed in the router alone. `services.js` exports the pure function `menuFor(role)`, which returns the destinations a role may open; the router uses it both to draw the navigation and to redirect any other route (FR-20). This makes the role restriction testable without a browser. Two further service queries, `getWorkerHome(workerId)` and `getOwnerHome()`, return in one call everything a Home screen needs, including the before/after effect of accepting or approving, so the Home screens contain no business logic of their own.

The *Demo tools* panel (a sheet on phones, a pop-over on desktops) holds what is needed only for demonstration: the demo-clock input, *+1 hour*, *+1 day* and *Reset demo data* with its confirmation. In the earlier design these controls occupied the header permanently.

4.9.3 Key screens

- **Sign-in as a demo guide (UC-13).** The title "달빛카페 데모" and the sentence that the demo needs no passwords are followed by a *3-minute tour* card with three numbered steps: view Lee Seoyeon's request, accept it as Choi Minhoo, approve it as the owner Park Jiyoung. Each step signs in as that person and opens the screen where the step happens, and completed steps are marked. Below, all five accounts are listed with their one-line summaries.
- **Worker Home.** In order, each section omitted when empty: a greeting with today's date; *Requests for you*, one card per pending request with the requester, shift, reason, deadline countdown, the effect on the worker's own week, and *Accept* (one tap, NFR-01) / *Decline*; the worker's own open requests as trackers; the next shift with *Find a substitute*; this week's hours with the holiday allowance in words and money and a bar against the 15-hour line; past shifts still to record.
- **Request flow (UC-04).** Step 1 lists the worker's own future scheduled shifts, with a shift that already has an open request shown disabled and explained. Step 2 offers reason chips (exam, hospital, family event, personal, free text) and deadline chips ("24 hours from now", "2 hours before the shift", custom), with the default of BR-03. Step 3 summarises the request and names the recipients computed at that moment (BR-01), or warns that nobody is available and that the owner will be told (FR-08). The result screen shows the tracker at its first step.
- **Owner Home.** A heading with the number of things to handle; a decision card per accepted swap with a sentence ("Seoyeon's Wednesday 18:00–23:00 shift is covered by Minhoo"), one impact line per person (scheduled hours before → after, holiday allowance before → after, a change highlighted in amber) and *Approve / Reject*, with *Details* opening the full table of FR-11; then requests waiting for replies, attendance to confirm, payroll warnings, and today's and this week's staffing.

- **Schedule (UC-03).** The time grid of the previous design on wide screens (hour axis from the earliest start to the latest end of the week, 44 px per hour, overlapping shifts side by side) and a day list with today expanded on phones. Selecting a shift opens a detail panel with its history ("Seoyeon → Minho · approved 9/28") and only the actions the viewer may take.
- **Pay (UC-08, UC-09).** The weekly summary as one card per worker, with eligibility stated in words and the reason ("contract 10 h per week · below 15 h"). The monthly payroll with *Create September payroll draft*, rows or cards with flags (estimated, probation minimum 90 %, below minimum wage), the below-minimum sentence with *I have checked* (acknowledgeMinWage), and *Confirm September payroll*, disabled with its reason until all warnings are acknowledged.
- **Staff, Me and Settings.** Worker cards and a fixed-schedule editor with weekday chips; the availability editor with the same pattern; settings in which the number of regular employees and the substitution-attendance policy each carry a sentence explaining their effect.

4.9.4 Visual design

The visual design is specified in section 3 of the final prompt with exact values, so that a regenerated system looks the same. The table below lists the design tokens.

Table 45. Design tokens of the final user interface

Token	Value	Use
--ink / --ink-2 / --ink-3	#191F28 / #4E5968 / #8B95A1	Main text; secondary text; captions
--surface	#F2F4F6	Page background
--paper	FFFFFF	Cards
--line	#E5E8EB	Dividers and grid lines
--shift / --shift-soft	#2B5BD7 / #EBF1FD	Primary buttons and the user's own shifts / their background
--handover / --handover-soft	#E9A23B / #FDF3E1	Substitution: handover edge and stripe, pending outline, changed eligibility
--alert / --alert-soft	#D93A30 / #FDECEB	Errors and the minimum-wage warning
--ok	#1F8A55	Eligible, confirmed, success
Type	Pretendard Variable	One font for all text, with tabular figures for times, hours and money. Body 16 px / 1.6; Home heading 26 px / 700; card title 17 px / 700; key number 28 px / 700; captions 13 px, never below 12 px.
Cards	16 px radius, 20–24 px padding	White on the grey page, no border, no shadow; 12 px between cards, 32 px between sections.
Buttons	52 px (phone) / 44 px (desktop) high, 12 px radius	Primary filled blue; secondary filled grey; text buttons; pressed state scales to 96 %; touch targets at least 44 × 44 px.
Icons and motion	24 px line icons; 180 ms	Inline SVG icons for tabs, bell, clock and chevrons; sheets and step changes slide and fade in 180 ms; no motion under prefers-reduced-motion.

As in the earlier design, one element carries meaning through decoration: a handed-over shift has a 6-pixel amber left edge and a faint 45° amber stripe — the only gradient in the style sheet — with the new worker's name in bold and the original worker struck through, and a shift with an open request has a dashed amber outline and the chip "대타 구하는 중" (looking for a substitute). Content is limited to 720 px width for Home, flows and lists, and uses up to 1,200 px for the schedule grid and the payroll table; at 360 px no screen scrolls horizontally (NFR-02). Accessibility rules remain: visible focus rings, real buttons, labelled inputs, status never shown by colour alone, and a step indicator that is announced to screen readers ("step 2 of 3").

5. Implementation by Vibe Coding

ShiftSwap was implemented by vibe coding: its source code was generated by an AI coding assistant from natural-language prompts, and the team's work consisted of writing the specification and the prompt, running and reviewing the result, and revising the prompt until it reliably produced the system specified in Chapters 3 and 4. This chapter describes how that was done. Section 5.1 lists the development environment. Section 5.2 describes the process that was repeated in every iteration. Section 5.3 explains the structure of the final prompt, whose full text is Appendix A. Section 5.4 presents the five iterations recorded in the prompt log (Appendix B), with examples of prompt sentences before and after revision. Section 5.5 shows how the prompt maps to the code, and Section 5.6 describes the reproducibility check, in which the system was rebuilt from the final prompt in an empty folder.

5.1 Development Environment and Tools

Table 46. Development environment and tools

Tool	Version	Use in the project
AI coding assistant	agent mode with file and shell access	Generated and modified all application and test code from the prompt and the specification; ran the tests and the browser checks; recorded ambiguities in the prompt log.
Node.js / npm	v24.14.0 / 11.9.0	Running the test suite (<code>node:test</code>) and installing the one development dependency.
sql.js	1.14.2	SQLite compiled to WebAssembly; the npm package provides the files copied into <code>app/vendor/</code> and the database for the Node tests [9].
Google Chrome (headless)	current stable	Browser checks of every screen at 1440 px and 360 px; printing the reports to PDF.
PlantUML	1.2025.2	Drawing the UML diagrams of Chapters 3 and 4 from text sources in <code>uml/src/</code> [14].
Git / GitHub	2.48.1	Version control of specification, prompts, code, models and reports in the repository <code>keemlogan/shiftswap</code> .
GitHub Pages	—	Static hosting of the application at https://keemlogan.github.io/shiftswap/app/ [13].

The application itself has no run-time dependency other than the vendored `sql.js` files; there is no framework, no bundler and no build step (NFR-08). The only entry in `package.json` besides the test script is `sql.js` as a development dependency for the tests.

5.2 The Vibe-Coding Process

The team used the same cycle for every version of the prompt. The figure below shows it; the numbered steps explain each stage and who performed it.

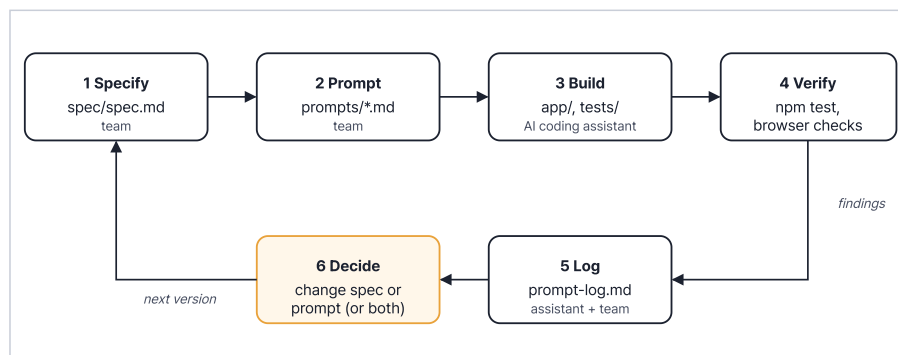


Figure 18. The vibe-coding cycle used for every prompt version

1. **Specify.** Everything the system must do is first written into `spec/spec.md`: actors, use cases, requirements, business rules, states, the SQL schema, the seed data and the test groups. The file is not reproduced in this report; it is `spec/spec.md` in the repository (<https://github.com/keemlogan/shiftswap/blob/main/spec/spec.md>). The specification

is the contract. A behaviour that is not in it is not requested, and a behaviour that is in it may not be changed by the prompt.

2. **Prompt.** The prompt tells the AI coding assistant *how* to build what the specification defines: the technology, the file layout, the names of the functions, the screens, the visual design and the conditions under which the work is finished. It refers to the specification by identifier rather than repeating it.
3. **Build.** The prompt, followed by the complete specification, is given to the AI coding assistant in an empty folder. The assistant creates the files, installs `sql.js`, vendors its WebAssembly files, writes the tests and runs them.
4. **Verify.** The team runs `npm test` and walks through the scenarios of Section 3.8 in the browser, including the done criteria of the prompt. Browser checks at 1440 px and 360 px are scripted with headless Chrome.
5. **Log.** Whenever the specification was silent or ambiguous, the assistant was instructed to make a reasonable choice, implement it, and record it in the prompt log in the form "what was unclear → what the implementation does → suggested wording for the next version". Review findings from the team were added to the same log.
6. **Decide.** For each logged item the team decided whether it belongs in the specification (it changes what the system does) or only in the prompt (it changes how the system is built), made the change, and started the next version. The rule of Section 3.11.3 applies: requirements change in the specification first.

The division of work in this cycle is deliberate. The AI coding assistant is fast at producing code but has no knowledge of the reference store or of the team's interpretation of the law, so every decision about behaviour is taken by the team and written down. Conversely, the working rule was not to fix behaviour by editing generated code by hand. When a result was wrong, the cause was traced to the prompt or the specification and corrected there, so that the next build — and any future rebuild from the final prompt — produces the corrected behaviour without depending on a manual patch.

5.3 Structure of the Final Prompt

The final prompt is version 5. It is 125 lines of Markdown and is used together with the 252-line specification, which is appended to it verbatim, and with the two acceptance test files, which are placed in the empty folder before the build starts. The full text is reproduced in Appendix A. The table below describes its parts in order and the reason each exists.

Table 47. Parts of the final prompt (version 5) and their purpose

Part	Content	Why it is there
Header and usage note	Version history (v1 → ... → v5, with the reason for each version and pointers to the earlier versions and the log); how to use the prompt: place the two test files in an empty folder, then paste the prompt followed by the complete specification into a coding agent.	Makes the prompt self-contained and states the reproduction procedure, so that anyone can repeat the build (Section 5.6).
Role and contract paragraph	What is being built and for whom; the instruction to treat the specification as the contract, to use its identifiers, table and column names, state names and seed data exactly, and not to add features that are not in it.	Prevents the most common failure of generated code: plausible but unrequested features and renamed identifiers that break traceability.
§0 Acceptance tests	The supplied files <code>tests/rules.test.js</code> and <code>tests/services.test.js</code> are the executable form of the specification and fix the API: module paths, exported names, argument order, return shapes, error codes and the Node-side database factory. They must be read first and must not be modified; where a test and the prose disagree on an API detail the test wins, and a disagreement on behaviour must be reported, not guessed.	Added in version 5 after reproduction run 1 (Section 5.6.2): names alone did not fix the interfaces, so two correct rebuilds disagreed at the API level. Tests are an unambiguous contract for signatures that prose cannot state compactly.
§1 Technology constraints	Static site with ES modules and no build step; <code>sql.js</code> vendored into <code>app/vendor/</code> ; persistence keys; the complete folder layout with the name of every module and every exported function; test files, test naming and the test script; when <code>expireOverdue()</code> runs.	Fixes the architecture of Chapter 4 before any code exists. Naming every service and rule function makes the code predictable across rebuilds, lets the tests of one build run against another, and keeps the traceability matrix valid.

Part	Content	Why it is there
§2 Screens and behaviour	§2.0 eight UX principles that override any individual screen description (one primary action per screen, consequence before commitment, one question at a time, and others); §2.1 navigation per role and the Demo tools panel; §2.2–§2.8 the sign-in demo guide, worker Home, request flow, owner Home, Schedule, Pay, and Staff / Me / Settings, each with its sections in order and the exact Korean wording of key labels.	The specification says what the system must do; this part says how the user meets each function. Stating the principles first gives the assistant a rule for every case the screen descriptions do not cover. It carries NFR-01, NFR-11 and NFR-13 into concrete screen behaviour.
§3 Visual design	Fonts, type scale, colour tokens, the single decorative element (the amber handover marker), layout widths and breakpoints, copy rules, accessibility rules; the heading says "do not deviate".	Without exact values, each generation produced a different look. Fixed tokens make the result reproducible and keep the interface consistent with the design of Section 4.9.
§4 Done criteria	Five checkable conditions: all test groups pass, including named boundary cases; the seed state and walkthrough S7 are reproduced; the app starts without console errors; the sign-in tour works from Seoyeon's tracker through Minho's one-tap acceptance to the owner's approval and the handover marker; every screen has exactly one primary button, every disabled button a visible reason, and all touch targets at least 44 × 44 px.	Tells the assistant when it has finished and gives the team the acceptance tests of Section 6.1. The service part of criterion 4 is automated as TC-063; criterion 5 is a headless count.
Specification (appended)	The complete spec/spec.md: §0–§14.	The contract itself. Keeping it in a separate file lets the reports and the UML models refer to the same text.

Three principles run through the prompt. First, it is **specific where variation is harmful** — names, schema, keys, colours, sizes — and says nothing where variation does not matter, such as the internal structure of a screen module. Second, it **refers rather than repeats**: rules are cited by their BR identifier, so there is only one definition of each rule. Third, it **states its own acceptance test**. A prompt whose done criteria are "the walkthrough of criterion 4 works" can be checked; a prompt that asks for "a good substitute workflow" cannot.

5.4 Iterations of the Prompt

The prompt went through five iterations, summarised in the table below and recorded in detail in the prompt log (Appendix B). Each of the first three iterations ended with the full test suite passing and a set of browser checks.

Table 48. Overview of the prompt iterations

Iteration	Prompt	Source of changes	Result
1	v1 → build	First build of the whole system from prompt v1; 21 ambiguities recorded by the AI coding assistant	Working system; 52 tests, all passing
2	v1 → v2	The 21 build ambiguities, plus gaps found in the document review of the proposal and interim report against the specification (20 numbered changes)	58 tests (6 new), 58 passing; headless Chrome 70 of 70 checks passed, 0 console errors
3	v2 → v3	UI review of the running v2 application (8 changes to screens and visual design; specification unchanged)	58 tests, 58 passing (no test changes needed); headless Chrome 82 of 82 checks passed, 0 console errors

Iteration	Prompt	Source of changes	Result
4	v3 → v4	UX redesign review of the running v3 application: task-first Home per role, one primary action per screen, consequence shown before commitment, three-step request flow, sign-in as a guided demo tour, Korean as default language; specification gains NFR-13 and a new wording of FR-20	Reported in Section 5.4.4 after the rebuild
5	v4 → v5	Reproduction run 1 (Section 5.6.2): a clean rebuild passed its own tests but not the team's, because the prompt fixed function names but not signatures and return shapes; the acceptance test files become part of the input. An independent code review found six defects, fixed in the same iteration. Specification: BR-02 (acceptor re-checked for overlap, ACCEPTOR_BUSY) and BR-10 (exact three-month window) clarified	Reported in Section 5.4.5

5.4.1 Iteration 1: building from version 1

Prompt version 1 was written after the requirements and design models, as planned for Week 9 of the schedule. The first build produced a working system, and the assistant recorded 21 points at which it had to make a choice that the specification did not determine: six concerned the precision of business rules (for example, whether a week in progress can have perfect attendance, and the rounding of holiday pay), six the data and seed, three the workflow and notifications, three the technical set-up and three the user interface. The individual findings, their resolution and the tests that verify them are listed in Section 6.9.

In nearly every case the implementation had already made a sensible choice. The value of the log was that it made these choices visible, so that the team could confirm them and write them into the contract; otherwise the next build might have chosen differently, and the behaviour described in this report would not be reproducible.

5.4.2 Iteration 2: version 2 (build ambiguities and document review)

Version 2 folded in all 21 findings of iteration 1 and the findings of a document review, in which the proposal and the interim report were checked against the specification. The review found, among other things, that the minimum-wage acknowledgement had no place in the data model, that the recipients of a cancelled request were not defined for UC-12, and that no scenario ever changed a worker's eligibility. Twenty numbered changes were made (Section 6.9 lists them with their tests). Two changed the code: the new column `Payroll.minWageAck` with the service `acknowledgeMinWage`, and the replacement of a saved database that predates the new column. Six tests were added.

The table below shows how the wording changed for four representative items. The first two changed the specification, the other two the prompt.

Table 49. Examples of wording before and after iteration 2

Where	Before (v1)	After (v2)
BR-06 (spec)	"perfect attendance: every shift of the week whose original worker is W and that was not given away was WORKED (not ABSENT)."	"... was not ABSENT (a shift that is still SCHEDULED does not break attendance, so the weekly result is a projection until the week is over)."
FR-19 (spec)	"A requester shall cancel their own request while it is REQUESTED or ACCEPTED; all targets are closed and notified."	"... all PENDING targets become CLOSED, and every target that was PENDING or ACCEPTED receives a REQUEST_CANCELLED notification."

Where	Before (v1)	After (v2)
Prompt \$1	"package.json with "type": "module" and script "test": "node --test tests/"	"... script "test": "node --test tests/*.test.js" (a bare directory argument fails on Node 24)."
Prompt \$2, Payroll	"minimum-wage warning row that must be acknowledged before "Confirm", confirmed rows read-only."	"minimum-wage warning on the row with an Acknowledge button (acknowledgeMinWage sets Payroll.minWageAck = 1, persisted); "Confirm" is disabled until every warning row is acknowledged; confirmed rows read-only; rows with estimated = 1 carry an "Estimated" chip."

The first example shows a requirements error that only an implementation could reveal. As written in v1, BR-06 made every worker ineligible until the week was over, because future shifts are not yet WORKED; the approval panel could therefore never show Choi Minhø as eligible, which contradicted the worked example of the specification. The fourth example shows the typical development of a prompt sentence: a requirement stated as an intention ("must be acknowledged") becomes a design with a stored field, a named function and a defined screen state.

5.4.3 Iteration 3: version 3 (UI review)

The team reviewed the running version 2 on desktop and mobile screens. The review found three problems: the text was too small, the content sat in a narrow centred column with large empty areas on wide screens, and the weekly board read as a list of cards rather than a schedule. All changes were made in sections 2 and 3 of the prompt; the specification did not change, so the service layer and the tests did not change either. The table below lists the eight changes.

Table 50. Changes of iteration 3 (UI review)

#	Change in the prompt	Change in the code	Verification
1	Sign-in rows show name, role and a one-line summary of the person's seed data	ui/login.js builds the summary from the fixed schedule and contract hours	Headless: all five summaries as expected
2	Every page starts with a title and one sentence on its purpose	pageHead() helper in ui/i18n.js, used by all eleven screens; sentences in English and Korean	Headless: every route for both roles
3	Owner overview strip above the board: approvals waiting, open requests, this week's shifts and hours	overviewStrip() in ui/schedule.js	Headless: "9 shifts · 50 h" for the seed week
4	Weekly board as a time grid with an hour axis, 44 px per hour, blocks placed and sized by time	ui/schedule.js rewritten around board() and layoutDay(); overlapping shifts share the width in lanes	Headless geometry: Sat 10:00–16:00 is 264 px high
5	Empty states say what will appear and offer the next action	emptyState() helper; used on seven screens	Manual review
6	Type scale: base 15 px; page title 28 px, weight 800; small text never below 12 px; inputs 16 px on mobile	styles.css heading and body rules	Headless: page title computes to 28 px / 800
7	Navigation 220 px; content left-aligned after it with 32 px padding; gaps between groups at least twice those inside a group	Grid layout; group spacing on the view container	Headless: navigation width 220 px
8	Kept: day-by-day list below 900 px, stacked-card tables below 720 px	Below 900 px the grid drops its axis and blocks become cards in day order	Headless: every screen, both roles, both languages, 1440 and 360 px, no horizontal scroll

The weekly board illustrates how an imprecise prompt sentence leads to an acceptable but wrong result. Version 2 asked for "seven columns Monday–Sunday, rows are shifts ordered by start time". The assistant implemented exactly that: each day was a column of cards in time order. The result met the words but not the purpose, because the owner could not see at a glance when the café was staffed. Version 3 describes the intended picture instead: "a time grid ... a vertical hour axis from the earliest shift start to the latest shift end of the displayed week, rounded to whole hours (seed: 10:00–23:00), 44 px per hour ... Each shift is a block positioned and sized by its start and end time". The concrete values, including the expected axis for the seed data, made the result checkable by measuring pixel positions in the headless browser.

The verification of iteration 3 also found one defect of its own: the 32-pixel gap between groups had first been set on the outer content element, but the screens render inside an inner view element, so the header, the overview strip and the week navigation touched each other. The gap was moved to the view element, and the check was repeated.

5.4.4 Iteration 4: version 4 (UX redesign)

The fourth iteration was started after the review of the version 3 application from the point of view of a consumer-app product team. The review accepted the functions and the visual quality of version 3 but found that the interface still followed the structure of the use cases rather than the tasks of its users: a candidate had to open the inbox to find a request, the owner had to open the approvals page to see the effect of a swap, and every screen offered several actions of equal weight. Version 4 therefore rewrote section 2 of the prompt around eight UX principles and a new navigation (Section 4.9), rewrote section 3 with new design tokens, and added done criterion 5. The specification gained the usability requirement NFR-13 and a new wording of FR-20 (sign-in through a guided demo tour or the account list); the business rules, the schema and the service commands did not change. Two read queries (getWorkerHome, getOwnerHome) and the pure function menuFor(role) were added to the service layer so that the new Home screens and the role menus contain no logic of their own.

The table below shows how two prompt sentences changed.

Table 51. Examples of wording before and after iteration 4

Where	Before (v3)	After (v4)
Prompt \$2, inbox / Home	" Inbox (UC-05, FR-21) : notifications newest first; a REQUEST_RECEIVED item for a PENDING target has Accept and Decline buttons directly on the item (NFR-01: at most 2 clicks)."	" 대타 요청이 왔어요 — one card per PENDING target: who asks, the shift, reason, deadline countdown, the effect on me ("수락하면 이번 주 16시간 → 21시간 · 주휴수당 그대로"), and two buttons 수락하기 (primary) / 거절 (text). Accept is one tap (NFR-01)."
Prompt \$2, request	" Request substitute (UC-04) : from one of the worker's own future shifts, open a form: reason (optional), deadline (default: the earlier of now + 24 h and shift start)."	"Requesting a substitute is a three-step flow (choose shift → reason and deadline → review and send) with a step indicator ("1/3"), a back button, and a review screen that names the people who will receive the request."

The results of the version 4 build — the number of changes made to the code, the test results and the browser checks — are recorded in the prompt log (Appendix B).

5.4.5 Iteration 5: version 5 (acceptance tests as input)

The fifth iteration followed the first reproduction run, described in Section 5.6.2. The rebuilt system worked and passed its own 54 tests, but only 13 of the team's 25 rule tests passed against it, and the team's service tests could not be loaded at all, because the rebuild had given the database factory a different name. The diagnosis was that the prompt fixed the names of modules and functions but not their signatures and return shapes, so two implementations that both satisfy the specification disagreed at the level of the interfaces. Version 5 therefore adds a section 0 to the prompt: the two test files are supplied with the prompt, must be read first and must not be changed, and they define the API. The same iteration folded two points reported by the clean session into the specification — the acceptor is re-checked for an overlapping shift at acceptance and at approval (BR-02, error ACCEPTOR_BUSY), and the three-month probation window is defined exactly (BR-10) — and fixed six defects found by an independent code review of the reference implementation. The defects and their fixes are listed in the prompt log (Appendix B).

5.5 From Prompt to Code

Section 1 of the prompt names every file and every exported function, so the structure of the code follows directly from the prompt. The tree below shows the files of the implementation with their line counts.

shiftswap/		
├─ app/		
│ ├─ index.html	22	page skeleton, loads vendor/sql-wasm.js and main.js
│ └─ main.js	225	bootstrap, hash router, header, role menus, System Clock
└─ tick		
├─ styles.css	358	design tokens, layout, board, responsive rules
└─ core/		
├─ clock.js	41	demo clock (A3)
├─ db.js	219	schema (spec §8), seed (spec §10), tx(), persistence
├─ rules.js	257	BR-01 ... BR-11 as pure functions
└─ services.js	754	one function per use-case step; read queries
└─ ui/		
└─ login.js	48	UC-13

— schedule.js	232	UC-03 weekly board (time grid), overview strip
— requests.js	104	UC-04, UC-12
— inbox.js	59	UC-05, FR-21
— approvals.js	110	UC-06
— attendance.js	79	UC-07
— weekly.js	37	UC-08
— payroll.js	107	UC-09
— workers.js	100	UC-01
— availability.js	71	UC-02 (and the shared slot editor)
— settings.js	65	UC-10
— i18n.js	764	English and Korean text, formatting helpers
— vendor/		sql-wasm.js, sql-wasm.wasm (sql.js 1.14.2)
— tests/		
— rules.test.js	214	unit tests of rules.js
— services.test.js	491	integration tests of services.js
— package.json	13	
— spec/spec.md	252	the contract (not reproduced; see the repository)
— prompts/		final-prompt.md (v5), prompt-v1.md ... prompt-v4.md,
prompt-log.md		

Line counts from `wc -l`; application code 3,652 lines (core 1,271, presentation 2,381) and test code 705 lines, excluding the vendored `sql.js` files.

Because the prompt names the modules and functions, the mapping from prompt to code is exact: the contract paragraph and spec §8 and §10 produce `core/db.js` (checked by TC-001 and TC-002); the rule and service lists of §1 produce `core/rules.js` and `core/services.js` (checked by the two test files); §2 produces the screen modules and the router; §3 produces the style sheet (checked in the headless browser); and §4 defines the acceptance checks of Section 5.6. This exactness is the reason the traceability matrix of Section 6.8 could be derived from the code by searching for function names.

5.6 Reproducibility Check

The course requires a final prompt that, when re-run through vibe coding, reliably reproduces a working system. A prompt that was refined on one code base may, however, depend on that code base without anyone noticing: a sentence may be understandable only to an assistant that has seen the previous version. The reproducibility check therefore rebuilds the system from nothing and tests the result with the team's test suite.

5.6.1 Method

1. **Clean start.** A new session of the AI coding assistant is started with no conversation history, in an empty folder that is outside the repository. The session has no access to the repository, the existing code, the prompt log or the reports.
2. **Input.** The session receives exactly what Appendix A prescribes: the text of the final prompt followed by the complete specification, the specification file `spec/spec.md` in the repository (<https://github.com/keemlogan/shiftswap/blob/main/spec/spec.md>). From version 5 on, the two team test files are placed in the folder before the session starts, as the prompt prescribes. No other instruction is given, apart from answering "continue" if the session stops before it reports completion.
3. **Build.** The assistant builds the system on its own, including its own tests, and reports when it considers the done criteria of the prompt met. The number of interventions and the elapsed time are recorded.
4. **Own tests.** The rebuilt system's own `npm test` is run and its result recorded.
5. **Team test suite.** The team's test files `tests/rules.test.js` and `tests/services.test.js` are run against the rebuilt code. In run 1 (prompt v3) they were copied into the rebuilt folder after the build, replacing the rebuilt tests; from run 2 (prompt v5) they are part of the input and must pass unchanged. Because the prompt fixes the module paths, the exported function names and the schema, the team's tests can call the rebuilt code directly. This is the decisive measurement: it checks the rebuilt system against the same 58 or more test cases that the reference implementation passes, including the boundary values of Section 6.5, the concurrency test of Section 6.6 and the notification recipients of spec §8.
6. **Done criteria 2–4.** The rebuilt application is served with a static server and opened in headless Chrome: the sign-in screen must appear without console errors (criterion 3), and the walkthrough of criterion 4 is performed. Criterion 2 (seed state and walkthrough S7) is covered by TC-001 and TC-08B of the team suite.
7. **Comparison.** Any failing team test is traced to its cause and classified as (a) a genuine deviation from the specification, (b) an ambiguity in the prompt or specification that the rebuild resolved differently, or (c) a difference that the tests over-specify (for example the wording of an error message) and that does not affect a requirement. Categories (a) and (b) are defects of the prompt or specification and are reported as such.

Two limits of the method should be stated. Each rebuild is a single run, and a generative assistant does not produce the same code twice, so one successful run shows that the prompt *can* reproduce the system, not that every run will. And the team's

tests check the rules, services and data; the layout of the screens is checked only by the done criteria and the headless checks, not pixel by pixel against the reference.

5.6.2 Results

Run 1 (prompt version 3)

The first run was made on 2026-09-27 with the prompt as it stood after iteration 3. A new assistant session with no history and no access to the project received one file — the prompt of version 3 followed by `spec/spec.md` — in an empty folder, with the instruction to do what it says. The table below gives the results.

Table 52. Results of reproduction run 1 (prompt v3)

Measurement	Result
Size of the rebuilt application	4,540 lines of application code, including UI and CSS
Rebuilt system's own tests	54 tests, 54 passed, 0 failed
Done criteria 1–4 of the prompt	Met: headless Chrome walkthrough without console errors; "Already taken by Choi Minh"; 16 → 21 scheduled hours in the approval panel; handover marker; no horizontal scroll at 360 px
Team's tests/ <code>rules.test.js</code> against the rebuild	25 tests, 13 passed, 12 failed
Team's tests/ <code>services.test.js</code> against the rebuild	Did not load: <code>SyntaxError: The requested module '../app/core/db.js' does not provide an export named 'createDatabase' (the rebuild exports <code>initDb</code>)</code>

The twelve failing rule tests were TC-014, TC-021, TC-022, TC-023, TC-082, TC-084, TC-086, TC-101, TC-103, TC-111, TC-112 and TC-113. They fail on the form of the interface, not on behaviour: for example, whether `validateRequest` returns a list of error codes or a boolean, and how `computeWeeklySummary` reports the reason for ineligibility. At the level of the specification the rebuild reproduced the behaviour the reports rely on — the worked example of 3.2 holiday hours and 33,024 won, first acceptance wins, and walkthrough S7.

The diagnosis, recorded in `docs/reproducibility.md` in the repository, is that version 3 fixed the *names* of modules and functions but not their *signatures and return shapes*. Two correct implementations of the same specification can therefore disagree at the API level, and a test suite written against one does not run against the other. By the classification of Section 5.6.1, the failures are of category (b): an underspecification of the prompt, not a deviation from the requirements. The clean session also reported fourteen ambiguities of its own. Most were presentation choices or agreed with the reference implementation; two were folded into the specification (BR-02 and BR-10), and three API-level choices — whether the notification list runs in a transaction, re-entrant transactions, and the re-check of the acceptor — are now fixed by the supplied tests.

The change made in response is iteration 5 (Section 5.4.5): the test files became part of the input and define the API. Run 1 thus did what a reproducibility check is for. It showed that the prompt reproduced the behaviour but not the interfaces, and it turned that finding into a change of the prompt.

Run 2 (prompt version 5)

6. Testing and Validation

This chapter shows how the team checked that ShiftSwap does what Chapter 3 requires. The lecture distinguishes unit testing, integration testing, system testing, acceptance testing and regression testing [6], and it defines validation as checking that the software is what the customer wants. Section 6.1 describes the test strategy and how these levels were applied. Section 6.2 gives the test environment. Section 6.3 lists every automated test with its expected result, and Section 6.4 reports the result of running them. Sections 6.5 and 6.6 explain the two families of tests that need the most care: boundary values of the legal rules and concurrent acceptance. Section 6.7 covers the non-functional requirements, Section 6.8 the traceability matrix, and Section 6.9 the defects and specification gaps found during the build iterations and how each was resolved.

6.1 Test Strategy

The layered architecture of Chapter 4 decides where each kind of test belongs. The table below maps the lecture's test levels to what was done for ShiftSwap.

Table 53. Test levels and how they were applied

Level	Target	Method
Unit testing	Domain rules (<code>rules.js</code>)	25 tests in <code>tests/rules.test.js</code> call the pure rule functions with hand-built inputs, including the boundary values of Section 6.5. No database is involved.
Integration testing	Services with the real schema (<code>services.js</code> + <code>db.js</code> + <code>rules.js</code>)	33 tests in <code>tests/services.test.js</code> create a fresh in-memory SQLite database with the seed data before every test (<code>beforeEach</code>), reset the demo clock, call the service functions and assert on the resulting rows, notifications and error codes.
System testing	The running application in a browser	Scenario walkthroughs of Section 3.8 on the screens, for both roles, in English and Korean, at 1440 px and 360 px width (Section 6.7 and Chapter 7).
Acceptance testing	Done criteria of the final prompt	The four done criteria of the prompt (Chapter 5) serve as acceptance tests. Criterion 4, the walkthrough from acceptance to the handover marker, is also automated as TC-063.
Regression testing	Every change of prompt or code	The whole suite is run after every generation or modification of the code; a change is accepted only if all tests pass (the practice of continuous integration in the lecture's description of extreme programming [6]).

Two principles guided the choice of test cases. First, the tests were derived from the requirements, not from the code: each test group corresponds to a business rule (spec §11), and each test is named with its identifier and a sentence stating the expected behaviour, for example TC-082 no holiday allowance at 14.99 h. This is the test-case generation technique of requirements validation described in Section 3.10.3. Second, the seed data of spec §10 doubles as the test fixture. The integration tests start from the same café, the same five people and the same open request that appear in the scenarios and in the screenshots, so a number checked in a test — 3.2 holiday hours, 33,024 won, 16 → 21 scheduled hours — is the number the reader sees in this report and on the screen.

The identifiers extend the groups of spec §11 by three groups that are not tied to a business rule: TC-00x for the seed data and storage, TC-07x for attendance (UC-07), and TC-13x for privacy and notifications (NFR-11, FR-21). Within a group, the third character is a sequence number (1–9, then A, B), so TC-01A follows TC-019.

6.2 Test Environment

Table 54. Test environment

Item	Value
Test runner	Node.js built-in test runner <code>node:test</code> with <code>node:assert/strict</code> [12]; command <code>npm test</code> , which runs <code>node --test tests/*.test.js</code>
Node.js version	v24.14.0 (<code>node -v</code>)
Database library	<code>sql.js</code> 1.14.2 (from <code>package-lock.json</code>), the same WebAssembly build as <code>app/vendor/</code>

Item	Value
Machine	Apple M2, 24 GB memory, macOS 26.5
Test data	Seed data of spec §10, recreated in memory before each integration test; demo clock reset to 2026-09-28 09:00
Browser checks	Google Chrome (headless and desktop) at 1440 px and 360 px width

The service tests import the application modules unchanged. `createDatabase(SQL, {seed: true})` builds the database without browser storage, and the demo clock is set with `setNow()` where a test needs a later time, for example one minute before and at a deadline. No test reads the real system time, which makes the results the same on every run and on every machine.

6.3 Test Cases

The table below lists all 58 automated tests in the order in which they appear in the two test files. The list was extracted from the files with a script, so it contains exactly the tests that run. The column "Covers" gives the business rules, requirements or specification sections each test checks; "Expected result" states what the assertions require.

Table 55. Automated test cases (tests/rules.test.js and tests/services.test.js)

ID	Name	File	Covers	Expected result
TC-011	overlap is half-open: touching ranges do not overlap	rules	BR-01	Touching ranges 10:00–16:00 and 16:00–22:00 do not overlap; one extra minute does.
TC-012	durationHours supports end <= start as next day	rules	BR-01, BR-11	18:00–23:00 = 5 h; 22:00–06:00 = 8 h (next day); overnight ranges overlap correctly.
TC-013	weekStartOf returns the ISO Monday	rules	BR-05	Wednesday 2026-09-30 and Sunday 2026-10-04 map to Monday 2026-09-28.
TC-014	eligible when active, not requester, free and covered by availability	rules	BR-01	Choi Minhho (active, free, available Wed 17–23) is eligible for Wed 18:00–23:00.
TC-015	not eligible: requester, inactive, overlapping shift, or availability too short	rules	BR-01	Requester, inactive worker, overlapping shift, too-short slot and wrong weekday are each not eligible.
TC-021	valid request passes	rules	BR-03, FR-06	Valid request returns no error, also with deadline equal to shift start.
TC-022	only the current worker of a SCHEDULED future shift can request	rules	BR-03	NOT_OWN_SHIFT, NOT_SCHEDULED and SHIFT_STARTED are reported.
TC-023	at most one open request, deadline after now and not after shift start	rules	BR-03	OPEN_REQUEST_EXISTS, DEADLINE_NOT_FUTURE (deadline = now) and DEADLINE_AFTER_START (one minute late).
TC-081	holiday allowance at exactly 15 h	rules	BR-06, BR-07	15.00 contractual hours: eligible, 3 holiday hours.
TC-082	no holiday allowance at 14.99 h	rules	BR-06	14.99 contractual hours: not eligible, 0 hours, reason CONTRACT_BELOW_15.
TC-083	holiday hours are capped at 40 contractual hours	rules	BR-07	holidayHours(40) = holidayHours(52) = 8; holidayHours(16) = 3.2; contract hours of Minhho = 16.
TC-084	an ABSENT own shift breaks perfect attendance	rules	BR-06	An ABSENT own shift breaks perfect attendance (reason ABSENT on 09-29); actual hours 5.
TC-085	given-away shift under policy EXCUSED keeps attendance	rules	BR-06	Shift given away under EXCUSED keeps eligibility; scheduled hours drop to 5.
TC-086	given-away shift under policy ABSENT breaks attendance	rules	BR-06	Same shift under ABSENT policy: not eligible, reason GAVE_AWAY.

ID	Name	File	Covers	Expected result
TC-087	substitute shifts taken do not change contractual hours; absence on them does not count	rules	BR-06	A substitute shift taken does not change contractual hours (16); an absence on it does not remove eligibility.
TC-091	premium is zero with 4 regular employees	rules	BR-08	Premium hours are all zero with 4 regular employees.
TC-092	premium with 5 employees: daily overtime and night hours add up	rules	BR-08	10:00–23:00 with 5 employees: 5 daily overtime + 1 night hour = 6.
TC-093	overnight shift night hours 22:00–06:00	rules	BR-08	Overnight 21:00–07:00: 8 night hours and 2 daily overtime hours.
TC-094	weekly hours beyond 40 not already counted as daily overtime	rules	BR-08	Six days (8 h × 5 + 10 h): 2 daily and 8 weekly overtime hours, no double counting.
TC-101	minimum wage by year	rules	BR-09	Minimum wage 2026 = 10,320; 2025 = 10,030; 2027 falls back to 10,320; 2024 = none.
TC-102	probation applies only when all BR-10 conditions hold	rules	BR-10	Probation holds for Jung Hana on 09-15 and 11-30; fails on each missing condition separately.
TC-103	minimum-wage warning uses 90 % only under probation	rules	BR-09, BR-10	Warning for 10,000 won without probation, none with probation (floor 9,288), warning for 9,000.
TC-111	payroll totals: base + holiday + premium, rounded down	rules	BR-07, BR-11	16 h × 10,320 = 165,120 + 33,024 holiday = 198,144 won.
TC-112	premium pay at 5 employees and estimated flag	rules	BR-08, BR-11	One 18:00–23:00 shift at 5 employees: premium 5,250 won; estimated flag set.
TC-113	fractional amounts are rounded down to the won	rules	BR-07, BR-11	Fractions rounded down: 0.33 h → 3,300 won; 1.33 h holiday → 13,301 won.
TC-001	seed reproduces spec §10	services	spec §10	Seed: Dalbit Café, 4 employees, EXCUSED, two minimum-wage rows, five accounts, 18 shifts, 9 confirmed attendances, the open request with two PENDING targets.
TC-016	seed request candidates equal the BR-01 computation	services	BR-01, FR-07	Recreated seed request yields exactly Choi Minhø and Kang Doyun.
TC-017	fixed schedules of one worker must not overlap (FR-02)	services	FR-02	Overlapping fixed shifts on the same weekday are refused (FIXED_OVERLAP).
TC-018	generating the same week twice creates no duplicates (FR-04)	services	FR-04	Seed week creates 0 new shifts; a new week creates 9; a second call creates 0.
TC-019	owner edits a shift; overlap with the same worker is refused (FR-05)	services	FR-05	Owner adds, moves and deletes a shift; a move onto another shift of the same worker is refused.
TC-01A	deleting a shift with a request or attendance is refused (FR-05)	services	FR-05	Deleting a shift with a request or an attendance record is refused (SHIFT_IN_USE).
TC-01B	deleting a fixed-schedule row keeps the shifts it generated (FR-02)	services	FR-02	Removing a fixed-schedule row keeps its shifts with fixedScheduleId = NULL.
TC-024	creating a request notifies each candidate (FR-07)	services	FR-07, BR-01	Kang Doyun’s Saturday request notifies Lee Seoyeon and Jung Hana (two REQUEST_RECEIVED).

ID	Name	File	Covers	Expected result
TC-025	no eligible worker still creates the request and notifies the owner (FR-08)	services	FR-08	Jung Hana's Sunday request has no candidate, stays REQUESTED, owner gets NO_CANDIDATE.
TC-026	invalid requests are refused (BR-03)	services	BR-03, FR-06	Open request, closed shift, foreign shift and late deadline are refused with their codes.
TC-051	two acceptances of the same request: only the first succeeds	services	BR-02, NFR-04	Minho's acceptance succeeds; Doyun's later acceptance fails with ALREADY_TAKEN naming Choi Minho; targets ACCEPTED / CLOSED.
TC-052	acceptance notifies requester and owner, closed target is told (FR-10)	services	FR-10	REQUEST_ACCEPTED to owner and requester; TARGET_CLOSED to Doyun.
TC-053	a decline keeps the request open for other candidates	services	FR-09	A decline leaves the request REQUESTED; another candidate can still accept.
TC-061	approval reassigns the shift and keeps the original worker; weekly summaries recomputed	services	BR-04, FR-12	Shift reassigned to Minho, original worker Seoyeon, status SCHEDULED; summaries 21 h / 5 h; two REQUEST_APPROVED.
TC-062	rejection leaves the shift unchanged; only ACCEPTED can be decided	services	BR-04, FR-12	Approval of a REQUESTED request refused; rejection leaves the shift unchanged; two REQUEST_REJECTED.
TC-063	done-criterion 4 walkthrough: accept, already taken, approval panel, approve, handover on board	services	FR-09, FR-11, FR-12, NFR-01	Done-criterion 4: accept, "already taken", preview 16 → 21 h with eligibility unchanged, approve, handover on the board.
TC-088	worked example: Minho 3.2 h holiday allowance, 33,024 KRW in September payroll	services	BR-06, BR-07, BR-11	Week of 09-21: Minho 3.2 h; Seoyeon not eligible; September holiday pay 33,024 won.
TC-089	marking a shift absent removes holiday eligibility for that week	services	BR-06, FR-13	Marking Minho's Tuesday 09-22 absent removes eligibility (reason ABSENT on 09-22).
TC-08A	both substitution-attendance policies through approval	services	BR-06, FR-17	After an approved swap Minho stays eligible under EXCUSED and loses eligibility under ABSENT.
TC-08B	demo walkthrough S7: ABSENT policy, Minho gives away Sat 10-03 and loses holiday allowance	services	BR-06, FR-11	Walkthrough S7: candidates Seoyeon and Hana, Doyun excluded; preview shows eligibility change; after approval not eligible.
TC-095	payroll premium is zero at 4 employees and non-zero at 5	services	BR-08	September premium 0 at 4 employees; Seoyeon 21,000 won at 5 (4 night hours × 10,500 × 0.5).
TC-104	payroll warns for Kang Doyun and blocks confirmation until acknowledged	services	BR-09, BR-10, FR-16	Warning for Doyun, none for Hana (probation); confirmation blocked until acknowledged.
TC-105	minimum-wage acknowledgement is persisted and a regenerated draft needs a new one	services	BR-09, NFR-05	Acknowledgement survives a reload from storage; a regenerated draft resets it and blocks confirmation again.
TC-114	September payroll for Choi Minho: worked + estimated hours, holiday week of 09-21	services	BR-11	Minho September: 21 h (16 worked + 5 estimated), 216,720 + 33,024 = 249,744 won, estimated.
TC-115	confirmed payroll is read-only; regenerating replaces drafts only	services	FR-15, BR-11	After confirmation, marking a shift absent and regenerating leaves the confirmed totals unchanged.

ID	Name	File	Covers	Expected result
TC-121	request expires at its deadline; requester and owner notified	services	BR-12, FR-18	No expiry at 20:59; expiry at 21:00; targets CLOSED; REQUEST_EXPIRED to owner and requester.
TC-122	an ACCEPTED request expires when the shift starts without approval	services	BR-12	An ACCEPTED request expires at the shift start and can no longer be approved.
TC-123	cancel closes and notifies all targets (UC-12)	services	FR-19	Cancel sets CANCELLED, notifies both targets, and a second cancel is refused.
TC-071	worker records attendance for a past shift; owner confirms it	services	FR-13	Recording before the shift starts or on a foreign shift is refused; record + confirm gives WORKED and 4.92 actual hours.
TC-131	a worker never receives other workers' phone or wage (NFR-11)	services	NFR-11	A worker sees null phone and wage for everyone else, own wage visible; the owner sees all phones.
TC-132	inbox lists newest first with unread count; mark read clears it (FR-21)	services	FR-21	Newest notification first, unread count 1, zero after marking read.
TC-133	notification recipients follow the spec §8 table	services	FR-19, FR-21, spec §8	Every notification kind reaches exactly the recipients of the spec §8 table; a decline notifies no one.
TC-002	a saved database from before spec v2 (no Payroll.minWageAck) is replaced by the seed	services	NFR-05	A saved database without Payroll.minWageAck is replaced by the seed.

6.4 Test Results

The table below gives the result of running `npm test` on the environment of Section 6.2 on 2026-09-27, against the final build (prompt v5, Chapter 5).

Table 56. Test results by group (npm test, 2026-09-27)

Group	Subject	Tests	Passed	Failed
TC-00x	Seed data and storage (spec §10, NFR-05)	2	2	0
TC-01x	Overlap, eligibility, schedule editing (BR-01, FR-02, FR-04, FR-05)	11	11	0
TC-02x	Request validity and creation (BR-03, FR-06–FR-08)	6	6	0
TC-05x	First acceptance wins (BR-02, NFR-04)	3	3	0
TC-06x	Approval effect (BR-04)	3	3	0
TC-07x	Attendance (UC-07)	1	1	0
TC-08x	Holiday allowance (BR-06, BR-07)	11	11	0
TC-09x	Premium pay (BR-08)	5	5	0
TC-10x	Minimum wage and probation (BR-09, BR-10)	5	5	0
TC-11x	Monthly payroll (BR-11)	5	5	0
TC-12x	Expiry and cancellation (BR-12, FR-19)	3	3	0
TC-13x	Privacy and notifications (NFR-11, FR-21)	3	3	0
Total (rules: 25, services: 33)		58	58	0

All 58 tests passed. The summary lines of the runner were:

```

i tests 58
i suites 0
i pass 58
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 270.204667

```

The suite contains no skipped, pending or "todo" tests. The whole run takes well under a second, which made it practical to run the suite after every change, as the regression strategy of Section 6.1 requires. The prompt log records the same count after iteration 2: 52 tests in version 1 of the build, plus six tests added in version 2 (TC-002, TC-01A, TC-01B, TC-105, TC-08B and TC-133), each introduced together with the specification change it verifies (Section 6.9).

6.5 Boundary-Value Analysis

The legal rules of ShiftSwap are threshold rules: a worker is either above or below 15 hours, a workplace either has five employees or not, a date is either inside or outside the probation period. Errors in such rules typically occur exactly at the threshold — a > written for $\geq$, or a date compared inclusively instead of exclusively. The tests therefore place inputs on both sides of every threshold, as the table below shows.

Table 57. Boundary values tested for the legal and workflow rules

Rule	Threshold	Values below / at / above	Tests
BR-06 contractual hours	≥ 15 h	14.99 h $\rightarrow$ not eligible; 15.00 h $\rightarrow$ eligible (3 holiday hours); 16 h $\rightarrow$ eligible (3.2 h)	TC-082, TC-081, TC-083
BR-07 holiday-hours cap	$\min(h, 40)$	16 h $\rightarrow$ 3.2; 40 h $\rightarrow$ 8; 52 h $\rightarrow$ 8 (capped)	TC-083
BR-08 employees	≥ 5 regular employees	4 $\rightarrow$ premium 0; 5 $\rightarrow$ premium paid	TC-091, TC-092, TC-095
BR-08 daily hours	> 8 h per day	8 h days $\rightarrow$ no daily overtime; 10 h and 13 h days $\rightarrow$ 2 h and 5 h	TC-092, TC-094
BR-08 weekly hours	> 40 regular h per week	48 regular hours in a week $\rightarrow$ 8 weekly overtime hours, excluding the 2 daily ones	TC-094
BR-08 night window	22:00–06:00	18:00–23:00 $\rightarrow$ 1 night hour; 21:00–07:00 $\rightarrow$ 8 night hours	TC-092, TC-093, TC-112
BR-10 contract length	≥ 365 days	2026-09-01 to 2027-08-30 (363 days) $\rightarrow$ no reduction; to 2027-09-01 (365 days) $\rightarrow$ reduction; open-ended $\rightarrow$ reduction	TC-102
BR-10 three months	date < contractStart + 3 months	2026-11-30 $\rightarrow$ applies (with probation end 11-30); 2026-12-01 $\rightarrow$ does not, even with a later probation end	TC-102
BR-10 probation end	date $\leq$ probationEnd	probation end 10-15, date 10-16 $\rightarrow$ does not apply; no probation end $\rightarrow$ does not apply	TC-102
BR-09 minimum wage	wage < floor	10,320 at minimum 10,320 $\rightarrow$ no warning; 10,000 $\rightarrow$ warning; 10,000 under probation (floor 9,288) $\rightarrow$ no warning; 9,000 $\rightarrow$ warning	TC-103
BR-01 overlap	half-open [s, e)	10:00–16:00 vs 16:00–22:00 $\rightarrow$ no overlap; 10:00–16:01 $\rightarrow$ overlap	TC-011
BR-03 deadline	now < deadline $\leq$ shift start	deadline = now $\rightarrow$ refused; = shift start $\rightarrow$ accepted; shift start + 1 min $\rightarrow$ refused	TC-021, TC-023
BR-12 expiry	deadline $\leq$ now	now 20:59 with deadline 21:00 $\rightarrow$ open; now 21:00 $\rightarrow$ expired	TC-121

The probation dates deserve a comment. For Jung Hana, whose contract starts on 2026-09-01, the three-month limit is 2026-12-01, computed by adding calendar months rather than 90 days. The test checks 2026-11-30, the last day on which the reduction may apply, and 2026-12-01, the first day on which it may not; it also checks 2026-12-01 with a probation end

deliberately set later (2027-03-01), to prove that the three-month limit is applied independently of the probation end written in the contract. The contract-length tests use two end dates two days apart around the 365-day threshold.

6.6 Concurrency Test

NFR-04 requires that two acceptances of the same request can never both succeed. TC-051 tests this directly with the seed request: Choi Minho accepts, and then Kang Doyun accepts the same request. The expected result has four parts: the first call returns ACCEPTED; the second throws a `ServiceError` with the code `ALREADY_TAKEN` and the parameter `name = 'Choi Minho'`, which is what the inbox displays; the request's acceptor is Minho; and the targets are ACCEPTED for Minho and CLOSED for Doyun. TC-052 checks the notifications of the same situation, and TC-063 checks the state the second candidate sees in the inbox (TAKEN, with the acceptor's name).

The test calls the two acceptances one after the other. This models the situation of SD-2 faithfully for the deployed system, in which one JavaScript thread executes one transaction at a time, so the second acceptance always begins after the first has committed. What the test verifies is the design property that makes the outcome independent of timing: the service re-reads the request inside the transaction, and the update is conditional on `status = 'REQUESTED'`, with the number of changed rows checked. A test with truly parallel transactions would require a shared database server, which the client-only architecture does not have; Sections 8.3 and 8.4 discuss this limitation.

6.7 Verification of Non-Functional Requirements

The non-functional requirements were verified with the methods named in their specification. The table below gives the evidence and the status for each. Where a check has not been carried out, the table says so.

Table 58. Verification of the non-functional requirements

ID	Method	Evidence	Status
NFR-01	UI walkthrough	The inbox item of a pending request has Accept and Decline buttons on the item itself: one click opens the inbox, the second accepts. Recorded in the headless walkthrough of the prompt log; the state logic is covered by TC-063.	Met
NFR-02	Browser check at 360 px and 1440 px	The prompt log records a headless Chrome check of every screen for both roles in English and Korean at 1440 px and 360 px with no horizontal scroll (70 of 70 checks passed, 0 console errors) for the version 2 build. The screens are re-checked after the UI review of iteration 3 (Chapter 7).	Met (v2); re-check for v3
NFR-03	Timed load with 20 workers and 12 weeks	Service-level measurement in Node on the machine of Section 6.2, with 16 extra workers (20 in total) and 12 generated weeks (492 shifts): <code>getWeekShifts</code> 1.2 ms, <code>recomputeWeek</code> for all workers 16.9 ms, <code>generatePayroll</code> 22.1 ms, <code>createSubRequest</code> 2.5 ms, <code>getApprovalPreview</code> 1.7 ms. The database work of any screen is therefore far below 1 second; a browser rendering measurement with the same data has not been recorded.	Data layer met; screen timing not measured
NFR-04	Unit test TC-05x	TC-051, TC-052, TC-053, TC-063 pass (Section 6.6).	Met
NFR-05	Manual test	TC-105 reloads the database from a storage object and finds the acknowledgement kept; TC-002 checks the replacement of an outdated copy; the prompt log records a headless reload and reset check.	Met
NFR-06	Manual test in four browsers	Chrome checked (headless and desktop). The code uses only standard ES modules, <code>localStorage</code> , <code>sessionStorage</code> and <code>WebAssembly</code> . Checks in Edge, Safari and Firefox are not recorded at the time of writing.	Chrome met; others pending
NFR-07	Deployed URL	<code>https://keemlogan.github.io/shiftswap/app/</code> returned HTTP 200 with the application's <code>index.html</code> when checked on 2026-09-27.	Met
NFR-08	Repository	Static HTML, CSS and ES modules; no build step; <code>sql.js</code> vendored in <code>app/vendor/</code> ; hosted on GitHub Pages.	Met
NFR-09	Review	UML models drawn with PlantUML [14] in UML 2.5 notation [8]; all code identifiers, table and column names in English.	Met

ID	Method	Evidence	Status
NFR-10	Unit tests TC-08x, TC-09x	All TC-08x, TC-09x and TC-10x tests pass, including the boundary values of Section 6.5.	Met
NFR-11	Role test	TC-131: a worker receives null for other people's phone and wage; the worker menus contain no screen with such data.	Met
NFR-12	Design review	A search of the application code finds no use of the geolocation interface; attendance is entered by the worker and confirmed by the owner (TC-071).	Met
NFR-13	Headless check that counts the filled primary buttons on every screen of both roles (exactly one expected), checks that every disabled button has a visible reason and that touch targets are at least 44 × 44 px (done criterion 5 of the final prompt); walkthrough of the sign-in tour (done criterion 4)		

6.8 Traceability Matrix

The specification requires that every functional requirement maps to at least one use case, every use case to at least one screen and service function, and every business rule to at least one test case (spec §12). The matrix below was derived from the code, not from the design documents: the screen column lists the UI modules that call the service functions, found by searching each module in `app/ui/` for its calls to `services.js`, and the test column lists the tests that call those functions or the rule functions behind them and assert the requirement's behaviour.

Table 59. Traceability matrix: functional requirement → use case → screen → service → business rule → tests

FR	UC	Screen (app/ui)	Service function(s)	BR	Tests
FR-01	UC-01	<code>workers.js</code>	<code>registerWorker</code>	BR-09, BR-10 (data)	— (walkthrough only)
FR-02	UC-01	<code>workers.js</code>	<code>saveFixedSchedule</code>	BR-06	TC-017, TC-01B, TC-083
FR-03	UC-02	<code>availability.js</code>	<code>saveAvailability</code>	BR-01	TC-08A, TC-122 (setup), TC-014, TC-015
FR-04	UC-03	<code>schedule.js</code>	<code>generateWeek</code>	BR-05	TC-018, TC-013
FR-05	UC-03	<code>schedule.js</code>	<code>editShift</code>	BR-01 (overlap)	TC-019, TC-01A
FR-06	UC-04	<code>requests.js</code>	<code>createSubRequest</code> , <code>listRequestableShifts</code> , <code>defaultDeadline</code>	BR-03	TC-021, TC-022, TC-023, TC-026
FR-07	UC-04	<code>requests.js</code>	<code>createSubRequest</code>	BR-01	TC-014, TC-015, TC-016, TC-024, TC-133
FR-08	UC-04	<code>requests.js</code>	<code>createSubRequest</code>	BR-01	TC-025, TC-133
FR-09	UC-05	<code>inbox.js</code>	<code>respondToRequest</code> , <code>listNotifications</code>	BR-02	TC-051, TC-053, TC-063
FR-10	UC-05	<code>inbox.js</code>	<code>respondToRequest</code>	BR-02	TC-052, TC-133
FR-11	UC-06	<code>approvals.js</code>	<code>getApprovalPreview</code> , <code>listOwnerRequests</code>	BR-06, BR-07	TC-063, TC-08B
FR-12	UC-06	<code>approvals.js</code>	<code>decideRequest</code> , <code>recomputeWeek</code>	BR-04	TC-061, TC-062, TC-133

FR	UC	Screen (app/ui)	Service function(s)	BR	Tests
FR-13	UC-07	attendance.js	recordAttendance, confirmAttendance, markAbsent	BR-06	TC-071, TC-089
FR-14	UC-08	weekly.js	recomputeWeek	BR-05, BR-06, BR-07	TC-081-TC-089, TC-08A
FR-15	UC-09	payroll.js	generatePayroll, confirmPayroll, getPayroll	BR-07, BR-08, BR-11	TC-111-TC-115, TC-095
FR-16	UC-09	payroll.js	generatePayroll, acknowledgeMinWage, confirmPayroll	BR-09, BR-10	TC-101-TC-105
FR-17	UC-10	settings.js	updateSettings, getSettings	BR-06, BR-08, BR-09	TC-08A, TC-08B, TC-095
FR-18	UC-11	(router main.js)	expireOverdue	BR-12	TC-121, TC-122, TC-133
FR-19	UC-12	requests.js	cancelRequest	—	TC-123, TC-133
FR-20	UC-13	login.js, main.js	listWorkers, getWorker	NFR-11	TC-131 (privacy); menus by walkthrough
FR-21	UC-04-06, 11, 12	inbox.js, main.js	listNotifications, markNotificationsRead, unreadCount	—	TC-132, TC-133

Two gaps are visible in the matrix and are stated here rather than hidden. FR-01 (registering a worker) has no automated test of its own: `registerWorker` is exercised only through the screens, and its validation messages were checked in the walkthrough. FR-20 (role menus) is enforced by the router in `main.js`, which is not loaded by the Node tests; the privacy part of the requirement is covered by TC-131, and the menus by the walkthrough. Both gaps are candidates for additional tests (Section 8.4).

The table below summarises coverage from the other direction. Every business rule has at least one test, as spec §12 requires; most have several, at the rule level and through the services.

Table 60. Business-rule coverage by test cases

BR	Rule	Implementing functions	Tests	Count
BR-01	Eligible candidates	isEligibleCandidate, overlaps	TC-011, TC-012, TC-014, TC-015, TC-016, TC-024, TC-08B	7
BR-02	First acceptance wins	respondToRequest	TC-051, TC-052, TC-053, TC-063	4
BR-03	Request validity	validateRequest	TC-021, TC-022, TC-023, TC-026	4
BR-04	Approval effect	decideRequest	TC-061, TC-062	2
BR-05	Week	weekStartOf	TC-013, TC-018	2
BR-06	Holiday-allowance eligibility	computeWeeklySummary, contractHours	TC-081, TC-082, TC-084-TC-089, TC-08A, TC-08B	10
BR-07	Holiday-allowance hours and pay	holidayHours, computePayrollRow	TC-081, TC-083, TC-088, TC-111, TC-113	5
BR-08	Premium pay	premiumHours	TC-091-TC-095, TC-112	6
BR-09	Minimum-wage check	minimumWageFor, computePayrollRow, acknowledgeMinWage, confirmPayroll	TC-101, TC-103, TC-104, TC-105	4
BR-10	Probation reduction	probationApplies	TC-102, TC-103, TC-104	3

BR	Rule	Implementing functions	Tests	Count
BR-11	Monthly payroll	computePayrollRow, generatePayroll	TC-111-TC-115, TC-088	6
BR-12	Expiry	expireOverdue	TC-121, TC-122, TC-133	3

6.9 Defects and Specification Gaps Resolved During the Iterations

The build iterations are described in Chapter 5; this section lists what they found and how each finding was verified. In iteration 1, the AI coding assistant built the system from prompt version 1 and recorded every point at which the specification was ambiguous or silent, together with the choice it made. A document review of the proposal and interim report against the specification found further gaps. In iteration 2 each finding was resolved by changing the specification or the prompt, and where necessary the code, and each resolution was tied to a test. The table below lists the findings. Most of them were gaps in the specification rather than errors in the code: the implementation had already made a reasonable choice, and the change made that choice part of the contract, so that a regenerated system must make the same choice.

Table 61. Findings of iterations 1 and 2, their resolution and verification

#	Finding	Resolution (specification / prompt / code)	Verified by
1	node --test tests/ fails on Node 24 (directory argument read as a module path)	Prompt: test script node --test tests/*.test.js	npm test runs
2	Read-only queries for the screens and an error type were not in the service list	Prompt: read-only query functions and ServiceError(code, message, params)	All service tests
3	BR-06 required every shift to be WORKED, so a week in progress could never be eligible, contradicting the worked example	BR-06: a SCHEDULED shift does not break attendance; the weekly result is a projection	TC-085, TC-063
4	Unclear which workers have a weekly summary, and whether the seed contains derived rows	Spec §8: a summary exists only for workers with a shift in the week; the seed has no WeeklySummary or Payroll rows	TC-001, TC-088
5	Meaning of "estimated" and of shifts without attendance in the payroll	BR-11: WORKED → recorded hours; SCHEDULED → scheduled hours and estimated = 1; ABSENT → 0	TC-114, TC-112
6	Premium pay at month level mixed daily, weekly and night rules without saying how a week split by the month end is treated	BR-08: weekly overtime uses only the days of the week inside the month; premium pay rounded down	TC-092-TC-095
7	Date on which BR-10 is evaluated for a monthly payroll	BR-10: first day of the month, or contract start if later	TC-102, TC-104
8	"Effective hourly wage" in BR-07 undefined; rounding of holiday pay unclear	BR-07: Worker.hourlyWage; rounded down per week, then summed	TC-111, TC-113
9	Format of attendance times	Spec §8: HH:MM on the shift date; end ≤ start means next day	TC-071
10	Recipients of cancel and expiry notifications not specified (also found in the review of UC-12)	Spec §8 recipients table; FR-19; a decline notifies no one	TC-133 (new), TC-052, TC-121, TC-123
11	Seed request creation time, phone numbers and stored name language not given	Spec §10: createdAt 2026-09-28T08:30, fictional phone numbers, English names	TC-001, TC-131
12	Deleting a shift that already has a request or attendance	FR-05: refused with an explanatory message	TC-01A (new)

#	Finding	Resolution (specification / prompt / code)	Verified by
13	Deleting a fixed-schedule row that already generated shifts	FR-02: shifts kept with <code>fixedScheduleId = NULL</code>	TC-01B (new)
14	Storage of the demo clock and language; effect of reset on the clock	Prompt: <code>keys shiftswap.clock.v1</code> , <code>shiftswap.lang</code> ; reset also resets the clock	Headless reset and reload check
15	No module for shared UI helpers	Prompt: escaping, formatting and the form error box in <code>i18n.js</code>	Review
16	Stripe pattern conflicted with "no gradients"; tables did not fit 360 px	Prompt: the stripe may use <code>repeating-linear-gradient</code> ; tables become stacked cards below 720 px	Headless check at 360 px
17	The minimum-wage acknowledgement was not stored, so it was lost on reload (document review)	Schema: <code>Payroll.minWageAck</code> ; BR-09; new service <code>acknowledgeMinWage</code> ; Confirm disabled until every warning is acknowledged	TC-104, TC-105 (new), TC-115
18	Saved databases from before the schema change lacked the new column	<code>openDatabase</code> checks the schema and replaces an outdated copy with the seed	TC-002 (new)
19	No scenario in which a swap changes eligibility, so FR-11's highlight was never exercised	Spec §10: demonstration walkthrough S7 with the ABSENT policy	TC-08B (new)
20	Date of Week 14 in NFR-07 inconsistent with the course calendar	NFR-07 corrected to 2026-12-04; spec §14 calendar	—

Two findings changed the code; the others changed only the specification and the prompt. Finding 17 is the most instructive. The original requirement said that a minimum-wage warning "blocks confirmation until acknowledged", but the data model had no place to store an acknowledgement, and the version 1 build handled it as a parameter of the confirm call. The review of the interim report noticed that an owner who acknowledged a warning and then reloaded the page would find the acknowledgement gone. The fix touched every level at once — a new column in the schema, a new clause in BR-09, a new service function, a new button, and two tests — which shows why the change procedure of Section 3.11.3 starts with the specification.

Findings from the UI review of the running application in iteration 3 (prompt version 3) concern layout and presentation rather than rules; they are listed with the prompt changes in Chapter 5, and the screens that result from them are shown in Chapter 7.

7. Deployment and User Guide

This chapter explains how ShiftSwap is deployed and how it is used. Section 7.1 describes the deployment on GitHub Pages, what happens when the application is opened for the first time, how to reset or run it locally, and which browsers have been checked. Section 7.2 is a user guide organised by role and task, which follows the demonstration data of spec §10 through one complete substitution and then through attendance, weekly summary and payroll.

7.1 Deployment

7.1.1 Hosting on GitHub Pages

ShiftSwap is a static web application (NFR-08), so deploying it means publishing files; there is no server program to install or start. The repository is hosted on GitHub and published with GitHub Pages [13] from the root of the main branch, with HTTPS enforced. An empty `.nojekyll` file in the root tells GitHub Pages to serve the files as they are, without processing them. The table below gives the addresses.

Table 62. Deployment addresses

Item	Address
Application	https://keemlogan.github.io/shiftswap/app/
Project page (deliverables hub)	https://keemlogan.github.io/shiftswap/
Source repository	https://github.com/keemlogan/shiftswap
Pages source	Branch <code>main</code> , folder <code>/</code> (repository root); HTTPS enforced

Publishing a new version consists of committing the changed files and pushing them to `main`; GitHub Pages rebuilds the site within a few minutes. Because all URLs inside the application are relative, the files work unchanged under the sub-path `/shiftswap/app/`. The deployment was checked on 2026-09-27: the application URL returned HTTP 200 with the application's `index.html`, and the Pages configuration reported the status "built".

7.1.2 What happens when the application is opened

1. The browser loads `app/index.html`, the style sheet, `vendor/sql-wasm.js` and the module `main.js`. While the database is being prepared, the page shows a loading message.
2. `main.js` initialises `sql.js`, which loads the WebAssembly file `vendor/sql-wasm.wasm` (about 650 KB) from the same site. No database server is contacted.
3. If the browser's `localStorage` holds a saved database under `shiftswap.db.v1` with the current schema, it is opened. Otherwise the schema of spec §8 is created, the demonstration data of spec §10 is inserted, and the database is saved. The demo clock is read from `shiftswap.clock.v1` (default Monday 2026-09-28 09:00) and the language from `shiftswap.lang`; without a saved choice the interface is in Korean.
4. The router calls `expireOverdue()` and renders the sign-in screen, which doubles as a guided demo tour (Section 4.9). From now on, every change is saved to `localStorage` after its transaction commits.

The fonts (Pretendard and JetBrains Mono) are loaded from public font services. If they cannot be reached, the browser uses its own fonts; the application works the same, only its appearance differs. The first load therefore needs a network connection for the application files, and later work happens entirely in the browser.

7.1.3 Resetting and running locally

Reset. The *Demo tools* panel, opened from the demo-clock chip in the header, contains *Reset demo data*. After a confirmation in the panel, it deletes the saved database, recreates the demonstration data, sets the demo clock back to 2026-09-28 09:00 and returns to the sign-in screen. This restores the exact starting state of the scenarios in Section 3.8 and of the user guide. Clearing the site data in the browser has the same effect.

Local run. To run the application from a copy of the repository, serve the folder with any static web server and open the `app/` path, for example:

```
git clone https://github.com/keemlogan/shiftswap.git
cd shiftswap
python3 -m http.server 8000          # then open http://localhost:8000/app/
npm install && npm test              # optional: run the automated tests (Chapter 6)
```

Opening `app/index.html` directly from the file system does not work, because browsers do not load WebAssembly and ES modules from `file://` addresses; the application then shows an error message that asks the user to serve the folder over HTTP and reload the page.

7.1.4 Browser support

The application uses only standard features: ES modules, `localStorage`, `sessionStorage` and WebAssembly. It was tested in Google Chrome on desktop (1440 px) and in Chrome's mobile emulation. NFR-06 also names Edge, Safari and Firefox; checks in these browsers had not been recorded when this chapter was written (Section 6.7). If a browser blocks storage, for example in some private-browsing modes, the application still runs, but data is kept only until the tab is closed.

7.2 User Guide

8. Conclusion

This report has described ShiftSwap, a web application that replaces the telephone-and-spreadsheet handling of substitute shifts in a small part-time workplace. This chapter assesses the result against the objectives of Chapter 1, summarises the size of the system, states its limitations, proposes future work, and records what the team learned about requirements engineering and vibe coding.

8.1 Achievement of the Objectives

The table below assesses each objective G1–G6 against its measure. Only what can be verified from the tests of Chapter 6, the code or the deployed site is claimed. The objectives concern the behaviour of the system; whether that behaviour reduces the owner's work in a real store has not been measured, because no field trial took place (Section 8.3).

Table 63. Achievement of objectives G1–G6

ID	Objective	Evidence	Assessment
G1	Remove the owner's one-to-one contacts	<code>createSubRequest</code> notifies every eligible worker in the transaction that creates the request (TC-024, TC-016); with no candidate, the owner is told at once (TC-025). The owner's only action in the workflow is one approve or reject decision (TC-061, TC-062).	Achieved in the system; effect in a store not measured
G2	Let a candidate answer quickly	A pending request appears as a card on the candidate's Home screen with <i>Accept</i> on the card, so accepting takes one tap after opening the application (NFR-01, NFR-13). Only the first acceptance succeeds (TC-051).	Achieved
G3	Keep schedule and weekly hours of both workers correct after a swap	Approval reassigns the shift, keeps the original worker and recomputes both weekly summaries in one transaction (TC-061). The approval panel shows before and after values for both workers — Choi Minh 16 → 21 scheduled hours, eligibility unchanged (TC-063) — and highlights a change of eligibility (TC-08B).	Achieved
G4	Apply the legal pay rules and flag violations before payday	BR-06 to BR-11 are pure functions tested at their boundary values (Section 6.5), and every business rule BR-01 to BR-12 has at least one test (Section 6.8). The payroll warns for Kang Doyun's wage and blocks confirmation until the owner acknowledges it (TC-104, TC-105).	Achieved
G5	Deliver a system of the size the course requires	13 use cases (9 core) against a minimum of 5; 12 database tables against a minimum of 4.	Achieved
G6	Deliver on time and reproducibly	The application is deployed at https://keemlogan.github.io/shiftswap/app/ (NFR-07). The final prompt and the specification are in the repository (Chapter 5); the reproduction run of the final prompt in an empty folder and its test result are reported in Chapter 5.	Deployment achieved; reproduction see Chapter 5

8.2 Size and Complexity of the System

The table below summarises the size of the specification, the models and the implementation. The counts of identifiers come from `spec/spec.md`; the line counts come from `wc -l` and exclude the vendored `sql.js` files.

Table 64. Size of the ShiftSwap specification, models and implementation

Item	Count	Remark
Actors	3	Worker, Owner, System Clock
Use cases	13	9 core (UC-01–UC-09), 4 supporting; 9 Must, 4 Should
Functional requirements	21	FR-01–FR-21
Non-functional requirements	13	7 product, 3 organisational, 3 external
Business rules	12	5 of them derived from labour law (BR-06–BR-10)

Item	Count	Remark
Database tables	12	SQLite schema of spec §8
State machines / states	5 / 17	SubRequest (6), SubRequestTarget (4), Shift (3), Attendance (2), Payroll (2)
Notification kinds	8	With a fixed recipients table (spec §8)
UML diagrams of the system	16	2 use-case, 1 class, 1 ER, 1 component, 1 deployment, 5 sequence (SD-1, SD-2, SD-3a, SD-3b, SD-4), 3 state, 2 activity
Screens	11	Sign-in and ten working screens (Section 4.9)
Automated tests	58	25 rule tests, 33 service tests; all pass
Lines of application code	3,652	Core modules 1,271 (services 754, rules 257, db 219, clock 41); presentation 2,381 (router, eleven screen modules, translations and style sheet)
Lines of test code	705	tests/rules.test.js 214, tests/services.test.js 491

Line counts were taken with `wc -l` after the UI work of iteration 3 was finished.

By the measures the course names — use cases and tables — the system is well above the minimum. Its complexity lies less in the number of screens than in the interaction of rules: a single approval changes a shift, two weekly summaries, possibly two holiday allowances and later a payroll, and each of these depends on settings that the owner can change. The 17 states and the 12 business rules are where most of the analysis and test effort went.

8.3 Limitations

The system has the following limitations. Most follow from the decision to build a client-only demonstration system (design decision D1); they are listed so that the reader can judge what the results above do and do not show.

- **One database per browser.** The database lives in the browser in which the application is opened. An owner on one phone and a worker on another do not share data; the demonstration works because all accounts are used in the same browser. This is the most important limitation: in a real store, the substitute workflow requires a shared database.
- **No authentication.** Sign-in is the choice of a demo account without a password (FR-20). Anyone who opens the application can act as the owner. The role restriction and the privacy rule NFR-11 therefore protect against accidental, not deliberate, disclosure.
- **Browser storage.** Data is kept in `localStorage`, which the user or the browser can clear and which is limited to a few megabytes per site. A store would need backups and export, which the system does not provide.
- **Serialised concurrency.** The first-acceptance rule is enforced correctly within one database, but true parallel acceptances from different devices cannot occur in the current deployment and could not be tested (Section 6.6).
- **Demo clock.** Expiry is triggered by navigation and by changes of the demo clock, not by the passage of real time. A request can expire only when someone uses the application.
- **Legal scope.** The pay rules cover the minimum wage, the weekly holiday allowance, overtime and night premiums and the probation reduction. Holiday-work premium, annual leave, severance pay, tax and social insurance are outside the scope, and the four-week average of the 15-hour rule is simplified to the weekly contractual value (BR-06). The treatment of a handed-over shift is a setting because the law does not decide it. The system applies these rules consistently but is not legal advice.
- **Evaluation.** The requirements came from one reference store, and the interviews and prototype sessions were used to confirm requirements, not to measure them. No store has used the system in operation, so the effect on the number of contacts per request (problem P1) has not been measured.
- **Test gaps.** Registering a worker (FR-01) and the role menus (FR-20) have no automated test (Section 6.8), and the browser checks of NFR-06 are recorded only for Chrome.

8.4 Future Work

The layered design makes the most important extension, a shared database, a change of one layer. The service functions already express each use-case step as one transaction, and the rules do not touch storage, so the data-access module could be replaced by calls to a small server with a shared SQL database while the rules and most of the services remain unchanged. With a server, the following become possible and would be the next increments:

1. a shared database and real accounts with passwords, so that owner and workers use their own devices, and NFR-11 is enforced on the server;
2. push or messenger notifications for new requests, in addition to the in-app inbox;
3. expiry driven by a scheduled job on the server instead of navigation;
4. a test of the first-acceptance rule with genuinely parallel transactions against the shared database;
5. a field trial in the reference store, measuring the number of contacts per request and the time to fill a shift before and after, which is the measurement G1 ultimately stands for;
6. automated tests for FR-01 and FR-20, and recorded checks in Edge, Safari and Firefox;
7. export of confirmed payroll as a payslip, and several stores per owner.

8.5 Lessons Learned

The specification is the program's real source. In vibe coding, the AI coding assistant writes the code, and the quality of the code is bounded by the precision of what it is told. The team's most useful decision was to keep one specification file with every identifier, rule, state and seed value, and to change it before changing anything else. When a question arose — who is notified when a request is cancelled, whether a week in progress can be "perfect" — the answer was written into the specification, and the next generated version had to follow it. This is requirements management as the lecture describes it [7]: individual requirements are tracked, their dependencies are recorded, and changes go through a defined procedure. Without it, three documents, a set of UML diagrams and the code would have drifted apart within a week.

Generation exposes incompleteness. The lecture states that in practice a complete and consistent requirements document is impossible [7]. The build iterations confirmed this concretely: in iteration 1 the AI coding assistant met more than twenty points where the specification was silent or ambiguous and had to choose. Recording each choice and turning it into specification text (Section 6.9) worked as a requirements review performed by the implementer. The lesson for vibe coding is that the log of the generator's decisions is as valuable as the code, because it shows where the requirements were weaker than the team believed.

Testable rules make generated code checkable. Code produced by an AI coding assistant cannot be trusted because it looks right. Placing every business rule in a pure function and deriving tests from the rules at their boundary values (14.99 and 15 hours, four and five employees, the last day of probation) gave the team an objective check that did not depend on reading every line. When the specification changed, the tests changed first. This is the test-first practice of extreme programming [6] applied to a prompt instead of to hand-written code.

Models find what text hides. The state diagram of the substitute request made it easy to see that every open state needs an exit by time and by the requester, and the traceability matrix showed which requirements had no test. The demonstration walkthrough S7 was added because the team noticed that no scenario ever changed a worker's eligibility, so the highlight required by FR-11 had never been exercised. Chapter 2 of the lecture presents models as a means of discussion, documentation and implementation [6]; in this project they served mostly as a means of checking.

The incremental model suited the course. Placing all Must use cases in the first increment meant that the substitute workflow, the core of the project, was working and testable before the payroll rules were built, as the lecture's description of the incremental model promises [6]. The weakness the lecture warns of, degradation of structure as increments are added, was contained by fixing the layered architecture before the first increment; the second increment added functions to the existing layers and did not change their boundaries.

Documents need the same discipline as code. Because code can be regenerated in hours, the documents became the slower part of the project, and they drifted more easily than the code. Several corrections in this report — the recipients of a cancelled request, the storage of the minimum-wage acknowledgement, the course calendar — were found by checking documents against the specification and the tests. A team using vibe coding should plan as much time for keeping the requirements, models and report consistent as for generating the software.

ShiftSwap shows that a small team can take a real problem from a part-time job through problem definition, requirements, design, implementation by vibe coding and testing within one semester, and arrive at a system whose behaviour is specified, traceable and tested. What remains is to move it from one browser to a shared server and to measure its effect in a real store.

References

1. Ministry of Employment and Labor, "2026 minimum wage KRW 10,320 per hour," MOEL notice, Aug. 2025. [Online]. Available: https://www.moel.go.kr/news/enews/report/enewsView.do?news_seq=18144
2. Korea Policy Briefing, "2026년 시간당 최저임금 10,320원" (2026 hourly minimum wage KRW 10,320). [Online]. Available: <https://www.korea.kr/news/policyNewsView.do?newsId=148956812>
3. Labor Standards Act (근로기준법), Art. 11, 18, 55, 56; Enforcement Decree of the Labor Standards Act, Art. 30. National Law Information Center. [Online]. Available: <https://www.law.go.kr>
4. Minimum Wage Act (최저임금법), Art. 5 ②; Enforcement Decree of the Minimum Wage Act, Art. 3. National Law Information Center. [Online]. Available: <https://www.law.go.kr>
5. Ministry of Government Legislation, Easy-Law (찾기쉬운 생활법령정보), "Part-time workers: rest and holidays." [Online]. Available: <https://easylaw.go.kr/CSP/CnpClsMain.laf?popMenu=ov&csmSeq=896&ccfNo=2&cciNo=3&cnpClsNo=1>
6. D. A. Gustafson, *Schaum's Outline of Software Engineering*. New York: McGraw-Hill, 2002 (course textbook; Ch. 1 software life cycle, Ch. 2 software process and other models, Ch. 8 requirements).
7. I. Sommerville, *Software Engineering*, 10th ed. Boston: Pearson, 2016 (requirements engineering and the classification of non-functional requirements used in lecture Chapter 8).
8. Object Management Group, *OMG Unified Modeling Language (OMG UML)*, Version 2.5.1, Dec. 2017. [Online]. Available: <https://www.omg.org/spec/UML/2.5.1>
9. sql.js contributors, "sql.js: SQLite compiled to JavaScript/WebAssembly," version 1.14.2. [Online]. Available: <https://sql.js.org>
10. Shift-scheduling products examined for related work: Shiftsee; Albamon work management (알바몬 근무관리); Alba Cheonguk schedule (알바천국 스케줄); When I Work; Homebase; Deputy. Public product descriptions on the vendors' websites, consulted in 2026.
11. International Organization for Standardization, *ISO 8601-1:2019 Date and time — Representations for information interchange — Part 1: Basic rules*. Geneva: ISO, 2019.
12. OpenJS Foundation, "Node.js documentation: Test runner (node:test)," Node.js v24. [Online]. Available: <https://nodejs.org/api/test.html>
13. GitHub, "About GitHub Pages," GitHub Docs. [Online]. Available: <https://docs.github.com/en/pages/getting-started-with-github-pages/about-github-pages>
14. PlantUML contributors, "PlantUML: open-source tool that uses simple textual descriptions to draw UML diagrams." [Online]. Available: <https://plantuml.com>

Appendix A. Final Vibe-Coding Prompt (v5)

The complete prompt, verbatim. To reproduce the system it is given to the coding assistant together with the two test files and followed by the specification file spec/spec.md (all in the repository). Source file: prompts/final-prompt.md.

ShiftSwap — Final Vibe-Coding Prompt (v5)

Version history: v1 was built in iteration 1; v2 (iteration 2) resolved the build ambiguities and document-review gaps; v3 (iteration 3) applied the UI review of the running app; v4 (iteration 4) applied the UX redesign review from the point of view of a consumer-app product team (task-first home screens, one primary action per screen, consequences shown before committing); v5 (iteration 5) makes the acceptance test files part of the input after reproduction run 1 showed that two correct rebuilds disagree on function signatures (see docs/reproducibility.md). Earlier versions are kept as prompts/prompt-v1.md ... prompts/prompt-v4.md; the reasons for every change are in prompts/prompt-log.md.

How to use: in an empty folder, place tests/rules.test.js and tests/services.test.js from the repository, then paste this whole prompt, followed by the complete contents of spec/spec.md, into a coding agent that can create files and run shell commands. The agent must produce the same system that is deployed at <https://keemlogan.github.io/shiftswap/app/>.

You are building **ShiftSwap**, a shift schedule and substitute management web application for a small café with part-time workers. The complete specification (actors, use cases UC-01...UC-13, functional requirements FR-01...FR-21, non-functional requirements NFR-01...NFR-13, business rules BR-01...BR-12, states, SQLite schema, seed data, test IDs) follows this prompt. Treat the specification as the contract: use its identifiers, table and column names, state names and seed data exactly. Do not add features that are not in it.

0. Acceptance tests (supplied — read them first)

The folder already contains tests/rules.test.js and tests/services.test.js. They are the executable form of the specification and they fix the API: module paths, exported names, argument order, return shapes, error codes and the Node-side database factory. Read both files completely before writing code. **Do not modify, rename, skip or delete any test.** Write the implementation so that npm test passes all of them unchanged. Where a test and the prose below seem to disagree about an API detail, the test wins; where they disagree about behaviour, stop and report the conflict instead of guessing. You may add further tests in a new file tests/extra.test.js.

1. Technology constraints

- Static site only: index.html, CSS, and JavaScript ES modules. No build step, no framework, no server. It must work when the folder is served by GitHub Pages under the sub-path /shiftswap/app/ (use only relative URLs).
- Database: SQLite in the browser with **sql.js** (WebAssembly). Vendor sql-wasm.js and sql-wasm.wasm into app/vendor/ (copy them from the sql.js npm package) so the app does not depend on a CDN at runtime.
- Persistence: after every committed transaction, export the database and save it to **localStorage** under the key shiftswap.db.v1 (base64). The demo clock is stored under shiftswap.clock.v1 and the language under shiftswap.lang. On start, load them if present, otherwise create the schema and insert the seed data. Provide "Reset demo data" in the header menu; it also resets the clock to 2026-09-28T09:00.
- Folder layout:
 - app/index.html, app/styles.css, app/main.js (router + bootstrap)
 - app/core/db.js — init, schema (exactly the SQL of spec §8), seed (spec §10), tx(fn) helper that runs BEGIN...COMMIT / ROLLBACK and persists after commit, all(sql, params), get(sql, params), run(sql, params).
 - app/core/clock.js — demo clock: now() returns the stored demo datetime (default 2026-09-28T09:00), setNow(iso), advance(minutes).
 - app/core/rules.js — pure functions only (no DB, no DOM): overlaps, durationHours (supports end ≤ start as next day), weekStartOf, isEligibleCandidate, validateRequest, contractHours, computeWeeklySummary, holidayHours, premiumHours, minimumWageFor, probationApplies, computePayrollRow. Each function has a JSDoc comment citing the BR it implements.
 - app/core/services.js — one exported function per use-case step, each running inside tx(): registerWorker, saveFixedSchedule, saveAvailability, generateWeek, editShift, createSubRequest, respondToRequest, decideRequest, cancelRequest, expireOverdue, recordAttendance, confirmAttendance, markAbsent, recomputeWeek, generatePayroll, confirmPayroll, updateSettings, listNotifications, markNotificationsRead, acknowledgeMinWage. Services call rules for every decision and write Notification rows exactly as the recipients table in spec §8 says. services.js also exports read-only query functions for the screens (not wrapped in tx), e.g. getWeekShifts, listMyRequests, listOwnerRequests, getApprovalPreview, getPayroll, unreadCount, defaultDeadline, and a ServiceError(code, message, params) class that the UI translates by code. It also exports the pure function menuFor(role) returning the navigation items a role may open (the router uses it to hide menus and to redirect routes the role may not open, FR-20), and home-screen queries getWorkerHome(workerId) and getOwnerHome() that return, in one call, every item §2.3 / §2.5 needs (including the before/after effect of accepting or approving).
 - app/ui/ — one module per screen: login.js, schedule.js (weekly board, both roles), requests.js (worker: my requests + new request form), inbox.js, approvals.js (owner), attendance.js, weekly.js (owner weekly summary), payroll.js, workers.js (owner: workers, fixed schedules), availability.js (worker), settings.js

(owner: workplace + minimum wage), `i18n.js` (translations plus shared helpers: HTML escaping, number/date/money formatting, the form error box).

- `tests/rules.test.js` and `tests/services.test.js` run with `node --test` (use the `sql.js` npm package in Node for service tests). Name each test with its spec test ID (e.g. `TC-081 holiday allowance at exactly 15 h`).
- `package.json` with `"type": "module"` and script `"test": "node --test tests/*.test.js"` (a bare directory argument fails on Node 24).
- `expireOverdue()` (the System Clock actor, UC-11) runs on app start, on every route change, and after the demo clock changes.

2. Screens and behaviour

2.0 UX principles (apply to every screen; they override any screen description that seems to conflict)

1. **One screen, one job.** Every screen has exactly one primary action, drawn as the only filled blue button on that screen. On screens narrower than 900 px the primary action is a full-width button fixed to the bottom (16 px side inset, above the tab bar, `env( safe-area-inset-bottom)` respected). Other actions are grey filled (secondary) or text buttons.
2. **What needs me comes first.** Each role opens on a **Home** screen that lists the things waiting for this person right now, most urgent first, and each item carries its own action button, so the user never has to find the right menu.
3. **Show the consequence before the commitment.** Whenever an action changes someone's hours or pay (accepting, approving, confirming payroll), the screen states the effect in one plain sentence plus numbers before the button — e.g. "민호님 이번 주 16시간 → 21시간 · 주휴수당 변화 없음 (33,024원 유지)".
4. **Ask one thing at a time.** Requesting a substitute is a three-step flow (choose shift → reason and deadline → review and send) with a step indicator ("1/3"), a back button, and a review screen that names the people who will receive the request.
5. **Say what happened and what happens next.** After every action show a result screen or a toast that states the outcome and the next step — e.g. "민호님, 도윤님에게 요청을 보냈어요. 누군가 수락하면 알려 드릴게요." — never a silent state change.
6. **Status as progress.** A substitute request is shown everywhere as a three-step tracker `요청 → 수락 → 승인` (Requested → Accepted → Approved); `REJECTED`, `EXPIRED` and `CANCELLED` are shown as a clearly labelled end state with the reason.
7. **The user's words.** The default UI language is Korean in polite 해요체 ("대타를 구하고 있어요"), with a complete English translation selectable in the menu. IDs such as `FR/UC/BR`, status codes and table names never appear in the UI. In the Korean UI the seed people and store are shown with their Korean names from spec §10 (박지영, 이서연, 최민호, 정하나, 강도윤, 달빛카페) and addressed as full name + 님 ("최민호님") — a display map in `i18n.js`, the stored names stay English; names not in the map are shown as stored. Dates read like "9월 30일(수) 18:00–23:00" / "Wed, Sep 30 · 18:00–23:00"; relative times where they help ("내일 21:00까지", "D-2").
8. **Nothing blocks without a reason.** A disabled button always has a one-line explanation beneath it (e.g. "최저임금 경고 1건을 확인해야 확정할 수 있어요").

2.1 Navigation

- Worker tabs: **홈** (Home), **근무표** (Schedule), **대타** (Swaps), **내 정보** (Me: availability and attendance). Owner tabs: **홈**, **근무표**, **직원** (Staff: workers and fixed schedules), **급여** (Pay: weekly summary and monthly payroll). **설정** (Settings), language and sign-out are in the header menu.
- ≥ 900 px: left navigation (220 px) with the same items and inline SVG line icons; < 900 px: bottom tab bar (64 px, icon + label), hidden while the request flow is open.
- Header: store name; a bell icon with the unread count opening the notification list (FR-21); the compact demo clock chip ("9/28(월) 09:00") that opens **Demo tools**; the current person's name with **다른 사람으로 보기** (switch account) that opens the account list without resetting data.
- **Demo tools** panel (sheet on mobile, popover on desktop): demo clock date-time input, **+1시간**, **+1일**, **데모 데이터 초기화** (Reset demo data, asks for confirmation in the panel itself). The clock no longer sits permanently in the header.

2.2 Sign-in = demo guide (UC-13)

- Title "달빛카페 데모" and one sentence: "비밀번호 없이 사람을 골라 체험해요."
- A **3분 체험** (3-minute tour) card listing three numbered steps — the order matters, so numbers are used: ① 이서연으로 대타 요청 보기 ② 최민호로 수락하기 ③ 박지영 사장님으로 승인하기. Each step has a button that signs in as that person and opens the screen where the step happens (Swaps / Home / Home). The card marks steps already done in this browser.
- Below, **모든 계정**: each account with name, role and its one-line summary (owner: "사장님 · 달빛카페 · 직원 4명"; worker: fixed shifts and weekly contract hours, e.g. "월·수 18:00–23:00 · 주 10시간").
- The chosen account is kept in `sessionStorage`.

2.3 Worker Home

In this order, each section omitted when empty:

1. Greeting "서연님, 안녕하세요" and today's date.
2. **대타 요청이 왔어요** — one card per `PENDING` target: who asks, the shift ("9월 30일(수) 18:00–23:00"), reason, deadline countdown ("내일 21:00까지 답해 주세요"), the effect on me ("수락하면 이번 주 16시간 → 21시간 · 주휴수당 그대로"), and two

buttons **수락하기** (primary) / **거절** (text). Accept is one tap (NFR-01). After accepting: result toast "수요일 근무를 맡았어요. 사장님 승인을 기다려요." If someone else was faster: "민호님이 먼저 수락했어요" and the card closes.

3. **내가 보낸 요청** — tracker cards for the worker's open requests with who has answered so far and a text button **요청 취소**.
4. **다음 근무** — a large card with date, time, D-day and the button **이 근무 대타 구하기** (starts the flow with this shift preselected).
5. **이번 주** — scheduled hours, contract hours, and holiday allowance in words and money: eligible "주휴수당 받아요 · 약 33,024원" or not "계약이 주 15시간 미만이라 주휴수당이 없어요 (주 10시간)", with a bar showing contract hours against the 15-hour line.
6. **기록할 근무** — past own shifts without attendance, each with **기록하기**.

2.4 Request substitute flow (UC-04, UC-12)

- Step 1 "어떤 근무를 바꿀까요?": own future SCHEDULED shifts as selectable cards; a shift that already has an open request is shown disabled with "이미 대타를 구하고 있어요".
- Step 2 "사유와 마감을 정해 주세요": reason chips 시험 / 병원 / 가족 행사 / 개인 사정 / 직접 입력 (optional); deadline chips "24시간 뒤" / "근무 2시간 전" / "직접 선택", default = the earlier of now + 24 h and shift start (BR-03); the chosen deadline is shown as a sentence.
- Step 3 "이렇게 보낼게요": summary of shift, reason and deadline, and **받는 사람** — the eligible candidates computed now (BR-01) with names, or the warning "지금 가능한 사람이 없어요. 보내면 사장님께 알려 드려요." (FR-08). Primary **대타 요청 보내기**.
- Result screen: the tracker at step 1, the recipients, and **홈으로**.
- **대타** tab: sections **받은 요청** (same cards as Home), **보낸 요청** (tracker cards with cancel), **지난 요청** (collapsed history with end states). Primary action of the tab: **대타 구하기**.

2.5 Owner Home

1. Heading "오늘 처리할 일 N개" (or "지금 처리할 일이 없어요" with the week's summary).
2. **승인 기다리는 대타** — a decision card per ACCEPTED request: the sentence "서연님의 9월 30일(수) 18:00-23:00 근무를 민호님이 대신해요", then one impact line per person (scheduled hours before → after this week, holiday allowance status and amount before → after; a changed eligibility is highlighted in amber with the words "주휴수당이 바뀌어요"), then **승인하기** (primary) / **거절** (text). A **자세히** toggle shows the full FR-11 before/after table.
3. **답을 기다리는 요청** — open REQUESTED requests with deadline countdown and each candidate's response.
4. **확인할 출근 기록** — recorded but unconfirmed attendance with **확인** / **결근 처리**.
5. **급여 경고** — unacknowledged minimum-wage warnings of the current draft, linking to Pay.
6. **오늘 근무** — who works today and when; **이번 주** — shifts, hours and the estimated holiday-allowance total.

2.6 Schedule (UC-03)

- ≥ 900 px: the time grid of v3 (day columns Mon-Sun, hour axis from earliest start to latest end rounded to whole hours, 44 px per hour, hourly lines, today marked, blocks positioned by time, overlapping shifts side by side). < 900 px: day-by-day list with today first expanded.
- Tapping a shift opens a detail sheet (mobile) or side panel (desktop) with its details, its history ("서연 → 민호 · 9/28 승인") and only the actions relevant to the viewer: the worker's own future shift → **대타 구하기**; owner → **시간 바꾸기**, **담당자 바꾸기**, **삭제** (refused with the reason when it has a request or attendance, FR-05).
- Owner: when the displayed week has no shifts, the empty state's primary button is **이번 주 근무표 만들기** (FR-04); otherwise the primary button is **근무 추가**. Week navigation: previous / next / 이번 주.
- The handover and pending markers of §3 are used here and on every shift card elsewhere.

2.7 Pay (owner; UC-08, UC-09)

- Two segments: **주간 집계** and **월 급여**; period chosen with < > buttons ("2026년 9월").
- Weekly summary: one card per worker: contract hours, scheduled/actual hours, eligibility as words with the reason ("계약 주 10시간 · 15시간 미만", "9/29(화) 결근") and holiday hours/amount.
- Monthly payroll: primary **9월 급여 초안 만들기** when no draft exists; then one card (mobile) / row (desktop) per worker with base hours, base pay, holiday pay, premium, total and flags (예상치 = estimated, 수습 최저임금 90 %, 최저임금 미만). A below-minimum row shows the sentence "도윤님 시급 10,000원이 2026년 최저임금 10,320원보다 낮아요" and **확인했어요** (acknowledgeMinWage). The sticky primary **9월 급여 확정하기** is disabled with its reason until all warnings are acknowledged; confirmed months are read-only and say so.

2.8 Staff, Me, Settings

- **직원** (UC-01): worker cards (name, contract hours per week, hourly wage, contract period); tapping opens the worker's detail with the fixed-schedule editor: weekday chips + start/end time selects, one row per weekday, add/remove; save shows the resulting weekly contract hours. Primary **직원 추가**.
- **내 정보** (UC-02, UC-07): availability editor with the same weekday-chip pattern ("대타 가능한 시간") and the list of attendance records; the worker's own phone and wage are visible here only.

- **설정 (UC-10):** store name, number of regular employees with the sentence of what it changes ("5명 이상이면 연장·야간 가산수당이 붙어요"), the substitution-attendance policy as two radio cards each explaining the effect in one sentence, and the minimum-wage table.
- Notifications list: each notification as a sentence with time; actionable ones link to the card where the action is.
- Role guard: a worker never sees other workers' phone numbers or wages (NFR-11).

3. Visual design (do not deviate)

- Font: Pretendard Variable for everything (<https://cdn.jsdelivr.net/gh/orioncactus/pretendard@v1.3.9/dist/web/variable/pretendardvariable-dynamic-subset.min.css>) with `font-variant-numeric: tabular-nums` on times, hours and money. No second font. Korean: `word-break: keep-all`.
- Type scale: body 16 px / 1.6; home heading 26 px / 700 / -0.02em; card title 17 px / 700; key number 28 px / 700; caption 13 px (never below 12 px).
- Colors: `--ink #191F28`, `--ink-2 #4E5968`, `--ink-3 #8B95A1`, `--surface #F2F4F6` (page background), `--paper #FFFFFF` (cards), `--line #E5E8EB`, `--shift #2B5BD7` (primary and own shifts), `--shift-soft #EBF1FD`, `--handover #E9A23B`, `--handover-soft #FDF3E1`, `--alert #D93A30`, `--alert-soft #FDECEB`, `--ok #1F8A55`.
- Cards: white on the grey page, 16 px radius, 20 px padding (24 px ≥ 900 px), no border, no shadow; 12 px between cards, 32 px between sections.
- Buttons: height 52 px on mobile / 44 px on desktop, 12 px radius, 16 px / 600 label; primary filled `--shift`; secondary filled `#E8EBF0` with `--ink`; text buttons `--ink-2`; pressed state `scale(0.96)`; minimum touch target 44 × 44 px.
- Icons: inline SVG line icons (24 px, 1.75 px stroke, `currentColor`) for tabs, bell, clock, chevrons; no icon fonts, no emoji.
- Signature element: a handed-over shift has a 6 px amber left edge and a subtle 45° amber stripe (`repeating-linear-gradient`, the only gradient), the new worker's name in bold and the original worker struck through; a shift with an open request has a dashed amber outline and the chip "대타 구하는 중".
- Motion: sheets and step changes slide/fade in 180 ms `cubic-bezier(0.2, 0, 0, 1)`; no motion under `prefers-reduced-motion`.
- Layout: content max-width 720 px for Home, flows and lists (centred in the content area), full width up to 1200 px for the schedule grid and payroll table; at 360 px no horizontal scroll.
- Accessibility: visible focus rings, real `<button>`s, labelled inputs, status never by colour alone, the step indicator announced ("3 단계 중 2단계").

4. Done criteria

1. `npm test` passes and covers every test ID group listed in spec §11 with at least one test each, including: two acceptances of the same request where only the first succeeds; holiday eligibility at 14.99 h vs 15 h; the 40 h cap; both substitution-attendance policies; premium zero at 4 employees and non-zero at 5; probation applies only when all BR-10 conditions hold; the worked example of spec §10 (Choi Minho 3.2 h, 33,024 KRW).
2. The seed state reproduces spec §10 exactly, including the open request with two candidates, and the S7 walkthrough of §10 works (a test proves it).
3. Serving the repository root with any static server and opening `/app/` shows the sign-in screen with no console errors.
4. Walking through the sign-in tour in the browser: as Lee Seoyeon the Swaps tab shows her open request as a tracker; as Choi Minho, Home shows the request card with the effect "16시간 → 21시간" and **수락하기** accepts it in one tap; as Kang Doyun the card says Minho was faster; as the owner, Home shows the decision card for Minho (16 → 21 h, holiday allowance unchanged) and **승인하기** approves it; the schedule then shows the handover marker.
5. Every screen has exactly one filled primary button (a headless check counts them), every disabled button has a visible reason, and all touch targets are at least 44 × 44 px.

Appendix B. Prompt Log

The log of every ambiguity, review finding and change made between prompt versions, verbatim. Source file: prompts/prompt-log.md.

ShiftSwap — Prompt log

Iteration 1 — ambiguities found while building

Each bullet: what was unclear → what the implementation does → suggested wording for the next prompt version.

- **node --test tests/ fails on Node 24** (a directory argument is treated as a module path, error Cannot find module .../tests) → the script is "test": "node --test tests/*.test.js" → "script "test": "node --test tests/*.test.js"".
- **Read queries are not in the service list** (the UI needs week shifts, request lists, the approval comparison, payroll rows, ...) → services.js also exports read-only queries (getWorkplace, getSettings, getWorker, listWorkers, getFixedSchedules, getAvailability, getWeekShifts, listRequestableShifts, listMyRequests, listOwnerRequests, getApprovalPreview, listAttendance, getPayroll, unreadCount, defaultDeadline) plus a ServiceError class carrying an error code → "services.js also exports read-only query functions for the screens (not wrapped in tx) and a ServiceError(code, message, params) that the UI translates by code."
- **Perfect attendance in a week that has not finished** (BR-06 says every own shift "was WORKED") → only an ABSENT shift (or a given-away shift under policy ABSENT) breaks attendance; a still-SCHEDULED shift does not. Without this, the approval panel could never show Minhø as "eligible" before the week ends, contradicting the worked example → "In BR-06, a shift that is still SCHEDULED does not break perfect attendance; only ABSENT does (the weekly result is a projection until the week is over)."
- **Which workers get a WeeklySummary, and which weeks count for holiday pay** → recomputeWeek covers each worker with at least one shift (as current or original worker) in the week; payroll holiday pay sums only those weeks, so weeks that were never generated pay nothing → "A WeeklySummary exists only for workers who have a shift in that week."
- **No WeeklySummary rows in the seed** (§10 does not list them) → none are seeded; they are computed when the owner opens the weekly summary, approves a swap, confirms attendance or generates payroll → "The seed contains no WeeklySummary/Payroll rows; they are derived."
- **Base hours for shifts without confirmed attendance** (BR-11) → WORKED shifts use the recorded clock-in/out; SCHEDULED shifts in the month use scheduled hours and set estimated = 1; ABSENT shifts count 0 → state this explicitly in BR-11.
- **Premium pay at month level** (BR-08 mixes daily, weekly and night rules) → computed over the month's worked intervals: daily excess over 8 h per work date, weekly excess over 40 h of the remaining regular hours per ISO week (only the part of the week inside the month), plus night minutes 22:00–06:00; premium pay = premium hours × wage × 0.5, rounded down → "Weekly overtime in a payroll month uses only the days of that week inside the month."
- **The date used for BR-10 in a monthly payroll** → the first day of the month, or the contract start if later → "Evaluate BR-10 on the first day of the payroll month (or the contract start if later)."
- **"Effective hourly wage" in BR-07** → the worker's own hourlyWage (BR-09 says payroll computes with the worker's wage) → "effective hourly wage = Worker.hourlyWage."
- **Holiday pay rounding** → rounded down per week, then summed (TC-113) → "Round holiday pay down per week, then sum."
- **Attendance time format** (clockIn/clockOut are TEXT) → stored as HH:MM on the shift's date, end ≤ start meaning next day, like shift times → "Attendance.clockIn/clockOut are HH:MM on the shift date."
- **Who is notified on cancel and on expiry** → cancel: every target that was PENDING or ACCEPTED gets REQUEST_CANCELLED; expiry: requester and owner get REQUEST_EXPIRED, pending targets are closed without a message; acceptance: the other pending targets get TARGET_CLOSED; a decline notifies no one → list the recipients of each Notification.kind in §8.
- **Seed request createdAt and notification times** are not given → 2026-09-28T08:30 (before the demo clock) → give the value in §10.
- **Worker phone numbers** are not in §10 but NFR-11 needs them → fictional numbers 010-3827-1150, 010-4172-2083, 010-5290-3316, 010-6631-4427, 010-7748-5539 for ids 1–5 → list them in §10.
- **Names in the database** (§10 gives English and Korean names, the schema has one name) → English names stored (Park Jiyoung, Dalbit Café, ...); the Korean UI shows them unchanged → "Store the English names; the Korean UI does not translate names."
- **Deleting a shift that already has a request or attendance** (FR-05) → refused with an explanatory message; the owner changes time or worker instead → state it in FR-05.
- **Deleting a fixed schedule row that already generated shifts** → the row is removed and those shifts keep existing with fixedScheduleId = NULL → state it in FR-02.
- **Demo clock persistence** → stored in localStorage under shiftswap.clock.v1; "Reset demo data" also resets the clock to 2026-09-28T09:00; the language choice is stored under shiftswap.lang → name these keys in section 1.
- **Shared UI helpers** (escaping, number/date formatting, form error box, week navigation, row editor) have no module in the layout → formatting and the form error box live in ui/i18n.js, week navigation in ui/schedule.js, the weekday/start/end row editor in ui/availability.js (reused by workers.js) → "Shared UI helpers may live in i18n.js."
- **Stripe pattern vs "no gradients"** → the 45° amber stripe of the handed-over shift uses repeating-linear-gradient, the only gradient in the stylesheet → "the stripe may use repeating-linear-gradient; no other gradients."

- **Tables at 360 px** (NFR-02, no horizontal scroll) → below 720 px each table row turns into a stacked card with the column name before each value → state the pattern in section 3.

Iteration 2 — changes from v1 to v2 and result

v2 of the prompt (prompts/prompt-v2.md; v1 kept as prompts/prompt-v1.md) and the matching spec edits fold in the 21 build ambiguities of iteration 1 and the gaps found in the document review. The app was then brought to v2.

#	Source	Change in prompt / spec	Change in code	Verification
1	Build ambiguity	Prompt §1: test script node -- test tests/*.test.js	none (already so)	npm test runs
2	Build ambiguity	Prompt §1: services.js also exports read-only queries and ServiceError(code, message, params)	none (already so)	all service tests
3	Build ambiguity	BR-06: a still-SCHEDULED shift does not break perfect attendance (projection until the week ends)	none (already so)	TC-085, TC-063
4	Build ambiguity	§8 note: WeeklySummary only for workers with a shift that week; seed has no WeeklySummary/Payroll rows	none (already so)	TC-001, TC-088
5	Build ambiguity + document review (BR-11 "estimated" semantics)	BR-11: WORKED → recorded hours, SCHEDULED → scheduled hours + estimated = 1, ABSENT → 0	none (already so)	TC-114, TC-112
6	Build ambiguity	BR-08: monthly weekly overtime uses only the days of the ISO week inside the month; premium pay rounded down	none (already so)	TC-092-TC-094, TC-095
7	Build ambiguity	BR-10: evaluated on the first day of the month, or contractStart if later	none (already so)	TC-102, TC-104
8	Build ambiguity	BR-07: pay uses Worker.hourlyWage, rounded down per week, then summed	none (already so)	TC-111, TC-113
9	Build ambiguity	§8 note: Attendance clockIn/clockOut are HH:MM on the shift date	none (already so)	TC-071
10	Build ambiguity + document review (UC-12 cancel recipients)	§8 recipients table; FR-19 (cancel notifies every PENDING or ACCEPTED target); a decline notifies no one	none needed (implementation already matched); recipients now asserted explicitly	TC-133 (new), TC-052, TC-121, TC-123
11	Build ambiguity + document review (seed request createdAt)	§10: seed request createdAt 2026-09-28T08:30; fictional phone numbers; English names in DB	none (already so)	TC-001, TC-131
12	Build ambiguity	FR-05: deleting a shift with a request or attendance is refused	none (already so)	TC-01A (new)

#	Source	Change in prompt / spec	Change in code	Verification
13	Build ambiguity	FR-02: deleting a fixed-schedule row keeps its shifts (<code>fixedScheduleId = NULL</code>)	none (already so)	TC-01B (new)
14	Build ambiguity	Prompt §1: keys <code>shiftswap.clock.v1</code> , <code>shiftswap.lang</code> ; Reset also resets the clock	none (already so)	headless check (reset, reload)
15	Build ambiguity	Prompt §1: shared UI helpers (escaping, formatting, form error box) in <code>i18n.js</code>	none (already so)	review
16	Build ambiguity	Prompt §3: stripe may use <code>repeating-linear-gradient</code> , no other gradient; tables become stacked cards below 720 px	none (already so)	headless check: 360 px, no overflow on any screen
17	Document review	Schema: <code>Payroll.minWageAck</code> INTEGER NOT NULL DEFAULT 0; BR-09: acknowledgement sets it; prompt: <code>acknowledgeMinWage</code> service, Acknowledge button on the warning row, Confirm disabled until every warning row is acknowledged, "Estimated" chip	<code>db.js</code> schema column; new <code>acknowledgeMinWage(yearMonth, workerId)</code> ; <code>confirmPayroll(yearMonth)</code> now checks the persisted <code>minWageAck</code> (the v1 <code>acknowledgeWarnings</code> option was removed); payroll screen: per-row Acknowledge button → "Acknowledged" chip, Confirm disabled with a hint until all are acknowledged	TC-104, TC-105 (new), TC-115; headless: Confirm disabled → Acknowledge → enabled → survives reload → confirmed
18	Follows from #17 (schema change)	Schema change vs. saved browser copies	<code>openDatabase</code> checks <code>PRAGMA table_info(Payroll)</code> for <code>minWageAck</code> ; an older saved copy is replaced by the seed (key stays <code>shiftswap.db.v1</code> , a console warning explains it)	TC-002 (new)
19	Document review	§10: demo walkthrough S7 (ABSENT policy, Minho's Sat 10-03 12:00–18:00; Seoyeon and Hana eligible, Doyun excluded by overlap; Minho loses eligibility after approval); done criterion 2 requires a test	none needed in the app; new test	TC-08B (new)
20	Document review (§14 calendar)	NFR-07 date corrected to 2026-12-04; §14 course calendar	none (not app behaviour)	—

Decisions made while building v2:

- **Regenerating a draft resets the acknowledgement.** "Re-generating a month replaces DRAFT rows only", so the new row starts with `minWageAck = 0` and must be acknowledged again (TC-105). This rule has since been added to BR-09.
- **Old saved databases are replaced, not migrated.** Replacing keeps the schema exactly as spec §8 (an `ALTER TABLE` would append the column at the end); the cost is that demo edits made before v2 are lost, which is acceptable for demo data.

Result: `npm test` 58 tests, 58 pass, 0 fail (52 in v1, plus TC-002, TC-01A, TC-01B, TC-105, TC-08B, TC-133). Headless Chrome: 70/70 checks pass, 0 console errors. The checks cover: sign-in; the done-criterion 4 walkthrough; the payroll acknowledgement flow including reload; and every screen for both roles in English and Korean at 1440 px and 360 px with no horizontal scroll.

Iteration 3 — changes from v2 to v3 and result

Source for every row: UI review of the running v2 app (text too small, content in a narrow centred column with large empty areas, weekly board read as a list of cards rather than a schedule). All prompt changes are in §2/§3 and `spec.md` is unchanged; the only service-layer change is row 9 (from the traceability check), which added tests.

#	Source	Change in prompt	Change in code	Verification
1	UI review	Sign-in rows show name, role and a one-line summary (owner: "Owner · Dalbit Café · 4 workers"; worker: fixed shifts + weekly contract hours)	ui/login.js: summary built from getFixedSchedules and contractHours, days with the same time range grouped ("Tue, Thu 18:00–23:00, Sat 12:00–18:00 · 16 h/week")	headless: all five summaries read back as expected
2	UI review	Every page starts with the title and one sentence on what the page is for	pageHead(title, purpose, actions) helper in ui/i18n.js, used by all 11 screens; purpose.* sentences in English and Korean (attendance has one per role)	headless: every route for both roles has .page-head .purpose
3	UI review	Owner overview strip above the board: approvals waiting, open substitute requests, this week's shifts · hours, each linking to its page	ui/schedule.js overviewStrip(); counts from listOwnerRequests, hours from getWeekShifts of the demo-clock week; links to Approvals / Weekly summary	headless: three figures ("This week 9 shifts · 50 h" in the seed)
4	UI review	Weekly board as a time grid: hour axis from the earliest start to the latest end (whole hours), 44 px/hour, hourly lines, hour labels in a left gutter, today's header marked, blocks positioned and sized by time; handover/pending markers unchanged	ui/schedule.js rewritten around board() + layoutDay(): blocks get --top/--height from their minutes; overlapping shifts of one day share the width in lanes; overnight ends extend past 24:00; today's header gets a "Today" chip and underline	headless geometry: Sat 10:00–16:00 = 264 px (6 h) at the axis start, Sat 12:00–18:00 starts 88 px lower and sits beside it, Mon 18:00–23:00 at 8 h, axis 10:00–23:00, today = Mon 09-28
5	UI review	Empty states say what will appear and offer the next action as a button	emptyState(text, action) helper; board (owner: "Generate week of ..."; worker: availability or "Go to this week"), payroll ("No payroll for September 2026 yet." + "Generate draft for September 2026"), approvals (+ "See open requests"), inbox, my requests, attendance, weekly summary	manual review of each screen; unused v2 empty-text keys removed
6	UI review	Type scale: base 15 px / 1.6; page title 28 px / 800 / –0.02em; section 18 px / 700; small 13 px (min 12 px); inputs 16 px on mobile	styles.css heading and body rules; inputs 16 px below 900 px; no font size below 12 px	headless: h1 computes to 28 px / 800
7	UI review	Layout: nav 220 px; content starts right after the nav, 32 px padding, max-width 1200 px, left-aligned; gaps between groups at least twice the gaps inside a group	nav column 220 px; content no longer centred; #view is a flex column with 32 px between groups, 8–12 px inside (.group)	headless: nav width 220 px, content left edge 220 px
8	UI review (kept)	< 900 px day-by-day list; < 720 px stacked-card tables	below 900 px the grid drops its gutter and hour lines and blocks become static cards in day order; the table rule is unchanged	headless: every screen for both roles, EN/KO, 1440 and 360 px, no horizontal scroll; at 360 px blocks are position: static

#	Source	Change in prompt	Change in code	Verification
9	Traceability check (FR-01, FR-20 / NFR-11 had no automated test)	none yet. Suggested for §1: "services.js exports menuFor(role), the screens a role may open (FR-20); the router uses it for the menu and the role guard."	The role-to-menu table moved from main.js into a pure exported menuFor(role) in services.js (returns a copy); main.js uses it for the navigation and the redirect of other routes. NFR-11 visibility was already decided in the service layer (listWorkers(viewer), listAttendance(week, viewer)), so no new helper was needed for it.	TC-01C, TC-01D (FR-01 register/edit and every validation error), TC-134 (FR-20 menus), TC-135 (NFR-11 attendance rows and phone/wage)

Found and fixed during verification: the 32 px group gap was first set on .content, but the screens render inside #view, so the header, overview strip and week navigation touched; the gap now sits on #view.

Found while capturing the report screenshots (docs/img) and fixed:

- the "Substitute requested" chip was clipped inside a narrow shift block (chips inside blocks now wrap);
- the reply-deadline date-time field was too narrow for its value (form columns now have a 240 px minimum);
- the Korean "수정" button broke onto two lines in the narrow overlapping Saturday blocks (block buttons no longer wrap);
- on phones the week navigation wrapped awkwardly (the week label now takes its own line above the buttons).

Result: npm test 62 tests, 62 pass, 0 fail (58 + TC-01C, TC-01D, TC-134, TC-135 from row 9). Headless Chrome: 82/82 checks pass, 0 console errors. Screenshots of the owner board: /tmp/shiftswap-owner-board-1440.png, /tmp/shiftswap-owner-board-360.png.

Iteration 5 — independent code review and reproduction run 1

Source of every row: independent code review / reproduction run 1 (a reviewer script reproduced each defect against the v3 code). Spec clarifications: FR-04, FR-05, FR-13, BR-02, BR-10, BR-12, §8 WeeklySummary note.

#	Source	Change	Code	Test IDs
1	Independent code review / reproduction run 1 (BR-02)	The acceptor must still have no overlapping shift on the date, re-checked at acceptance and again at approval (ACCEPTOR_BUSY)	services.js: acceptorClash() + acceptorBusyError(); respondToRequest(accept) and decideRequest(approve) throw ACCEPTOR_BUSY inside the transaction, so nothing changes	TC-054, TC-055
2	Independent code review / reproduction run 1 (spec §7, FR-13)	Confirm and mark-absent require a SCHEDULED shift; WORKED/ABSENT are final (ATTENDANCE_CLOSED)	services.js: status guard in confirmAttendance and markAbsent; TC-089 now marks a SCHEDULED shift of the current week absent instead of an already WORKED one	TC-072, TC-073, TC-089 (changed)
3	Independent code review / reproduction run 1 (FR-04)	A fixed slot is skipped when that fixed schedule already produced a shift anywhere in the week, or when the worker already has an overlapping shift that day	services.js generateWeek: week-range lookup by fixedScheduleId plus an overlap check on the worker's shifts of the date	TC-01E, TC-01F
4	Independent code review / reproduction run 1 (§8 WeeklySummary)	Recomputing a week deletes the rows of workers who no longer have a shift (current or original) that week, so payroll never counts stale holiday pay	services.js recomputeWeek: whole-week run deletes rows of other workers; a listed worker without shifts has the row deleted instead of upserted	TC-08C
5	Independent code review / reproduction run 1 (FR-05)	Worker, date or time of a shift with an open (REQUESTED/ACCEPTED) request cannot change (SHIFT_OPEN_REQUEST)	services.js editShift: guard before validation; ui/i18n.js error dictionary: err.SHIFT_OPEN_REQUEST (ko, en)	TC-01G

#	Source	Change	Code	Test IDs
6	Independent code review / reproduction run 1 (BR-12)	Accept/decline and approve/reject expire an overdue request themselves (deadline $\leq$ now or shift start $\leq$ now) and refuse with REQUEST_CLOSED	services.js: isOverdue(), expireRequest() shared with expireOverdue; the expiry is committed and REQUEST_CLOSED is thrown after the transaction (throwIfExpired) so the throw does not roll it back	TC-124, TC-125
7	Independent code review / reproduction run 1 (BR-10, BR-09)	Probation window contractStart (inclusive) to +3 months (exclusive), only with probationEnd; the 90 % floor must be exact	rules.js probationApplies already matched (boundary test added); computePayrollRow uses minimumHourly * 9 / 10 because 10320 * 0.9 is not exactly 9,288 in floating point	TC-106, TC-107
8	Follows from #1	New error code shown in the UI	ui/i18n.js error dictionary: err.ACCEPTOR_BUSY (ko, en); REQUEST_CLOSED and ATTENDANCE_CLOSED already translated	—

Result: npm test 77 tests, 77 pass, 0 fail.

Iteration 4 — changes from v3 to v4 and result

Source for every row: the UX redesign review requested by the product owner ("the UI is too thin; redesign it the way a consumer-app product team would: intuitive, user-centred, obvious which button to press"). The prompt's §2 and §3 were rewritten; the spec gained NFR-13 and the new FR-20 wording. Business rules, schema and the existing services did not change: the only service additions are read-only queries.

#	Change in prompt / spec	Change in code	Verification
1	§2.0 rule 1 and done criterion 5: one filled primary button per screen, fixed to the bottom below 900 px	Every screen puts its page-level action in pageHead({ cta }); .cta-bar is fixed above the tab bar below 900 px and inline on desktop. On Home the most urgent item's own button is the primary, and every other action is secondary (grey) or text	tools/check.mjs DC-5: 26 scenes × 2 languages × 1280 / 390 px, each with exactly 1 visible .btn-primary
2	§2.0 rule 8: a disabled button always explains why	button({ disabled, reason }) renders the reason beneath the button and links it with aria-describedby	DC-5: every disabled button has a visible reason
3	§3 touch targets $\geq 44 \times 44$ px	Buttons 52 px (mobile) / 44 px (desktop); chips, weekday chips, icon buttons and the brand link are at least 44 px	DC-5 touch-target audit (radios, checkboxes and switches inside a label are measured by their label row)
4	§2.1 navigation: Home / Schedule / Swaps / Me and Home / Schedule / Staff / Pay; bell, demo-clock chip → Demo tools, account switcher; Settings in the header menu	menuFor(role) returns the four tabs; new routesFor(role) adds the header screens (notifications; settings for the owner) for the role guard. Left navigation with line icons ≥ 900 px, 64 px tab bar below; Demo tools as a popover (desktop) / sheet (mobile) with the clock and the reset confirmation inside the panel	TC-134 (updated); DC-3, DC-4
5	§2.2 sign-in = 3-step demo tour + all accounts with one-line summaries	login.js: numbered tour card; step 1 is remembered in this browser (shiftswap.tour.v1), steps 2–3 follow the state of the seed request; the next step is the primary button; "Start over" once all three are done	DC-4: the tour drives the walkthrough and marks all 3 steps done

#	Change in prompt / spec	Change in code	Verification
6	§2.3 Worker Home; spec: <code>getWorkerHome(workerId)</code>	New service <code>getWorkerHome</code> : incoming requests with the effect of accepting, "someone else was faster", my open requests, next shift, this week's hours and holiday allowance, shifts to record. New <code>home.js</code> renders it	TC-064 (Minho 16 → 21 h, ₩33,024 unchanged; Doyun sees Minho was faster; Seoyeon's tracker); DC-4
7	§2.5 Owner Home; spec: <code>getOwnerHome()</code>	New service <code>getOwnerHome</code> : decision cards with the effect for both people (holiday pay before → after), open requests, attendance to confirm, unacknowledged pay warnings, today, this week; <code>taskCount.approvals.js</code> renders the decision card with a compact before/after comparison behind "Details"	TC-065; DC-4 (decision card shows 16 → 21 h and "33,024원 유지", Approve works)
8	§2.0 rule 3: show the consequence before committing	The shared <code>swapEffect()</code> in services (also behind <code>getApprovalPreview</code>) adds <code>holidayPay</code> before and after; <code>effectLines()</code> turns it into the sentences on the accept and approve cards; a change in eligibility is highlighted in amber	TC-064, TC-065
9	§2.4 three-step request flow with step indicator, reason/deadline chips, review with recipients, result screen	<code>requests.js</code> : <code>#/swaps/new/1...3/done</code> , draft kept in <code>sessionStorage</code> , back / close, tab bar hidden; deadline chips that cannot be used are disabled with the reason; new service <code>previewCandidates(shiftId, requesterId)</code> lists the recipients (BR-01) before sending	TC-066; scenes flow-1/2/3/done in DC-5
10	§2.0 rule 6: request status as the tracker 요청 → 수락 → 승인, end states labelled	<code>tracker(status)</code> in <code>components.js</code> , used on Home, Swaps, the result screen and history	DC-4 (Seoyeon's tracker)
11	§2.6 schedule: time grid ≥ 900 px, day list < 900 px, detail sheet with history and viewer-specific actions	<code>schedule.js</code> : shift blocks are buttons opening a side panel / bottom sheet; new service <code>getShiftDetail(shiftId)</code> gives the history and whether the shift can be deleted (FR-05, refused with its reason); owner primary "근무 추가" or "이번 주 근무표 만들기"; worker primary "대타 구하기"	DC-4 handover marker; scene schedule-detail
12	§2.7 Pay with Weekly / Monthly segments; acknowledgement; confirm disabled with its reason	<code>payroll.js</code> (route) + <code>weekly.js</code> (cards); monthly table becomes cards below 720 px; "확인했어요" per warning; "9월 급여 확정하기" disabled with "최저임금 경고 1건을 확인해야 확정할 수 있어요"	scenes pay-week, pay-month, pay-month-empty
13	§2.8 Staff (weekday-chip fixed-schedule editor with the resulting weekly hours), Me (availability, attendance, own phone/wage only), Settings (radio cards)	<code>workers.js</code> , <code>availability.js</code> (shared weekday-chip editor), <code>settings.js</code>	TC-135 (NFR-11); scenes staff-detail, me, settings
14	§2.0 rule 7: Korean 해요일 by default, complete English, human dates ("9월 30일(수) 18:00-23:00", "내일 21:00까지", D-2), no IDs in the UI	<code>i18n.js</code> rewritten: ko is the default, full en table, <code>fmtDate / fmtWhen / dday / fmtClock / fmtYMD</code> ; English plurals handled where a count can be 1	review screenshots in both languages
15	§3 visual tokens (Pretendard only, 16 px body, borderless 16 px-radius cards on #F2F4F6, 52 px buttons, inline SVG icons, 180 ms motion)	<code>styles.css</code> rewritten; JetBrains Mono removed; icons in <code>components.js</code> ; sheets/steps animate 180 ms and not under <code>prefers-reduced-motion</code>	review screenshots; DC-5 no horizontal scroll at 390 px

#	Change in prompt / spec	Change in code	Verification
16	Harness moved into the repository	tools/lib.mjs (static server + DevTools client, Chrome forced to an en-US system locale so native date inputs read the same everywhere), tools/scenes.mjs (26 named states), tools/check.mjs (DC-3/4/5), tools/screens.mjs (review and report screenshots), tools/nfr03.mjs	node tools/check.mjs: 115/115; node tools/nfr03.mjs (see docs/nfr03-measurement.md)

Ambiguities found while building v4 (what was unclear → what the implementation does → suggested wording):

- **No module for Home, and no place for shared components** in the §1 module list → added app/ui/home.js (both Homes) and app/ui/components.js (icons, buttons, page header with its primary action, tracker, sheets, toasts) → add both files to the §1 list.
- **Screens reached from the header** (notifications; settings for the owner) are not tabs, but the router must allow them → routesFor(role) = tabs + header screens; menuFor(role) stays the tabs → "routesFor(role) returns every route a role may open (its tabs plus the header screens)".
- **§1 puts "Reset demo data" in the header menu, §2.1 in Demo tools** → it lives in Demo tools, with the confirmation in the panel (§2.0 says §2 wins) → change §1 to "in Demo tools".
- **The example copy uses Korean given names ("서연님"), but spec §10 stores English names** → the Korean UI uses the given name from the stored name with 님 ("Seoyeon님"); lists show the full stored name → either store Korean display names in the seed or state that the given name is used as stored.
- **Home can list several actionable items; each has "its own action button"** → the first (most urgent) item's button is the screen's primary; the others are secondary. On Swaps the primary is "대타 구하기" (§2.4), so the accept buttons there are secondary → state this in rule 1.
- **Primary action on the sticky bar vs. on a card** → page-level actions (Swaps, flow steps, Schedule, Pay, Staff, Me, Settings, Notifications) go in the sticky bar; on Home the primary stays on its card, and a sticky "근무표 보기" appears only when nothing needs the person → state it.
- **Screens with no natural single action** → Notifications: "모두 읽음으로 표시" (disabled with "새 알림이 없어요" when there are none); Pay → Weekly: "9월 급여 보기"; Demo tools: "이 시각으로 맞추기" (the clock changes on the button, not on every input change); a confirmed month: "9월 급여 확정 완료", disabled with its reason → list these in §2.
- **"Day-by-day list with today first expanded"** → days stay in calendar order; today is expanded and the other days are collapsed with their shift count (for another week, the first day with shifts is expanded) → "days in order, today expanded, others collapsed".
- **Where step 1 of the tour counts as done** → when its button is used (remembered in localStorage shiftswap.tour.v1, cleared by the demo reset); steps 2 and 3 are read from the seed request's state → state it.
- **Reason chips** → the chosen chip is stored as its label in the current language ("시험" / "Exam"); "직접 입력" stores the typed text → state it.
- **Staff save touches two services** → registerWorker then saveFixedSchedule (two transactions); if the schedule is invalid after a new worker was created, the screen continues on that worker's page, so a second save cannot create a duplicate → acceptable; or add a combined service if one transaction is required.

Found and fixed while reviewing the screenshots (both languages, 1280 and 390 px):

- The flow at desktop width was squeezed into the 220 px navigation column (the flow hides the nav, but the grid kept it).
- A toast from the previous person stayed visible after switching accounts.
- The FR-11 details table stacked into four tall cards on phones. It is now a compact now / after comparison per person.
- The header overflowed by 4 px in English at 390 px, and the brand link was 29 px tall.
- Weekday chips were 39 px wide at 390 px.
- Fieldset legends sat on the card edge.
- The payroll warning row painted each cell separately on phones.
- The shift history read "요청 · 요청".
- English copy errors: "1 things", "Seoyeon This week", "1 shifts", "warning(s)", "Please reply tomorrow".
- Sheets opened with a focus ring on the first button (they now focus their title).

Result:

- npm test: 77 tests, 77 pass, 0 fail. That is v3's 62, plus TC-064, TC-065 and TC-066 for the new queries (TC-134 was updated), plus 12 tests added by another team member in parallel: TC-01E/F/G, 054, 055, 072, 073, 08C, 106, 107, 124 and 125.
- node tools/check.mjs: 115/115 pass with 0 console errors. That covers DC-3, the Korean walkthrough of DC-4 (10 checks), and DC-5 for 26 scenes × 2 languages × 2 widths.
- NFR-03 re-measured on v4: the slowest screen renders in 18 ms and paints in 43 ms.